MindVideo AI

JAX

من البداية إلى الإنتاج





المؤلف

م/أحمد صالح المغـز

2026

الإصدار الأول · 2026

JAX من الصفر إلى الإنتاج

الدليل الشامل في التعلّم العميق والتبديلات القابلة للتركيب: من jax.numpy إلى أنظمة الإنتاج الموزّعة

كتاب عربي شامل يشرح **JAX 0.12** كما هي: تبديلات قابلة للتركيب فوق مترجم XLA، على **Python 3.14**. من أوّل `jnp.array` إلى نموذج موزّع يعمل في الإنتاج، بأسلوب سردي استفهامي، وأمثلة محلولة، وتمارين مُجابة.

jit ∘ grad ∘ vmap ∘ shard_map

32	6	5	+150	+40
فصلاً دراسياً	أجزاء متدرّجة	ملاحق مرجعية	مثالاً برمجياً	مخططاً توضيحياً

م/أحمد صالح المغز

إعداد وتأليف — رحلة تعلّم عملية ومنهجية

الطبعة: الإصدار الأول (1.0)

بيئة العمل: JAX 0.12 · Flax 0.12 · Optax 0.2 · Python 3.14 · XLA

إِهْدَاء

إِلَى زَوْجَتِي الغَالِيَةِ... رَفِيقَةِ الدَّرْبِ،

الَّتِي كَانَتْ لِصَبْرِي سَنَداً، وَلِهَمِّي رَاوِيَةً،

وَلِهَذَا الكِتَابِ وَقْتاً لَا أَسْتَعِيدُهُ.

مَا بَيْنَ دَفَّتَيْ هَذَا الكِتَابِ مِنْ جُهُودٍ، وَمَا فِيهِ مِنْ نَقْصٍ، كِلَاهُمَا لَكِ الحَقُّ فِيهِ أَوَّلاً؛ فَبِاسْمِكِ، وَبِاسْمِ مَا وهَبْتِهِ مِنْ سَكِينَةٍ وَاتِّزَانٍ، أُهْدِيهِ إِلَيْكِ.

مُقَدِّمَةُ المُؤَلِّف

لَمْ يُكْتَبْ هَذَا الكِتَابُ لِيَكُونَ دَلِيلاً تَقْنِيّاً يَحْفَظُ وَاجِهَاتِ JAX غَيْباً، بَلْ لِيَبْنِيَ نَمُوذَجاً ذِهْنِيّاً يَجْعَلُ كُلَّ تَحْدِيثٍ لَاحِقٍ مَفْهُوماً مِنْ أَصْلِهِ. فَـ JAX لَيْسَتْ مَكْتَبَةً أُخْرَى لِلتَّعَلُّمِ العَمِيقِ؛ بَلْ فِكْرَةٌ وَاحِدَةٌ جَرِيئَةٌ: أنَّ التَّحْوِيلَاتِ القَابِلَةَ لِلتَّرْكِيبِ – `jit` وَ `grad` وَ `vmap` وَ `shard_map` – تَكْفِي لِبِنَاءِ مَا لَا تَحُدُّهُ حُدُودٌ.

كَثِيرٌ مِمَّنْ يَقْرَأُ هَذِهِ الصَّفَحَاتِ سَبَقَتْ لَهُ تَجْرِبَةٌ مَعَ إِطَارِ عَمَلٍ تَقْلِيدِيٍّ، فَيَجِدُ فِي JAX عَدَمَ حَالَةٍ وَنَقَاءً وَظِيفِيّاً يَبْدُوَانِ غَرِيبَيْنِ فِي البِدَايَةِ. لِذَلِكَ نَبْدَأُ كُلَّ فَصْلٍ بِسُؤَالٍ أَوْ مُفَارَقَةٍ، لَا بِتَعْرِيفٍ قَامُوسِيٍّ؛ لِأَنَّ السَّبَبَ – لِمَاذَا تَعْمَلُ JAX عَلَى هَذَا النَّحْوِ – هُوَ تَحْدِيداً مَا لَا تَجِدُهُ فِي التَّوْثِيقِ الرَّسْمِيِّ.

الكِتَابُ مُقَسَّمٌ إِلَى سِتَّةِ أَجْزَاءٍ مُتَدَرِّجَةٍ: مِنْ قِصَّةِ وِلَادَةِ JAX وَبِيئَتِهَا، مَرُوراً بِالنَّوَاةِ الأَرْبَعِ، وَبِنَاءِ الشَّبَكَاتِ العَصَبِيَّةِ، وَالبُنَى الحَدِيثَةِ، وَالأَدَاءِ وَالتَّوَسُّعِ، وَحَتَّى الإِنْتَاجِ وَالمَسْؤُولِيَّةِ المِهْنِيَّةِ. كُلُّ فَصْلٍ يَقِفُ عَلَى أَقْدَامِ مَنْ سَبَقَهُ، وَيُشِيرُ إِلَى مَنْ يَلِيهِ؛ فَلَا حَاجَةَ لِقَطْعٍ سِوَى المَسَارِ المُتَّصِلِ.

نَصِيحَتِي لَكَ وَأَنْتَ تَقْرَأُ: لَا تَكْتَفِ بِقِرَاءَةِ الشِّيفْرَةِ، بَلْ نَفِّذْهَا بِيَدِكَ؛ ثُمَّ اكْسِرْهَا عَمْداً لِتَرَى كَيْفَ تَتَصَرَّفُ عِنْدَ الفَشَلِ، فَالبَحْثُ عَنِ الخَطَأِ نِصْفُ التَّعَلُّمِ. وَحَيْثُمَا وَجَدْتَ خِيَاراً، اعْلَمْ أَنَّهُ وُضِعَ لِيَكُونَ لَكَ طَرِيقاً آمِناً لِلْعَوْدَةِ إِلَى الأَصْلِ.

رَغِبْتُ أَنْ يَخْرُجَ الكِتَابُ عَرَبِيّاً فَصِيحاً، أَكَادِيمِيّاً فِي أَصْلِهِ، وَسَرَدِيّاً فِي لُغَتِهِ؛ فَالمَعْرِفَةُ أَبْقَى مَا اسْتَقَرَّتْ فِي قِصَّةٍ. فَإِنْ آمَنْتَ بِهَذَا النَّهْجِ كَمَا آمَنْتُ بِهِ، فَقَدْ بَدَأْتَ الرِّحْلَةَ، وَعَلَيْكَ بَقِيَّتُهَا.

كلّ كتاب تقني يبدأ بسؤال. وسؤال هذا الكتاب ليس «كيف أكتب `jax.jit`؟» – فهذا يُجاب عنه في سطر واحد – بل: **لماذا يفكّر مجتمع كامل بهذه الطريقة؟** ما الذي يجعل باحثاً في DeepMind وباحثاً في OpenAI ومهندساً في شركة أدوية يستخدمون المكتبة نفسها، رغم أن مشكلاتهم لا تتشابه؟ الجواب ليس في الواجهة البرمجية، بل في نموذج ذهني واحد: الدالّة كائن يمكن تحويله.

في 2018 ظهرت JAX داخل Google كتجربة صغيرة على هامش مشروع `autograd`. لم تكن تهدف إلى منافسة TensorFlow – كانت تهدف إلى الإجابة عن سؤال أنيق: ماذا لو تعاملنا مع التفاضل والتجميع والتوجيه والتوزيع كـ**تحويلات (transformations)** تُطبَّق على دالّة بايثون عادية، بدل أن تكون ميزات مبنيّة داخل إطار عمل ضخم؟ بعد ثمانية أعوام، صار هذا السؤال هو الأساس الذي تُبنى عليه مكتبات Flax وEquinox وOptax، وتُدرَّب به نماذج بمليارات المعاملات على آلاف شرائح TPU.

لمن كُتب هذا الكتاب؟

كُتب لمن يعرف بايثون، ويفهم ما يكفي من الجبر الخطّي ليعرف لماذا نضرب مصفوفتين، ولمن جرّب PyTorch أو TensorFlow ويريد أن يفهم **الطبقة التي تحتها**. لا يشترط الكتاب خبرة سابقة بـJAX؛ لكنه لا يبدأ من الصفر المطلق في التعلّم العميق. من لا يعرف ما هي دالّة الخسارة أو الانتشار العكسي سيجد في الجزء الأول ما يكفيه، لكنه سيحتاج إلى صبر.

> ★ **الوعد**
>
> بعد الفصل الثاني عشر ستكون قادراً على قراءة شيفرة JAX لأيّ بحث منشور وفهم **لماذا** كُتبت بهذا الشكل. وبعد الفصل الثلاثين ستكون قادراً على نشر نموذج يعمل، لا على تشغيل دفتر ملاحظات فقط.

لماذا كتاب آخر عن JAX؟

الوثائق الرسمية ممتازة، لكنها مكتوبة لمن يعرف ما يبحث عنه. والدروس المنتشرة على الإنترنت تشرح `jax.jit` في ثلاثين سطراً ثم تنتقل إلى Flax، فتترك القارئ يكتب شيفرة تعمل ولا يفهم سبب عملها – ثم تنكسر في أوّل نموذج حقيقي. هذا الكتاب يحاول أن يفعل العكس: يبدأ من السبب، ثم يصل إلى الواجهة، ثم يشرح كيف تنكسر الواجهة ومتى.

لذلك ستجد في كل فصل أربعة عناصر لا تتغيّر: **سؤال يفتح الفصل**، و**معادلة أو نموذج ذهني** يفسّره، و**شيفرة قابلة للتشغيل**، و**تمرين يُجاب عنه**. وإن غاب أحدها، فذلك خطأ في الكتاب لا في المنهج.

ما الذي تغطّيه هذه النسخة؟

الكتاب مبنيّ على **JAX 0.12** (إصدار أكتوبر 2026) و**Python 3.14**. هذا مهمّ لأن JAX تتغيّر بسرعة، وبعض ما ستجده في دروس قديمة لم يبقَ صالحاً: `pmap` لم يعد الطريق الموصى به، و`jax.experimental.pjit` اندمج في `jax.jit`، و`jax.custom_remat` أُزيل لصالح `jax.custom_vjp.defremat`، و`jax2tf` أُهمل لصالح `jax.export`. حيثما وُجد فرق بين ما تقوله الدروس القديمة وما تعمل به النسخة الحالية، سأقول ذلك صراحةً.

المكوّن	الإصدار	ملاحظة
Python	`3.14`	JAX 0.12 يتطلّب 3.12 أو أحدث، ويدعم 3.14 و3.14t
jax / jaxlib	`0.12.0`	NumPy ≥ 2.2 إلزامي
Flax	`0.12.10`	Linen مستقرّ، وNNX هو الجيل الجديد
Optax	`0.2.8`	المحسّنات كسلسلة تحويلات
Orbax	`0.12.x`	نقاط الحفظ المتوافقة مع التقطيع

جدول 0.1 — بيئة الكتاب المرجعية

كيف تقرأ الكتاب؟

ثلاثة مسارات ممكنة، وكلّها مشروعة:

- **المسار الكامل:** من الفصل الأول إلى الثاني والثلاثين بالترتيب. مناسب لمن يبدأ من الصفر.
- **المسار العملي:** الفصول 2، 3، 6، 7، 8، 13، 14، 15، 16 — ثمّ العودة إلى ما تحتاجه.
- **المسار المرجعي:** الفهرس والملاحق. اقرأ عند الحاجة، واستخدم الملحق (ج) لبنك المعادلات.

نصيحة واحدة: **لا تقرأ فصل شيفرة دون أن تشغّل شيفرته**. JAX مكتبة تُفهم بالأصابع أكثر مما تُفهم بالعين، ورسائل خطئها — خاصة أخطاء التتبّع — لا تُنسى بعد أن تراها مرّة.

✓ نصيحة عملية

أنشئ مستودعاً واحداً لكل تمارين الكتاب، وفعّل `jax.config.update("jax_debug_nans", True)` أثناء التطوير. ستوفّر على نفسك ساعات من البحث عن `NaN` لا تعرف من أين جاء.

شكر وتنويه

لا يوجد كتاب تقني يُكتب وحده. أفكار هذا الكتاب مدينة لمجتمع JAX المفتوح: لفريق JAX في Google، ولمن كتبوا `autograd` وشرحوا فكرته، ولمن أداروا نقاشات المسألة `#jax` ليلاً. وأي خطأ في الصفحات هو خطأ المؤلّف وحده.

وأخيراً: هذا الكتاب أداة، لا غاية. إن انتهيت منه وأنت تعرف كيف تقرأ شيفرة JAX لم تكتبها، فقد أدّى غرضه.

لكل كتاب عقدٌ غير مكتوب مع قارئه: ما الذي سيقدّمه، وكيف، وما الذي لن يقدّمه. هذا الفصل يوضّح العقد بصراحة، حتى لا تنتظر من الكتاب شيئاً لم يَعِد به – أو تفوّت شيئاً وعد به.

عناصر الفصل الثابتة

كل فصل مبنيّ على خمسة أنواع من العناصر. تعلّمها مرّة واحدة، وستقرأ الكتاب كلّه بسرعة.

خمسة عناصر، وخمس وظائف

الافتتاحية السردية

مشهد أو سؤال يربط الموضوع بالواقع قبل أيّ معادلة. اقرأها كاملة؛ فهي تحدّد المشكلة التي يحلّها الفصل.

المعادلة والنموذج الذهني

كل معادلة يسبقها سؤال «لماذا هذه المعادلة؟» ويتبعها شرح بالكلمات. لا معادلة بلا تفسير.

الشيفرة القابلة للتشغيل

كل كتلة شيفرة كاملة بما يكفي لتشغيلها وحدها، مع تعليقات عربية على الأسطر المهمّة.

الصناديق التوضيحية

ستّة أنواع: ملاحظة، تحذير، نصيحة، مفتاح، رؤية، خطأ شائع. وظيفتها تثبيت ما لا يجب نسيانه.

التمرين والحلّ

تمرين في نهاية كل فصل، يليه حلّ نموذجي كامل. الحلّ مطويّ في النسخة الرقمية، وظاهر في المطبوعة.

دلالة الألوان والرموز

◆ ملاحظة

توضيح جانبي مفيد، لكنه ليس ضرورياً لفهم الفقرة التي يليها.

★ مفتاح

فكرة جوهريّة يجب أن تخرج بها من الفصل. إن نسيت كل شيء آخر، فاحفظ هذه.

⚠ تحذير

سلوك قد يُنتج نتائج خاطئة بصمت – لا خطأً يوقفه المترجم. الأخطر في JAX من نوعه.

✕ خطأ شائع

نمط يقع فيه المبتدئون (وأحياناً المحترفون)، مع تفسير سببه وكيفية تجنّبه.

✓ نصيحة

ممارسة عملية توفّر وقتاً أو ذاكرة، ومجرّبة على شيفرة حقيقية.

✦ رؤية

ربط بين مفهوم في JAX وفكرة أعمّ في التعلم العميق أو في هندسة البرمجيات.

تشغيل الشيفرة

كل الشيفرة مكتوبة لـ**Python 3.14** و**JAX 0.12**. لتشغيلها:

أوامر الطرفية

```
$ python -m venv .venv
$ source .venv/bin/activate        # على Windows: .venv\Scripts\activate
$ pip install "jax[cuda13]" flax optax orbax-checkpoint
$ python -c "import jax; print(jax.__version__, jax.devices())"
```

إن لم يكن لديك GPU، احذف `[cuda13]`؛ ستعمل كل الأمثلة على المعالج، لكن أبطأ. الأمثلة التي تحتاج أكثر من جهاز موسومة صراحةً، ويمكن تشغيلها على جهاز واحد مع `XLA_FLAGS=--xla_force_host_platform_device_count=8`.

✓ مصدر الحقيقة

عند التعارض بين هذا الكتاب والوثائق الرسمية، الوثائق هي المرجع. الإصدارات تتغيّر، والكتاب يشرح الفكرة التي تبقى بعد أن تتغيّر الواجهة.

ما لا يقدّمه الكتاب

- ليس مرجعاً كاملاً لواجهة JAX – فذلك دور الوثائق، وهي آلاف الصفحات.
- ليس مقدّمة إلى التعلّم العميق من الصفر المطلق؛ يلزمك فهم أساسي للشبكات.
- لا يغطّي تدريب نماذج لغوية بمليارات المعاملات عملياً؛ يشرح المبادئ والتقطيع، لا هندسة العناقيد.
- لا يقدّم «أفضل الممارسات» كقوانين مقدّسة؛ يقدّمها كقرارات لها أسباب وثمن.

> الاصطلاحات ليست تفاصيل شكلية. حين يكتب كتابٌ «الشدّة» مرّة و«التينسور» مرّة أخرى، يظنّ القارئ أنهما شيئان. هذا الفصل يثبّت المصطلح والرمز والخط قبل أن نبدأ، حتى يبقى انتباهك للمعنى لا للشكل.

سياسة المصطلحات

أستخدم المصطلح العربي عند أوّل ظهور، وأُتبعه بالإنجليزي بين قوسين، ثم أستمرّ بالعربي أو بالإنجليزي حسب شيوعه في الممارسة. فـ«التبديل» مصطلح واضح، لكن `jit` اسم علم لا يُترجم؛ و«التقطيع» أوضح من sharding، لكن `Mesh` تبقى `Mesh`. القاعدة: **لا أُعرّب ما لا يُعرَّب في الممارسة اليومية**.

العربي	الإنجليزي	ملاحظة
تبديل	transformation	دالّة تأخذ دالّة وتُرجع دالّة: `jit`، `grad`، `vmap`
تتبّع	tracing	تشغيل الدالّة على قيم مجرّدة لبناء الرسم البياني
تجميع	compilation	تحويل الرسم إلى شيفرة جهاز عبر XLA
تقطيع	sharding	توزيع المصفوفة على عدّة أجهزة
شبكة أجهزة	mesh	تنظيم الأجهزة في محاور مسمّاة
شجرة	pytree	بنية متداخلة من قواميس وقوائم
نقاء	purity	غياب الآثار الجانبية
مفتاح	PRNG key	مفتاح عشوائية صريح يُقسَّم ولا يُستهلك
حقل استقبالي	receptive field	مساحة المدخل التي تؤثّر في نخرَج واحد

جدول 0.2 — أهمّ المصطلحات كما تُستخدم في هذا الكتاب

الأرقام

أستخدم **الأرقام الغربية** (0-9) في كل الكتاب، بما في ذلك أرقام الصفحات والفصول. السبب عملي: كل الشيفرة والأشكال والمصفوفات مكتوبة بها، والخلط بين نظامين داخل الصفحة نفسها يُربك العين. أرقام الفصول مكتوبة بالحروف في العناوين («الفصل السابع») لأنها أسماء لا قيماً.

الرموز الرياضية

الرمز	المعنى
x, y	متجه أو مصفوفة مدخل/خرج
W, b	مصفوفة أوزان ومتجه انحياز
θ	مجموعة المعاملات كلّها (pytree)
L	دالّة الخسارة
$\nabla_\theta L$	تدرّج الخسارة بالنسبة إلى المعاملات
$\odot$	ضرب عنصراً بعنصر
$@$	ضرب المصفوفات
$\mathbb{E}[\cdot]$	التوقّع الرياضي
$\sigma(\cdot)$	دالّة سيغمويد أو انحراف معياري حسب السياق

جدول 0.3 — الرموز المتّبعة

اصطلاحات الشيفرة

- الشيفرة بلون داكن، واتجاهها من اليسار إلى اليمين دائماً – حتى داخل صفحة عربية.
- التعليقات داخل الشيفرة بالعربية، والأسماء (المتغيّرات والدوال) بالإنجليزية.
- كل كتلة تحمل اسم اللغة في أعلاها: `Python` أو `أوامر الطرفية`.
- المخرجات تُعرض في صندوق فاتح أسفل الكتلة، موسوم بـ«المخرجات».
- السطر الذي يبدأ بـ `$` أمر طرفية، لا شيفرة بايثون.

✦ لماذا الاتجاه مهمّ؟

لو عُرضت الشيفرة من اليمين إلى اليسار، لانقلبت الأقواس وعلامات الإسناد، وصار `x = y + 1` شيئاً آخر. لذلك تُعزل الشيفرة باتجاه LTR مستقلّ عن اتجاه الفقرة – وهذا قرار طباعي مقصود، لا سهو.

بنية الإحالات

الأشكال مرقّمة بالفصل: «الشكل 7.2» هو الشكل الثاني في الفصل السابع. الجداول كذلك. المعادلات مرقّمة فقط حين يُشار إليها لاحقاً. الإحالات إلى الفصول مكتوبة بالحروف («الفصل الحادي عشر») لتسهيل القراءة الخطّية.

فهرس الكتاب

يضمّ الكتاب ستّة أجزاء و32 فصلاً و5 ملاحق مرجعية. الفهرس قابل للنقر في النسخة الرقمية، وأرقام الصفحات مطابقة لنسخة PDF.

الجزء الأول: الأسس — لماذا JAX، وكيف تفكّر بها

القصة، والبيئة، والمصفوفات، والأجهزة: ما تحتاج معرفته قبل أوّل تبديل

الفصل الأول القصة: لماذا وُلدت JAX؟ **18**
من autograd إلى التبديلات القابلة للتركيب — المشكلة التي حَلّتها، والثمن الذي تدفعه

الفصل الثاني بيئة عمل احترافية على Python 3.14 ..... **28**
التثبيت، والأجهزة، والتحقق، والحتمية، وهيكل المشروع

الفصل الثالث jax.numpy: المصفوفات كما لم تعرفها **36**
الأنواع، والترقية، وx64، والفروق الخفيّة عن NumPy

الفصل الرابع الأشكال والبثّ والفهرسة المتقدّمة **44**
einsum، gather/scatter، وعمليات المقاطع، وقواعد البثّ

الفصل الخامس الأجهزة والذاكرة: jax.Array **52**
أين تعيش البيانات، وكيف تنتقل، ومتى تُنسخ، ولماذا يهمّ ذلك

الجزء الثاني: التبديلات الأربع

jit وgrad وvmap وshard_map: قلب JAX ونموذجها الذهني

الفصل السادس jit: التجميع الذي يغيّر كل شيء **62**
التتبّع، والرسم البياني، وإعادة التجميع، وحدود ما يمكن تجميعه

الفصل السابع grad: التفاضل التلقائي بلا سحر **71**
العكسي والأمامي، value_and_grad، والقيود التي يجب أن تعرفها

الفصل الثامن vmap: الدُفعات بلا حلقات 79
التوجيه التلقائي، والمحاور، والتفاعل مع الحالة والتفرّع

الفصل التاسع النقاء والحالة وأشجار pytree 87
لماذا لا حالة في JAX؟ وكيف تبني نماذج بحالة نظيفة

الفصل العاشر random: العشوائية الصريحة 97
المفاتيح، split، fold_in، وفلسفة التكرار الحتمي

الفصل الحادي عشر من pmap إلى shard_map: التوازي 105
تاريخ التوازي في JAX، والطريق الحديث إلى عدّة أجهزة

الفصل الثاني عشر تركيب التبديلات: القواعد الذهبية 113
jit(grad(vmap(f))): كيف تتركّب، وأين تنكسر، وكيف تتجنّب ذلك

الجزء الثالث: بناء الشبكات العصبية

من الطبقة الأولى إلى حلقة تدريب إنتاجية: Flax Optax Orbax

الفصل الثالث عشر شبكة عصبية من الصفر بـ jax.numpy 122
MLP كامل، وتدرّج يدوي، وتدريب حقيقي بلا مكتبات عالية

الفصل الرابع عشر Flax: من Linen إلى NNX 132
فلسفة المكتبة، وأنماط النماذج، والفرق بين الجيلين

الفصل الخامس عشر Optax: المحسّنات كتحويلات 140
التدرّج، والزخم، وآدم، والجدولة، والقصّ، والتركيب

الفصل السادس عشر حلقة تدريب احترافية 148
الخطوة، والقياس، ونقاط الحفظ، وEMA، والاستئناف

الفصل السابع عشر التنظيم والتعميم 159
Dropout، weight decay، والتطبيع، Mixup، وتعزيز البيانات

الفصل الثامن عشر jaxtyping و Equinox: النمط البايثوني 168
نماذج ككائنات عادية، وأنواع للأشكال، وتحويل آمن

الجزء الرابع: البنى العصبية الحديثة

الصور، والتسلسلات، والانتباه، والمحوّلات، والنماذج التوليدية

الفصل التاسع عشر الشبكات التلافيفية في JAX 177

الالتفاف، والتجميع، والحقل الاستقبالي، وبنى ResNet

الفصل العشرون التسلسلات: RNN وLSTM وGRU عبر scan **186**
الحلقة كدالة، والتدرّج عبر الزمن، والسلاسل الطويلة

الفصل الحادي والعشرون الانتباه والمحوّلات **195**
الانتباه متعدّد الرؤوس، والترميز الموضعي، وذاكرة KV

الفصل الثاني والعشرون GPT صغير من الصفر **203**
الترميز، والكتلة، والتدريب، والتوليد، والتقطيع

الفصل الثالث والعشرون النماذج التوليدية **212**
VAE، وGAN، ونماذج الانتشار: كيف يولّد النموذج ما لم يره

الجزء الخامس: الأداء والتوسّع
XLA، والتقطيع، Pallas، والذاكرة، والدقة العددية

الفصل الرابع والعشرون XLA من الداخل **221**
HLO، والدمج، وحدود التجميع، وذاكرة الترجمة

الفصل الخامس والعشرون التقطيع: Mesh وNamedSharding **229**
GSPMD، وأنماط البيانات والمعاملات والتسلسل، والتحميل الموزّع

الفصل السادس والعشرون Pallas: نوى مخصّصة **238**
متى تكتب نواة؟ وكيف؟ ومن TPU إلى GPU

الفصل السابع والعشرون الذاكرة والأداء **245**
remat، وoffload، وdonation، والقياس، والتشخيص

الفصل الثامن والعشرون الدقة العددية والتكميم **252**
bf16، وfp8، والتكميم، والاستقرار العددي

الجزء السادس: من المختبر إلى الإنتاج
التصدير، والنشر، والاختبار، ودراسات الحالة، والمسؤولية المهنية

الفصل التاسع والعشرون التصدير والنشر **260**
jax.export، وONNX، وFastAPI، والحزمة، والمراقبة

الفصل الثلاثون الاختبار والحتمية وتشخيص الأخطاء **268**
من رسالة الخطأ إلى السبب الجذري: دليل عملي

الفصل الحادي والثلاثون دراسات حالة من الحياة العملية 277
الصحة، والتمويل، والصناعة، والتوصية، والطاقة، والروبوتات

الفصل الثاني والثلاثون الأخلاقيات والخصوصية وخارطة الطريق 288
العدالة، والخصوصية التفاضلية، والأمن، واثنا عشر شهراً

الملاحق

مراجع سريعة تُستشار أثناء العمل

الملحق A خريطة المقابلة بين JAX وPyTorch 296
كل مفهوم بمقابله، مع الفروق الجوهرية

الملحق B مسرد المصطلحات 302
عربي ↔ إنجليزي، مع تعريف موجز

الملحق C بنك المعادلات والخلاصات التشغيلية 306
كل معادلة مهمّة في مكان واحد

الملحق D التمارين الإضافية والحلول 313
تمارين من كل فصل مع حلول كاملة

الملحق E المراجع والمصادر 323
الوثائق الرسمية، والأبحاث المؤسِّسة، والكتب

الجزء الأول

الأسس — لماذا JAX، وكيف تفكّر بها

القصة، والبيئة، والمصفوفات، والأجهزة: ما تحتاج معرفته قبل أوّل تبديل

القصة: لماذا وُلدت JAX؟

من autograd إلى التبديلات القابلة للتركيب – المشكلة التي حَلّتها، والثمن الذي تدفعه

تخيّل أنك باحث في مختبر أدوية، أمامك مليون مركّب كيميائي، ولكل مركّب 2048 وصفاً رقمياً. تريد أن تختبر أثر تغيير صغير في كل وصف على احتمال الفاعلية – أي تريد **المشتقّة** بالنسبة إلى مليون متغيّر. ثم تريد أن تفعل ذلك على 64 بطاقة رسوميات في وقت واحد، وأن تكون النتيجة قابلة للتكرار بعد سنة. الآن تخيّل أن كل هذا يُكتب في ملف واحد، بدالّة واحدة، بلا إطار عمل يخفي عنك ما يحدث. هذا هو الوعد الذي أطلقت JAX نفسها لأجله. فهل أوفت به؟ ولماذا احتاجت إلى ثمانية أعوام لتُفهم فهماً صحيحاً؟

سؤالان لا يجيب عنهما إطار العمل التقليدي

حين تكتب نموذجاً في إطار عمل تقليدي، أنت لا تكتب دالّة رياضية؛ أنت تصف بناءً ثم تُشغّله. الإطار يملك الحلقة، ويملك المحسّن، ويملك الجهاز، ويملك الحالة. هذا مريح جداً في 90% من الحالات. لكنه يصبح قيداً في الحالات التي وُجدت JAX لأجلها:

1. **أريد أن أشتقّ أي شيء.** لا الخسارة فقط: مشتقّة الخرج بالنسبة إلى المدخل، ومشتقّة المشتقّة، وتدرّج دالّة تُرجع مصفوفة كاملة (Jacobian)، وتدرّج دالّة فيها حلقة زمنية.
2. **أريد أن أُجمّع أي شيء.** لا الطبقة فقط: خطوة تدريب كاملة، أو دالّة فيزيائية، أو محاكاة، أو حتى خوارزمية تحسين ليست شبكة عصبية أصلاً.

> لو كانت المكتبة تمنحك التدرّج تلقائياً، فلماذا تحتاج إلى كتابة التدرّج كتحويل؟ لأن التحويل يعني أنك تستطيع تركيبه: أن تأخذ الدالّة، وتشتقّها، ثم تُوجّهها على دُفعة، ثم تُجمّعها، ثم توزّعها — كل ذلك بلا أن يعرف أيّ من هذه الخطوات شيئاً عن الآخر.

من autograd إلى JAX: فكرة واحدة، ثلاث مرات

في 2015 نشر ثلاثة باحثين من هارفارد ورقة عن `autograd`: مكتبة صغيرة تُمكّن بايثون من التفاضل تلقائياً. الفكرة المركزية لم تكن «حساب التدرّج»، بل شيء أعمق: **التفاضل عملية على الدوال، لا على الأرقام**. إذا كان لديك $f: \mathbb{R}^n \to \mathbb{R}$، فإن `grad(f)` دالّة جديدة بنفس التوقيع. وهذا يعني أنك تستطيع كتابة:

$$g = \text{grad}(f) \rightarrow g(x) = \nabla f(x) \rightarrow \text{grad}(g)(x) = \nabla^2 f(x)$$

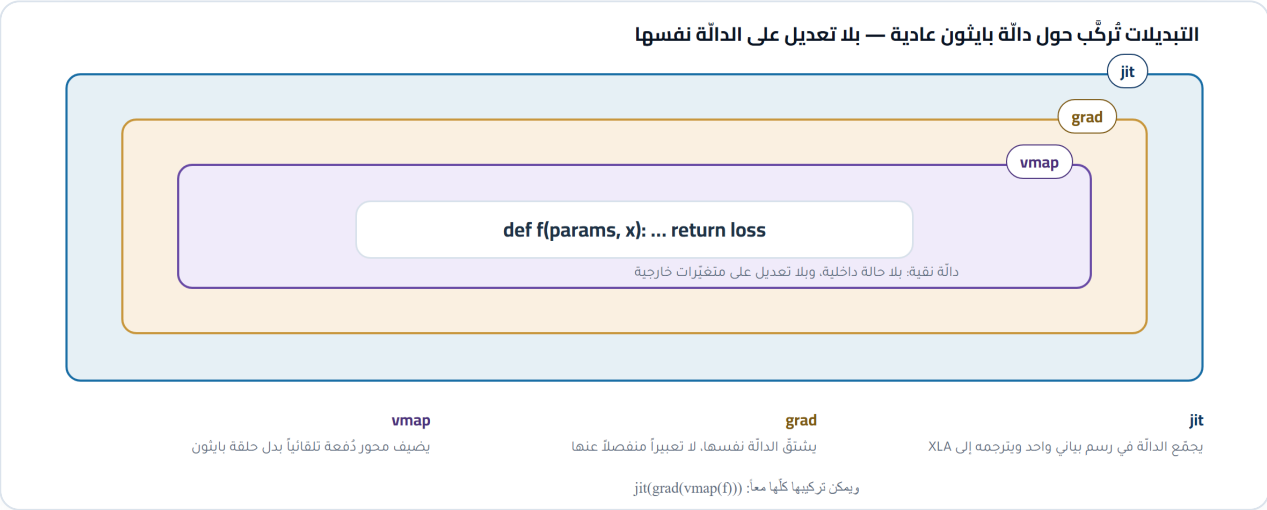
تركيب التفاضل مع نفسه يعطي المشتقّة الثانية بلا كود إضافي

JAX أخذت هذه الفكرة ووسّعتها إلى أربعة تحويلات متماثلة في الشكل. كلّها دوالّ من النوع «دالّة ← دالّة»، وكلّها قابلة للتركيب مع بعضها بلا حدود مصمّمة مسبقاً:

$$\text{jit}(f) : f \rightarrow f_{\text{compiled}}$$
$$\text{grad}(f) : f \rightarrow \nabla f$$
$$\text{vmap}(f) : f \rightarrow f_{\text{batched}}$$
$$\text{shard_map}(f) : f \rightarrow f_{\text{distributed}}$$

التحويلات الأربعة: كل واحد يأخذ دالّة ويُرجع دالّة بنفس التوقيع تقريباً

هذا التماثل هو كل شيء. لأنه يعني أن `jit(grad(vmap(f)))` ليست حيلة، بل نتيجة طبيعية لقاعدة الأنواع: كل تحويل يُرجع دالّة، والدالّة تُقبل كمدخل لأي تحويل آخر. وهذا ما لا تستطيعه الأطر التي تُخفي الحلقة: فيها التدرّج ميزة داخل المحرّك، والتوجيه ميزة أخرى، والتوزيع ثالثة، ولا تُركَّب الثلاثة إلا بما يسمح به المحرّك.



الشكل 1.1 — التبديلات تُركَّب حول دالّة بايثون عادية. الدالّة لا تعرف أنها ستُشتقّ أو تُجمَّع أو تُوجَّه؛ وكل تحويل يرى الدالّة كصندوق أسود.

ولماذا XLA؟ لأن التجميع هو الفرق بين 1× و4×

لو كانت JAX مجرّد واجهة أنيقة فوق NumPy، لما كانت تستحق كتاباً. الفرق الحقيقي أنها تبني **رسمًا بيانيّاً** من دالّتك ثم تسلّمه إلى **XLA** – مترجم كان أصلاً لتسريع TensorFlow على شرائح TPU. وXLA لا ينفّذ عملياتك واحدة واحدة؛ بل يدمجها، ويعيد ترتيبها، ويحذف ما لا يُستخدم، ويخصّص الذاكرة مسبقاً.

المثال الكلاسيكي: كتابة `relu(x @ W + b)` في NumPy تُنتج ثلاث مصفوفات وسيطة في الذاكرة. في XLA تصبح نواة واحدة تقرأ `x` و`W` و`b` مرّة، وتحسب الناتج، وتكتبه مرّة. على مصفوفة كبيرة، هذا الفرق بين أن تكون الذاكرة عنق الزجاجة أو لا تكون.

> ★ **المفتاح**
>
> JAX ليست «NumPy على GPU». JAX هي **لغة بناء برامج** تُترجم إلى XLA. حين تكتب `jnp` فأنت تكتب برنامجاً، لا سلسلة حسابات فورية – وهذا الفرق يفسّر كل سلوك غريب ستقابله لاحقاً.

ولترى الفرق عملياً، قارن النسختين. الأولى تحسب على المعالج بأربع مصفوفات وسيطة، والثانية تُجمّع الحساب كلّه في برنامج واحد:

```python
import numpy as np

x = np.random.randn(4096, 4096).astype("float32")
W = np.random.randn(4096, 4096).astype("float32")
b = np.random.randn(4096).astype("float32")

h = x @ W          # 1 مصفوفة وسيطة
h = h + b          # 2 مصفوفة وسيطة
h = np.maximum(h, 0)   # 3 مصفوفة وسيطة
print(h.shape)
```

```python
import jax
import jax.numpy as jnp

@jax.jit
def block(x, W, b):
    return jax.nn.relu(x @ W + b)

x = jax.random.normal(jax.random.key(0), (4096, 4096))
W = jax.random.normal(jax.random.key(1), (4096, 4096))
b = jnp.zeros(4096)

h = block(x, W, b)      # نواة واحدة، لا مصفوفات وسيطة
```

الفرق ليس في عدد الأسطر، بل في ما يحدث في الذاكرة: النسخة الثانية لا تُنشئ `h` ثلاث مرّات. المترجم يرى التعبير كوحدة واحدة، ويولّد شيفرة تقرأ المدخلات مرّة وتكتب الخرج مرّة. على مصفوفة 4096×4096، هذا يعني توفير مئات الميغابايتات من الحركة – والحركة، لا الحساب، هي ما يستهلك الوقت على GPU.

التبديلات الأربعة: خريطة الكتاب كلّها

يمكنك أن تقرأ الكتاب كلّه كشرح مطوّل لهذا الجدول. احفظه الآن، وستفهم كل فصل مكانه فيه.

التبديل	ماذا يأخذ	ماذا يُرجع	السؤال الذي يجيب عنه
`jax.jit`	دالّة بايثون	دالّة مجمّعة	كيف أُسرّع الحساب وأدمج العمليات؟
`jax.grad`	دالّة عددية	دالّة التدرّج	كيف أشتقّ بلا كتابة المشتقّة يدوياً؟
`jax.vmap`	دالّة لعيّنة	دالّة لدُفعة	كيف أتخلّص من حلقة `for`؟
`jax.shard_map`	دالّة لجهاز	دالّة لعدّة أجهزة	كيف أوزّع الحساب بلا تغيير المنطق؟

جدول 1.1 — التبديلات الأربعة وماذا تغيّر فعلاً

ولاحظ ما لا تجده في الجدول: لا يوجد «محسّن»، ولا «طبقة»، ولا «حلقة تدريب». هذه ليست نقصاً؛ إنها قرار تصميمي. JAX توفّر اللبنات الأولية، وتترك بناء النموذج لمكتبات مثل **Flax** و**Equinox**، والتحسين لـ**Optax**، والحفظ لـ**Orbax**. وهذا هو سبب وجود مكدّس كامل حول مكتبة تبدو صغيرة.

✓ كيف تحفظ الجدول؟

اربط كل تبديل بالفعل الذي يمنعه: `jit` يمنع البطء، و `grad` يمنع الاشتقاق اليدوي، و `vmap` يمنع حلقة `for`، و `shard_map` يمنع نسخ الشيفرة لكل جهاز. حين تنسى أيّها تستخدم، اسأل: «ما الشيء الذي أريد أن أتخلّص منه؟»

لو أردت أن تحسب مشتقّة الخسارة بالنسبة إلى المدخلات لا المعاملات — كما في الهجمات العدائية أو في رسم خرائط الأهمية — أيّ تبديل تستخدم؟ وهل يغيّر ذلك شيئاً في الشيفرة؟ ؟



الشكل 1.2 — مكدّس JAX: كل طبقة تُخفي تعقيد ما تحتها، وتكشف ما يكفي للتحكّم. JAX نفسها تقع في المنتصف: لا تعرف شيئاً عن «الطبقات»، وتعرف كل شيء عن التبديلات.

الثمن: ما تخسره حين تختار JAX

كل تصميم ثمن. ومن يبيعك JAX بلا ثمن يبيعك وهماً. الثمن هنا ثلاثة أشياء، وكلها ستظهر في الفصول القادمة:

⚠ الثمن الأول: لا حالة

لا يمكنك تعديل مصفوفة في مكانها، ولا الاحتفاظ بعدّاد داخل النموذج، ولا طباعة قيمة أثناء التجميع. كل شيء يجب أن يمرّ كمدخل ومخرج. هذا يبدو قيداً في البداية، ثم تكتشف أنه يجعل الشيفرة قابلة للاختبار والتوزيع بشكل لم تجرّبه قبل.

⚠ الثمن الثاني: التتبّع ليس بايثون

داخل `jit`، الدالّة تُشغَّل على قيم «مجرّدة» لا تحمل أرقاماً. أيّ قرار يعتمد على قيمة مدخل (`if x > 0`) سيفشل أو يجمّد على أوّل قيمة رآها. هذا أكثر خطأ يُصادفه المبتدئ، وسنفرد له فصلاً كاملاً.

جرّب هذا بنفسك. الدالّة تبدو بريئة تماماً، وهي مكتوبة ببايثون صحيحة:

Python

```python
import jax
import jax.numpy as jnp

@jax.jit
def bad(x):
    if x[0] > 0:          # قرار يعتمد على قيمة!
        return x * 2
    return x + 1

bad(jnp.array([1.0, 2.0]))
```

‹‹ المخرجات ››

```
TracerBoolConversionError: Attempted boolean conversion of
traced array. The tracer cannot be converted to a Python bool…
```

الرسالة صحيحة ودقيقة، لكنها لا تقول «لا تضع `if` هنا». تقول: «هذه القيمة ليست رقماً، بل رمز». والفرق بين الفهمين هو الفرق بين ساعة من البحث وأسبوع.

⚠ الثمن الثالث: منحنى تعلّم حقيقي

PyTorch تسمح لك بالتفكير بـ«ماذا يحدث الآن؟». JAX تطلب منك التفكير بـ«ما شكل الرسم البياني الذي أبنيه؟». هذا الانتقال يستغرق أسبوعين عادةً، ثم يصبح طبيعياً – وأسرع في التفكير من الأول.

متى تختار JAX؟ ومتى لا؟

السؤال ليس «أيّ مكتبة أفضل؟» بل «أيّ مكتبة تناسب شكل مشكلتي؟». الجدول التالي ليس حكماً نهائياً، بل ملخّص لتجربة عملية.

المعيار	JAX 0.12	PyTorch 2.x
نموذج التفكير	دوالّ نقية + تبديلات	كائنات + تنفيذ فوري
التصحيح	أصعب: أخطاء تتبّع مجرّدة	أسهل: تتبّع مكدّس عادي
الأداء على الأجهزة المتخصّصة	ممتاز (TPU أولاً، XLA)	جيد جداً (CUDA/ROCm)
التوزيع على عدّة أجهزة	مدمج في اللغة (sharding)	DDP/FSDP عبر وحدات
النظام البيئي للنماذج الجاهزة	أصغر، لكنه ينمو بسرعة	الأكبر في المجال
البحث العلمي والاشتقاق المتقدّم	الأقوى	ممكن، لكنه أقل مرونة
النشر في الإنتاج	ممكن (export/ONNX/serving)	أنضج وأكثر أدوات

جدول 1.2 — مقارنة عملية بين JAX وPyTorch (بحسب حالة أكتوبر 2026)

اختر JAX إذا...

- تبحث في أفكار جديدة تحتاج مشتقّات غير قياسية.
- تعمل على TPU أو على عدّة أنواع من العتاد.
- تحتاج توزيعاً على عشرات الأجهزة بمنطق واضح.
- تحبّ أن تفهم ما يحدث تحت الغطاء.

لا تختر JAX إذا...

- تحتاج نموذجاً جاهزاً في يومين.
- فريقك لا يعرف نموذج «الدالّة النقية».
- تعتمد على مكتبات لم تُنقل بعد إلى JAX.
- تحتاج تصحيحاً تفاعلياً سريعاً كل خمس دقائق.

خريطة الطريق

الكتاب ستّة أجزاء. الجزء الأول يبني الأرضية: البيئة، والمصفوفات، والأجهزة. الجزء الثاني هو القلب: التبديلات الأربعة وكيف تتركّب. الجزء الثالث يبني الشبكات فعلياً. الجزء الرابع يغطّي البنى الحديثة: CNN وRNN

والمحوّلات والتوليد. الجزء الخامس للأداء والتوسّع. الجزء السادس ينزل بك إلى الإنتاج والحياة العملية.

ستّة أجزاء، وثلاثون فصلاً بعد هذا

1. الأسس: لماذا JAX، والبيئة، والمصفوفات، والأجهزة
2. التبديلات: jit، grad، vmap، pytree، random، shard_map، التركيب
3. الشبكات: من الصفر، Flax، Optax، حلقة التدريب، التنظيم، Equinox
4. البنى: RNN، CNN، الانتباه، GPT، التوليد
5. الأداء: XLA، التقطيع، Pallas، الذاكرة، الدقّة
6. الإنتاج: التصدير، الاختبار، دراسات الحالة، المسؤولية

تمرين الفصل الأول

1. اكتب دالّة بايثون عادية تحسب $f(x) = x^3 + 2x$. ثم اشتقّها يدوياً. الآن اكتب نسخة تستخدم `jax.grad` وقارن القيمتين عند `x = 1.5` .
2. بأيّ معنى يكون `grad(f)` «دالّة» وليس «عدداً»؟ اذكر مثالاً يوضّح الفرق عملياً.
3. اقرأ الجدول 1.2، واكتب في ثلاثة أسطر أيّ مكتبة تناسب مشروعك الحالي ولماذا. ثم اذكر سبباً واحداً يجعل الاختيار الآخر مقنعاً أيضاً.

الحلّ النموذجي

1. المشتقّة اليدوية هي $f'(x) = 3x^2 + 2$ ، وعند `x = 1.5` تساوي `3(2.25) + 2 = 8.75` .

```python
import jax
import jax.numpy as jnp

def f(x):
    return x ** 3 + 2 * x

df = jax.grad(f)
print(f(1.5), df(1.5))          # 6.375 8.75
print(jax.grad(jax.grad(f))(1.5))  # 9.0  ← 6 المشتقّة الثانية لـx
```

2. لأن `grad(f)` تُرجع شيئاً قابلاً للاستدعاء على أيّ قيمة، لا رقماً محسوباً عند نقطة واحدة. الدليل العملي: السطر الأخير أعلاه – أخذنا تدرّج التدرّج بلا كتابة أيّ كود جديد. لو كان `grad` يُرجع عدداً، لما أمكن استدعاؤه مرّة ثانية.

3. الإجابة مفتوحة، لكن الإجابة الجيّدة تذكر شكل المشكلة لا شعبية المكتبة. مثال: «أختار PyTorch لأن فريقي يبني خدمة تعتمد على `torchvision` ونحتاج تسليماً في أسبوعين؛ والاختيار الآخر مقنع لأن نموذجنا سيبقى ثابتاً سنوات، وJAX ستعطينا عمراً أطول للشيفرة وتوزيعاً أبسط.»

خلاصة الفصل

- JAX ليست إطار عمل، بل **لغة تبديلات** فوق XLA: أربع تحويلات تُركَّب بلا حدود.
- قوّتها من التماثل: كل تحويل يأخذ دالّة ويُرجع دالّة، فتصبح التركيبة طبيعية.
- ثمنها: لا حالة، وتتبّع لا يشبه بايثون، ومنحنى تعلّم حقيقي.
- ليست بديلاً عن PyTorch في كل حالة؛ هي الأفضل حين تكون المشكلة رياضية أو موزّعة.

مصطلحات الفصل

transformation (تبديل)

دالّة تأخذ دالّة وتُرجع دالّة، وتُغيّر طريقة تنفيذها لا معناها.

XLA

مترجم جبري خطّي مُسرَّع، يدمج العمليات ويولّد شيفرة للجهاز.

pure function (دالّة نقية)

دالّة لا تعتمد إلا على مدخلاتها ولا تُغيّر شيئاً خارجها.

tracing (تتبّع)

تشغيل الدالّة على قيم مجرّدة لاستخراج الرسم البياني للحساب.

SPMD

برنامج واحد، بيانات متعدّدة: الشيفرة نفسها تعمل على كل الأجهزة.

بيئة عمل احترافية على Python 3.14

التثبيت، والأجهزة، والتحقق، والحتمية، وهيكل المشروع

في أحد المختبرات، أمضى باحث أسبوعاً كاملاً في مطاردة نتائج «تتغيّر قليلاً» بين تشغيلين متتاليين. جرّب كل شيء: بدّل معدّل التعلّم، غيّر البذرة، أعاد تثبيت المكتبة. في النهاية اكتشف أن المشكلة لم تكن في النموذج: كانت نسخة `jaxlib` مختلفة عن نسخة `jax` بواحد. أسبوع من العمل، بسبب سطر لم يقرأه أحد. هذا الفصل عن ذلك السطر: كيف تبني بيئةً لا تخونك بعد شهر، وتتحقّق منها في ثلاثين ثانية.

لماذا Python 3.14 تحديداً؟

JAX 0.12 يتطلّب Python 3.12 أو أحدث، ويدعم 3.14 و3.14t (النسخة بلا قفل GIL). هذا ليس ترفاً: الإصدارات الأحدث جلبت تحسينات في الأداء وفي نموذج الأنواع، والأهمّ أن **jaxlib** – النواة الأصلية – تُبنى وتُختبر على نطاق ضيّق من إصدارات بايثون. الخروج عن هذا النطاق يعني أنك ستُبنى من المصدر، وهنا تبدأ المشكلات.

> ⚠ **قاعدة صفر**
>
> لا تخلط أبداً بين `jax` و`jaxlib` من إصدارين مختلفين. حدّثهما معاً دائماً في أمر واحد، أو استخدم ملف قيود (`constraints.txt`) يثبّتهما.

> إذا كانت JAX تعمل على Python 3.12 وما بعده، فلماذا يصرّ كثير من الفرق على تثبيت إصدار واحد بعينه في ملف القيود، بدل السماح بالتحديث التلقائي؟

التثبيت: أربع حالات، وأمر واحد

JAX تُوزَّع بواجهة أساسية بلا أي عتاد متخصّص، ثم «إضافات» (extras) لكل نوع عتاد. اختر واحدة فقط:

العتاد	الأمر	ملاحظة
CPU فقط	`pip install -U jax`	كل الأمثلة تعمل، أبطأ

العتاد	الأمر	ملاحظة
NVIDIA (CUDA 13)	`pip install -U "jax[cuda13]"`	المسار الموصى به في 2026
NVIDIA (CUDA 12)	`pip install -U "jax[cuda12]"`	لبيئات قديمة
Google TPU	`pip install -U "jax[tpu]"`	على أجهزة Cloud TPU
AMD (ROCm)	`pip install -U "jax[rocm7-local]"`	بناء محلي؛ راجع الوثائق

جدول 2.1 — أوامر التثبيت بحسب العتاد (JAX 0.12)

مع `uv` – وهو الأسرع اليوم – يصبح الأمر:

أوامر الطرفية

```
$ uv venv --python 3.14
$ source .venv/bin/activate
$ uv pip install -U "jax[cuda13]" flax optax orbax-checkpoint
$ uv pip freeze > requirements.lock
```

✓ لماذا ملف قيود؟

`pip freeze` يثبّت كل شيء بما فيه التبعيات العابرة. بعد شهر، قد يُصدر `jaxlib` تحديثاً يكسر توافقاً دقيقاً؛ ملف القيود يمنع ذلك. اجعله جزءاً من المستودع، لا من حاسوبك فقط.

التحقّق في ثلاثين ثانية

لا تبدأ أيّ مشروع قبل أن تشغّل هذا السكربت وترى خرجه. احفظه باسم `check_env.py`.

```python
import jax
import jax.numpy as jnp
import jaxlib

print("jax     :", jax.__version__)
print("jaxlib  :", jaxlib.__version__)
print("backend :", jax.default_backend())
print("devices :", jax.devices())

# اختبار وظيفي صغير: جمع + تجميع + تدرّج
@jax.jit
def f(x):
    return jnp.sum(jnp.tanh(x) ** 2)

x = jnp.arange(8.0)
print("f(x)    :", f(x))
print("grad    :", jax.grad(f)(x))

# تحقّق من أن التجميع يعمل فعلاً على الجهاز المطلوب
print("lowered :", f.lower(x).compile().devices())
```

الخرج المتوقّع على جهاز فيه بطاقة واحدة:

‹‹ المخرجات ››

```
jax     : 0.12.0
jaxlib  : 0.12.0
backend : gpu
devices : [CudaDevice(id=0)]
f(x)    : 20.741589
grad    : [0.0000000e+00 7.6391354e-01 ...]
lowered : {cuda:0}
```

× خطأ شائع: `backend: cpu` مع بطاقة موجودة

هذا يعني أن JAX لم يجد مكتبات CUDA، لا أن البطاقة غير مدعومة. تحقّق من `nvidia-smi`، ثم من إصدار CUDA المطلوب مقابل المثبّت، ثم من أنك لم تثبّت `jax` العادي فوق نسخة `cuda`.



الشكل 2.1 — خمس خطوات من الصفر إلى أوّل تجميع ناجح. كل خطوة قابلة للتحقّق، ولا تنتقل إلى التالية قبل أن ترى نتيجتها.

الأجهزة: كيف يختار JAX؟

عند الاستيراد، يبحث JAX عن كل الأجهزة المتاحة ويرتّبها بأولوية: TPU ثم GPU ثم CPU. ويمكنك التحكّم بذلك عبر متغيّر البيئة `JAX_PLATFORMS` :

أوامر الطرفية

```
$ JAX_PLATFORMS=cpu python train.py        # للتجريب على المعالج
$ JAX_PLATFORMS=cuda,cpu python train.py   # جرّب CUDA، وارجع إلى CPU عند الفشل
```

وللتحكّم في الذاكرة – وهو أهمّ إعداد على GPU – استخدم:

أوامر الطرفية

```
$ # لا تحجز كل ذاكرة البطاقة مسبقاً (مفيد عند المشاركة)
$ export XLA_PYTHON_CLIENT_PREALLOCATE=false
$ # أو: احجز 80% فقط
$ export XLA_PYTHON_CLIENT_MEM_FRACTION=.8
```

✦ لماذا يحجز XLA كل الذاكرة؟

لأن التخصيص الديناميكي أثناء التجميع يكلّف، ولأن المترجم يخطّط للذاكرة مسبقاً. هذا ممتاز في جهاز مخصّص للتدريب، وسيّئ في دفتر ملاحظات مشترك. لهذا يوجد المتغيّران أعلاه.

لو شاركت بطاقة واحدة بين زميلين، وأطلق كلٌّ منكما عملية تدريب تحجز كل الذاكرة، فما الذي سيحدث؟ وأيّ إعداد يمنع المشكلة قبل أن تحدث؟

الحتمية: أن تحصل على النتيجة نفسها مرّتين

الحتمية في JAX ثلاث طبقات، وكل طبقة تُحلّ بطريقة مختلفة:

1. **حتمية العشوائية.** JAX لا تستخدم مولّداً عالمياً؛ تستخدم مفاتيح صريحة. نفس المفتاح ← نفس الأرقام، دائماً. (الفصل العاشر.)

2. **حتمية الترتيب.** عمليات مثل `segment_sum` قد تُنفَّذ بترتيب غير محدّد على GPU. الحل: `XLA_FLAGS=--xla_gpu_deterministic_ops`، بثمن في الأداء.

3. **حتمية الأجهزة.** تشغيل الشيفرة نفسها على GPU وTPU قد يعطي فروقاً في آخر ثلاث خانات عشرية. لا تعتمد على تطابق تامّ عبر عتاد مختلف.

```python
import jax
# فور حدوثه بدل البحث عنه لاحقاً NaN اكتشاف
jax.config.update("jax_debug_nans", True)
# (معطّل افتراضياً لأسباب أداء) تفعيل الأعداد 64-بت
jax.config.update("jax_enable_x64", True)
```

> ⚠ **تحذير بخصوص x64**
>
> تفعيل `jax_enable_x64` يضاعف استهلاك الذاكرة ويُبطئ GPU بشكل ملموس. لا تفعّله إلا إذا كنت تحتاج دقّة عددية حقيقية (محاكاة، تحليل عددي). معظم نماذج التعلّم العميق لا تحتاجه.

هيكل مشروع لا يخونك

بعد عشرات المشاريع، يتشابه الهيكل الذي يصمد: فصل الإعداد عن الشيفرة، وفصل البيانات عن النموذج، ووجود مكان واحد للحتمية.

نص

```
project/
├── pyproject.toml
├── requirements.lock
├── configs/
│   ├── base.yaml
│   └── exp001.yaml
├── src/
│   ├── __init__.py
│   ├── data.py          # التحميل والتحويل
│   ├── model.py         # تعريف الشبكة
│   ├── train.py         # حلقة التدريب
│   ├── evaluate.py
│   └── utils.py         # الحتمية، السجلات، المسارات
├── tests/
│   ├── test_shapes.py   # اختبار الأشكال قبل القيم
│   └── test_train.py    # خطوة واحدة تنتهي بلا NaN
└── artifacts/           # نقاط الحفظ (خارج git)
```

★ اختبار الأشكال قبل القيم

أكثر من نصف أخطاء JAX أخطاءُ أشكال لا أخطاءُ قيم. اكتب اختباراً يمرّر دفعة وهمية ويتحقّق من `out.shape` – قبل أن تنتظر ساعة من التدريب لتنهار في الخطوة الأخيرة.

تمرين الفصل الثاني

1. أنشئ بيئة جديدة بـ `uv`، وثبّت JAX، وشغّل سكربت التحقّق. ما قيمة `jax.default_backend()` عندك؟ ولماذا؟
2. شغّل السكربت مرّتين، وقارن `grad`. هل النتيجتان متطابقتان تماماً؟ جرّب إضافة `--xla_gpu_deterministic_ops` ولاحظ الفرق.
3. اكتب اختباراً واحداً يتحقّق من أن دالّة `f(x) = x @ W` تُرجع شكلاً صحيحاً لدفعة من 32 عيّنة بـ16 ميزة وخرج بـ10 أصناف.

الحلّ النموذجي

1. على جهاز بلا GPU ستكون القيمة `cpu`؛ ومع بطاقة مدعومة ونسخة `[cuda13]` ستكون `gpu`. المهمّ ألّا تكون `gpu` مع بطاقة موجودة — فذلك يعني مشكلة توافق لا مشكلة عتاد.

2. غالباً ستتطابق القيم على CPU، وقد تختلف في آخر خانة على GPU بسبب ترتيب الجمع في العمليات المتوازية. مع `--xla_gpu_deterministic_ops` يصبح الترتيب ثابتاً، لكن الأداء ينخفض بنسبة تتراوح عادةً بين 10% و30%.

3.

```python
import jax
import jax.numpy as jnp

def test_shape():
    W = jnp.ones((16, 10))
    f = jax.jit(lambda x, W: x @ W)
    out = f(jnp.ones((32, 16)), W)
    assert out.shape == (32, 10), out.shape

test_shape()
print("ok")
```

خلاصة الفصل

- ثبّت `jax` و `jaxlib` معاً، واحفظ ملف قيود في المستودع.
- شغّل سكربت تحقّق يطبع الإصدارات والأجهزة ويجمّع دالّة صغيرة.
- تحكّم في ذاكرة GPU بـ `XLA_PYTHON_CLIENT_*`، وفي الجهاز بـ `JAX_PLATFORMS`.
- الحتمية ثلاث طبقات: العشوائية، والترتيب، والعتاد — ولا تُطلب كلها في كل مشروع.
- اختبر الأشكال قبل القيم: فهي أرخص وأسرع.

مصطلحات الفصل

jaxlib

النواة الأصلية التي تنفّذ العمليات وتتحدّث إلى XLA والأجهزة.

backend (خلفية)

العتاد الذي ينفّذ عليه الحساب: `cpu` أو `gpu` أو `tpu`.

XLA_FLAGS

متغيّر بيئة يمرّر أعلاماً مباشرة إلى مترجم XLA.

deterministic ops

وضع يفرض ترتيباً ثابتاً للعمليات غير الترابطية، بثمن في السرعة.

extras (إضافات)

لاحقات في أمر التثبيت مثل `[cuda13]` تجرّ حزم العتاد المناسبة.

jax.numpy: المصفوفات كما لم تعرفها

الأنواع، والترقية، وx64، والفروق الخفيّة عن NumPy

جرّب هذا: اطبع نوع `jnp.array([1.0, 2.0])` وستجد `float32` لا `float64`. ثم اكتب `x[0] = 5` وستُقابَل برسالة خطأ. ثم قِس حلقة صغيرة وستجد JAX أبطأ من NumPy بأضعاف. ثلاث مفاجآت في ثلاث دقائق، وليست واحدة منها خطأً؛ كلها نتائج مباشرة لنموذج تنفيذ مختلف.

لماذا jnp وليس numpy؟

استبدل `np.` بـ `jnp.` في سكربت قديم، وسيعمل في معظم الحالات. لكن NumPy تُنفّذ `a + b` بحلقة بلغة C على المخزن فوراً، بينما كل دالّة في `jnp` **عنصر أوّلي** (primitive) في لغة وسيطة يفهمها XLA، والناتج `jax.Array` يعيش على الجهاز وقد لا تكون قيمته جاهزة لحظة عودته.

خارج `jax.jit` يترجم JAX كل عنصر وحده ويرسله (op-by-op). وداخله تُلتقط السلسلة كاملة في رسم بياني واحد ويُترجم مرّة: الفرق كألف رسالة قصيرة ورسالة واحدة طويلة.

> ★ **الواجهة عقد، والتنفيذ ليس كذلك**
>
> أن تُرجع `jnp.dot` النتيجة نفسها التي تُرجعها `np.dot` لا يعني أن التكلفة نفسها ولا الأخطاء نفسها.

إذا كانت الواجهة متطابقة إلى هذا الحدّ، فلماذا لم يكتفِ فريق JAX بترقيع NumPy من الداخل بدل بناء مكتبة جديدة؟ ؟

الأنواع والترقية: لماذا يعود float32 دائماً؟

تحسب متوسط خسارة في محاكاة عددية، فتختلف النتيجة عن مرجعك في الخانة الثامنة؛ والسبب سطر واحد: النوع افتراضياً `float32`. تثبّت JAX هذا الافتراضي لأن نصف الذاكرة وضعف عرض النطاق على المسرّعات هو ما تحتاجه الشبكات. والأعداد المكتوبة بلغة بايثون **أنواع ضعيفة** (weak types): العدد `1.0` لا يفرض عرضاً، بل يتبع النوع الآخر.

Python

```python
import jax.numpy as jnp

print(jnp.array(1.0).dtype)                    # float32 لا float64
print((jnp.ones(3, jnp.float32) + 1.0).dtype)  # float32: العدد الضعيف تبع المصفوفة
print((jnp.ones(3, jnp.float16) + 1.0).dtype)  # float16: بقي على عرضه
print(jnp.result_type(1.0, jnp.float32))       # float32
print(jnp.result_type(jnp.bfloat16, jnp.float32))  # float32
```

‹‹ المخرجات ››

```
float32
float32
float16
float32
float32
```

لاحظ السطر الثالث: العدد `1.0` لم يرفع `float16` إلى `float32` كما في NumPy القديم؛ فتحسب مع ثابت دون أن تُفسد نوع مصفوفتك.

> ✕ **خطأ شائع: «float64 هو الأصل، وfloat32 تبسيط»**
>
> في JAX العكس صحيح: `float32` هو الأصل، و`float64` ميزة تُفعَّل صراحةً. من يعود من NumPy يفسّر فروق الدقة على أنها «عدم استقرار» وهي نوعٌ لم يطلبه.

x64: القرار الذي تتّخذه مرّة واحدة

إن كنت تحسب تدرّجات عبر تفاضل عددي، أو تحلّ معادلة تفاضلية، أو تقارن بمرجع `float64`، فأنت تحتاج x64. والمفتاح سطر واحد: `jax.config.update("jax_enable_x64", True)`. والتوقيت مهمّ: الإعداد يغيّر سلوك التتبّع، فضعه قبل إنشاء أيّ مصفوفة وقبل أوّل استدعاء لـ `jax.jit`، أو استخدم `JAX_ENABLE_X64=1`؛ فتغييره في منتصف الجلسة يترك مصفوفات قديمة بنوع مختلف.

> ⚠ **الثمن مزدوج**
>
> تفعيل x64 يضاعف حجم كل موتر وعرض النطاق، ويُبطئ بعض نوى GPU بفارق ملموس لأن العتاد مُحسَّن لـ `float32` و`bfloat16`.

عدم القابلية للتغيير: لماذا لا تستطيع كتابة x[0] = 5؟

السطر `x[0] = 5` يرفع `TypeError`. ولماذا؟ لأن التعديل في المكان يجعل الدالّة تعتمد على الزمن: من عدّل المصفوفة، ومتى، وبأيّ ترتيب. وحين يعتمد الحساب على ترتيب الأحداث يستحيل على المترجم أن يعيد ترتيب العمليات أو أن يوزّعها على أجهزة.

البديل تحديث وظيفي عبر `.at`: تُنتج مصفوفة جديدة ويبقى الأصل كما هو، وتُقرأ السلسلة كجملة: خُذ `x`، عند الموضع صفر، ضع خمسة.

```python
import jax.numpy as jnp

x = jnp.zeros(5)
y = x.at[0].set(5.0).at[2].add(1.0)

print(y)   # [5. 0. 1. 0. 0.]
print(x)   # [0. 0. 0. 0. 0.]  الأصل لم يُمسّ

idx = jnp.array([1, 3])
print(x.at[idx].set(jnp.array([7.0, 9.0])))   # تحديث بعدّة مواضع دفعةً
```

> ✦ **لماذا هذا مكسب لا عبء؟**
>
> لأن المصفوفة صارت قيمة لا مكاناً؛ والمترجم يستطيع حينها أن يعيد استخدامها ويتبرّع بذاكرتها بعد آخر استخدام.

الفروق الخفية عن NumPy

الواجهة واحدة، والقواعد ليست. هذا جدول بالفروق التي تُكلّف وقتاً حقيقياً عند الانتقال.

السلوك	NumPy	JAX
التعديل في المكان	`x[0] = 5`	`x.at[0].set(5)`
معامل `out=`	مدعوم	غير موجود
العشوائية	حالة عالمية عبر `seed`	مفاتيح عبر `jax.random.key`
النوع الافتراضي	`float64` و `int64`	`float32` و `int32`

السلوك	NumPy	JAX
مصفوفات الكائنات	`dtype=object`	غير مدعومة
الأشكال	ديناميكية	ثابتة وقت التتبّع
تساوي عناصر الفرز	حسب الخوارزمية	صرّح بـ `stable=True`
التقطيع	عرض قابل للكتابة	قيمة جديدة فقط
السلاسل والقوائم	مصفوفات نصّية	ممنوعة في الحساب

جدول 3.1 — تسعة فروق بين `numpy` و `jax.numpy`

لنأخذ صفّين ونرى ما يحدث على الطرفية.

```python
import numpy as np
import jax, jax.numpy as jnp

a = np.zeros(3)
np.add(a, 1.0, out=a)          # يعمل: الكتابة في المكان

b = jnp.zeros(3)
try:
    jnp.add(b, 1.0, out=b)     # JAX في out لا معامل
except TypeError as e:
    print("out:", e)

key = jax.random.key(0)        # لا حالة عالمية، بل مفتاح صريح
print(jax.random.normal(key, (2,)))
print(jax.random.normal(key, (2,)))   # القيم نفسها تماماً
```

✓ قاعدة الترجمة الذهنية

حين تنقل شيفرة من NumPy، ابحث عن ثلاثة أنماط: إسناد إلى شريحة، ومعامل `out=`، واستدعاء لدالّة عشوائية.

الأداء: لماذا jnp أبطأ خارج jit؟

قِس حلقة من ألف عملية صغيرة على مصفوفة 64×64، وستجد JAX أبطأ من NumPy وربما بعشرة أضعاف. هي أسرع حين تُترك لتُترجم، لا حين تُستدعى عنصراً عنصراً.

$$T_{total} \approx N \cdot (t_{dispatch} + t_{kernel}) + T_{compile}$$

خارج التجميع تدفع كلفة الإرسال لكل عملية؛ وداخله تدفع كلفة الترجمة مرّة واحدة.

كلفة الإرسال ($t_{dispatch}$) هي ثمن عبور الحدّ بين بايثون والجهاز: بناء الوسائط، والتحقّق من الأنواع، وإطلاق النواة. على موتر كبير تضمحلّ أمام زمن الحساب، وعلى صغير تسيطر عليه – وهذا ما يفعله `jax.jit` : يدفعها مرّة لكل رسم بياني.

Python

```python
import time
import jax, jax.numpy as jnp

def f(a):
    return jnp.tanh(a) * a + 1.0

x = jnp.ones((64, 64))
f(x).block_until_ready()          # تسخين: أوّل نداء يترجم

def bench(fn):
    t0 = time.perf_counter()
    for _ in range(1000):
        fn(x)
    jax.block_until_ready(fn(x))  # لا تقِس قبل أن ينتهي العمل فعلاً
    return round(time.perf_counter() - t0, 3)

print("eager:", bench(f))
print("jit  :", bench(jax.jit(f)))
```

‹‹ المخرجات ››

```
eager: 0.412
jit  : 0.007
```

الفارق في عدد مرّات عبور الحدّ إلى الجهاز، لا في الحساب. وسنعود إلى شروط التجميع في الفصل السادس؛ يكفي الآن أن قياس JAX بلا `block_until_ready` قياس كاذب.

متى تستخدم jnp ومتى تستخدم lax؟

السؤال ليس أيّهما أفضل، بل أيّهما يطابق نيّتك: `jnp` طبقة رفيعة تبثّ الأشكال وترقّي الأنواع وتتصرّف كـNumPy، و`jax.lax` هي العناصر نفسها بلا مجاملة: أشكال مطابقة، وأنواع مطابقة، وتدفّق صريح.

jnp — الوضوح

حساب مصفوفي عادي، بثّ تلقائي، شيفرة تُقرأ كالرياضيات.

lax — التحكّم

حلقة بعدد غير معروف وقت التتبّع (`lax.while_loop`)، أو تفرّع يعتمد على قيمة (`lax.cond`).

القاعدة: اكتب بـ `jnp` حتى تصطدم بجدار. الجدران ثلاثة: شكل يتغيّر، أو تفرّع يعتمد على قيمة، أو حلقة لا تعرف طولها. عندها انزل إلى `lax` .

لو كتبت حلقة `while` بايثون داخل دالّة مجمّعة تنتظر حتى يصغر خطأ ما، فما الذي سيحدث للمترجم؟ ولماذا لا يستطيع الانتظار؟ ؟

تمرين الفصل الثالث

1. ما نوع ناتج `jnp.array([1, 2, 3]) / 2` ؟ ولماذا لا يعطيك `float64` ؟
2. حوّل حلقة تحدّث `x[i]` في مكانه إلى تحديث وظيفي واحد عبر `.at` .
3. قِس زمن الدالّة خارج `jit` وداخله، وفسّر الفرق.

الحلّ النموذجي

1. الناتج `float32` ، لأن المصفوفة الصحيحة تُرقّى إلى النوع العائم الافتراضي؛ وطلب `float64` بلا x64 يُختزل بتحذير.

```python
import jax, jax.numpy as jnp

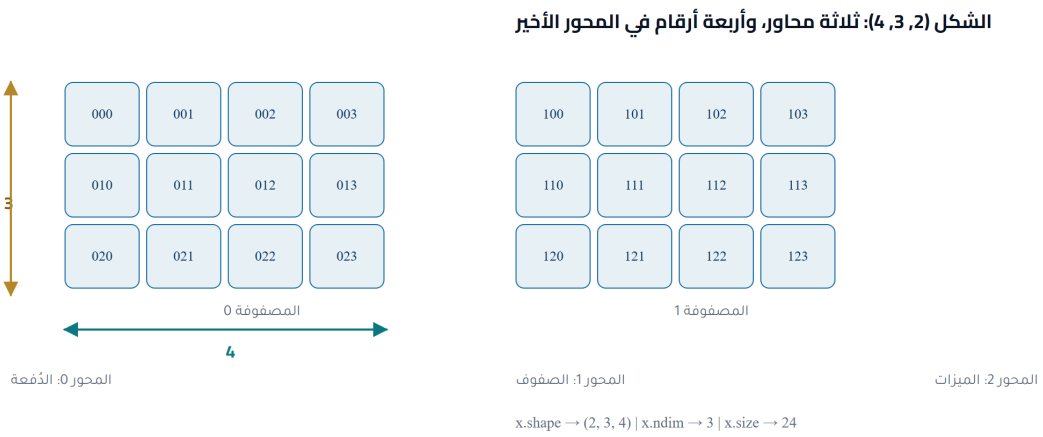
print((jnp.array([1, 2, 3]) / 2).dtype)              # float32
print(jnp.array([1, 2, 3], dtype=jnp.float64).dtype) # float32: x64 معطّل

jax.config.update("jax_enable_x64", True)
print(jnp.array([1, 2, 3], dtype=jnp.float64).dtype) # float64
```

2. سلسلة واحدة، والأصل ثابت:

```python
x = jnp.zeros(4)
x = x.at[0].set(1.0).at[1].set(2.0).at[2].set(3.0)
print(x)        # [1. 2. 3. 0.]
```

3. على CPU يكون الفارق عادةً بين 5× و50× لصالح التجميع، ويتقلّص كلّما كبر الموتر لأن t_{kernel} يطغى على $t_{dispatch}$.



الشكل 3.1 — الشكل أوّل ما يقرأه JAX من المصفوفة، قبل النوع وقبل القيم.

خلاصة الفصل

- `jnp` تشبه NumPy في الواجهة وتختلف في التنفيذ: كل دالّة عنصر أوّلي.
- الافتراضي `float32`، وأعداد بايثون أنواع ضعيفة تتبع المصفوفة.
- x64 قرار يُتّخذ مرّة واحدة قبل إنشاء المصفوفات، وثمنه ذاكرة وسرعة.
- لا تعديل في المكان، وخارج `jit` تدفع كلفة إرسال لكل عملية.

مصطلحات الفصل

primitive (عنصر أوّلي)

أصغر عملية يفهمها XLA، مثل الجمع أو الظلّ الزائدي.

weak type (نوع ضعيف)

نوع لا يفرض عرضاً، فيتبع النوع الآخر.

promotion (ترقية)

قاعدة تحدّد النوع الناتج عند اجتماع نوعين مختلفين.

op-by-op execution

تنفيذ كل عملية على حدة خارج `jit` بكلفة إرسال مستقلة.

functional update (تحديث وظيفي)

إنتاج قيمة جديدة بدل التعديل، كما في `x.at[i].set(v)`.

الأشكال والبثّ والفهرسة المتقدّمة

einsum، gather/scatter، وعمليات المقاطع، وقواعد البثّ

نموذج يعمل، وخسارة تنخفض، ونتائج معقولة. ثم تكتشف أن الدقّة صفر. لا خطأ، لا تحذير، لا انهيار – فقط رقم خاطئ. تتبّع السلسلة، فتجد أن الشيفرة طرحت مصفوفة شكلها `(32, 10)` من أخرى شكلها `(32,)`، فبثّ JAX الأولى إلى `(32, 32)` في صمت. هذا الفصل عن ذلك الصمت.

الشكل هو العقد

كل موتر يحمل وعداً: «سأعطيك دفعةً من 32 عيّنة، كلٌّ منها 10 أرقام». والدالّة التالية تبني على ذلك الوعد. والخطأ لا يقع حين يتغيّر الشكل، بل حين يتغيّر في صمت ويبقى متوافقاً بما يكفي ليمرّ.

لهذا تُقرأ الأشكال بصوت مسموع: `(32, 784)` «دفعة × ميزات»، `(8, 32, 784)` «رؤوس × دفعة × ميزات». وحين يمرّ الشكل بلا معنى في الجملة، فهو غالباً خطأ لم يظهر بعد.

> ★ **الشكل عقد يُوقَّع قبل القيم**
>
> اختبار الشكل أسرع بألف مرّة من اختبار القيمة، ويكشف ما لا تكشفه الخسارة: أن الترتيب نفسه خطأ.

لماذا لا يشتكي JAX حين يطرح `(32, 10)` من `(32,)`؟ وما الذي يجعل هذا السكوت أخطر من رسالة خطأ صريحة؟

قواعد البثّ الثلاث

البثّ ليس سحراً، بل ثلاث قواعد تُطبَّق من اليمين إلى اليسار: حاذِ الأبعاد من الطرف الأيمن، ثم قارن كل زوج: إن تساوى البُعدان مرّ، وإن كان أحدهما واحداً تمدّد، وإلّا فخطأ.

$$(3, 1, 4) \odot (2, 4) \to (3, 2, 4)$$

البُعد الأيمن 4 = 4، ثم 1 يتمدّد إلى 2، ثم 3 يبقى وحده فيُكرَّر.

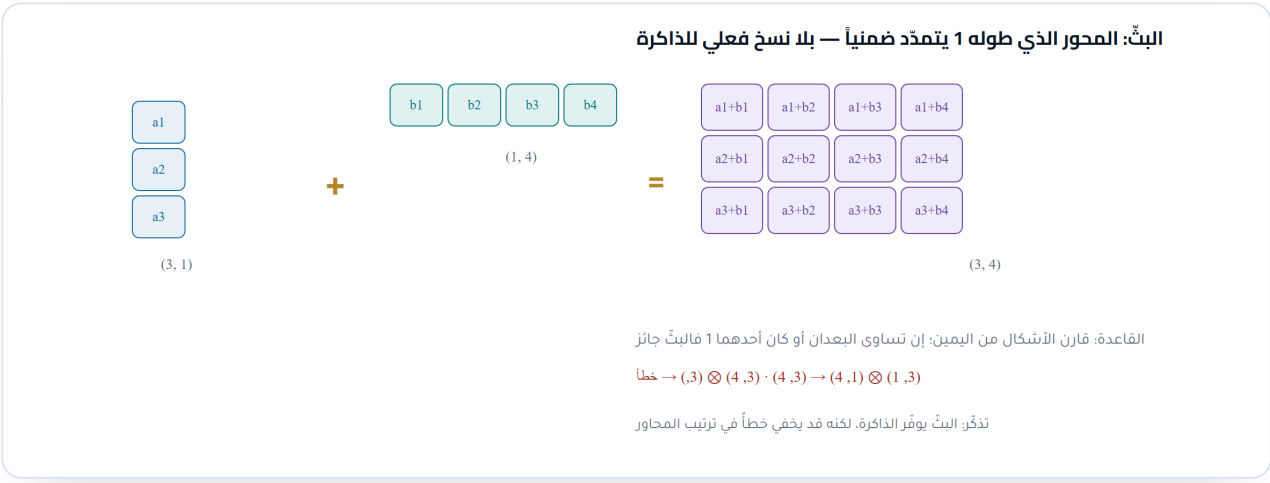
```python
import jax.numpy as jnp

a = jnp.ones((3, 1, 4))
b = jnp.ones((2, 4))
print((a + b).shape)        # (3, 2, 4)

c = jnp.ones((3, 4))
d = jnp.ones((3,))
try:
    c + d                   # 4 لا توافق :3 مقابل
except ValueError as e:
    print("broadcast:", e)
```

لاحظ الحالة الأخيرة: `(3, 4)` و`(3,)` لا يتوافقان، لأن المحاذاة من اليمين تقارن 4 بـ3. لو كان الثاني `(4,)` لمرّ.

> **⚠ البثّ الصامت هو الفخّ**
>
> حين تكون الأبعاد متوافقة بالصدفة يمرّ الحساب وينتج شكلاً أكبر ممّا توقّعت. أضف `assert` على الشكل المتوقّع بعد كل عملية غير بديهية.



الشكل 4.1 — البثّ يبدأ من اليمين: البُعد ذو القيمة واحد يتمدّد، والبُعد الغائب يُعامَل كواحد.

إعادة التشكيل والتبديل

`reshape` يعيد ترتيب القراءة الخطّية للذاكرة، و`transpose` يبدّل محاور، و`moveaxis` ينقل محوراً إلى موضع، و`expand_dims` يضيف محوراً بطول واحد. الأول يغيّر التجميع، والثلاثة الأخرى تغيّر الترتيب.

```python
import jax.numpy as jnp

x = jnp.arange(12).reshape(3, 4)
print(x.T.shape)                    # (4, 3) تبديل
print(jnp.moveaxis(x, 0, 1).shape)  # (4, 3) نقل محور
print(jnp.expand_dims(x, 0).shape)  # (1, 3, 4) إضافة محور

y = jnp.ones((2, 3, 4))
print(y.transpose(0, 2, 1).shape)   # (2, 4, 3)
print(y.reshape(2, 12).shape)       # (2, 12) نفس ترتيب الذاكرة
```

إعادة التشكيل قد تنسخ البيانات فعلاً حين لا يطابق الترتيب الجديد الترتيب الخطّي؛ أمّا التبديل فيغيّر الوصف لا المحتوى.

> **× خطأ شائع: `reshape` حيث تريد `transpose`**
>
> تنتقل من `(B, T, D)` إلى `(B, D, T)` بـ `reshape`، فيمرّ الشكل ويختلط الزمن بالميزات. الأعراض: تدريب يتقارب ببطء بلا سبب ظاهر. القاعدة: إن تغيّر معنى المحاور فاستخدم `transpose`.

einsum: لغة واحدة لكل الضربات

`jnp.einsum` تكتب المعادلة كما تكتبها على الورق: أسماء المحاور حروف، والتكرار يعني الجمع، وما يظهر يسار السهم فقط هو ما يبقى.

$$C_{ij} = \sum_k A_{ik} \cdot B_{kj} \iff \text{"ik,kj→ij"}$$

الحرف k يظهر في المدخلين ولا يظهر في الخرج، فهو محور الجمع.

```python
import jax.numpy as jnp

A, B = jnp.ones((2, 3)), jnp.ones((3, 4))
print(jnp.einsum("ik,kj->ij", A, B).shape)      # (2, 4)

X, Y = jnp.ones((5, 2, 3)), jnp.ones((5, 3, 4))
print(jnp.einsum("bik,bkj->bij", X, Y).shape)   # (5, 2, 4) دفعة كاملة

M = jnp.arange(9).reshape(3, 3)
print(jnp.einsum("ii->", M))                    # الأثر: مجموع القطر
print(jnp.einsum("ii->i", M))                   # القطر نفسه
```

القوة هنا أن المعادلة نفسها تصف الحالة المفردة والمجمّعة: أضف حرفاً في الأول وكرّره في الثاني. وسنستخدم هذا في فصل الانتباه، حيث تُكتب الدرجات كلها بـ `einsum` واحدة.

segmentg scatterg gather

حين تحتاج عناصر بأعيانها لا شرائح متّصلة، استخدم `jnp.take` للجمع و `x.at[idx].set` أو `.add` للتوزيع. وحين تكون لديك مجموعات بأطوال مختلفة فالحلّ `segment_sum` و `segment_max`: تُعطى لكل عنصر رقم مجموعته، فتُجمع المجموعات دفعةً واحدة.

```python
import jax, jax.numpy as jnp

x = jnp.array([10.0, 20.0, 30.0, 40.0])
idx = jnp.array([2, 0, 2])
print(jnp.take(x, idx))          # [30. 10. 30.]
print(x.at[idx].add(1.0))        # [11. 20. 32. 40.] تراكم على المتكرّر

# بيانات غير مستوية: ثلاث مجموعات بأطوال مختلفة
vals = jnp.array([1.0, 2.0, 3.0, 4.0, 5.0])
seg = jnp.array([0, 0, 1, 2, 2])
print(jax.ops.segment_sum(vals, seg, num_segments=3))   # [3. 3. 9.]
```

✓ لماذا المقاطع لا الحلقات؟

لأن الشكل يبقى ثابتاً: خمسة عناصر، ثلاثة مقاطع، دائماً. الحلقات على بيانات غير مستوية تُنتج أشكالاً متغيّرة، وأشكالاً متغيّرة لا تُترجم.

الفهرسة المنطقية والمتقدّمة

`x[mask]` يعمل خارج `jit` ويُعيد شكلاً يعتمد على البيانات. وداخل `jit` يستحيل ذلك: الشكل يجب أن يُعرف قبل التشغيل. البديل `jnp.where(mask, x, 0.0)` : يُبقي الشكل ويصفّر ما لا يطابق.

```python
import jax, jax.numpy as jnp

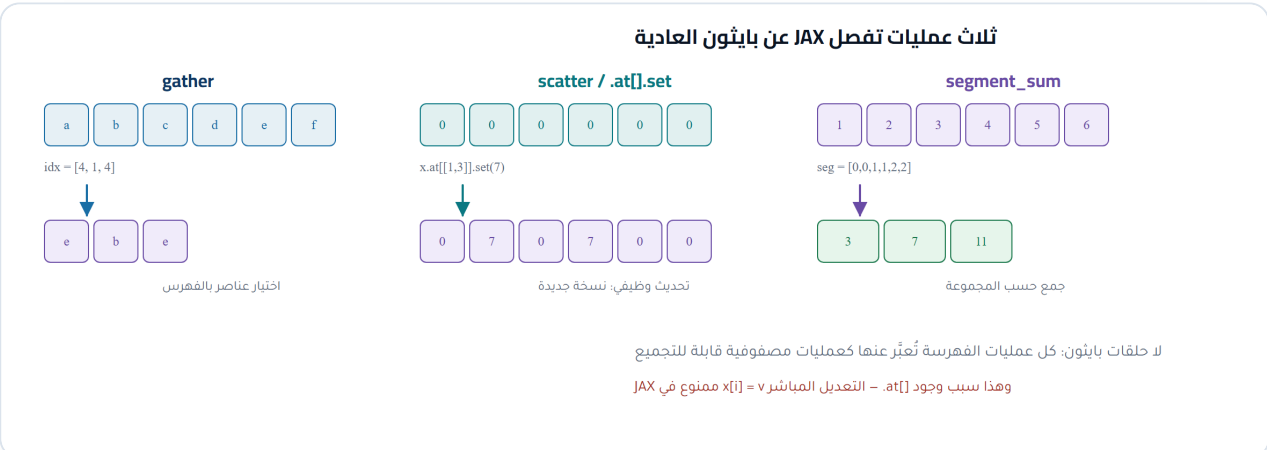
x = jnp.array([-1.0, 2.0, -3.0, 4.0])
mask = x > 0

print(x[mask])                    # [2. 4.] خارج jit: شكل ديناميكي
print(jnp.where(mask, x, 0.0))    # [0. 2. 0. 4.] شكل ثابت
print(jnp.sum(jnp.where(mask, x, 0.0)))   # 6.0 مجموع الموجب فقط

f = jax.jit(lambda v: jnp.sum(jnp.where(v > 0, v, 0.0)))
print(f(x))                       # يعمل: الشكل معروف
```

والقيد نفسه ينطبق على الشرائح: `x[i:j]` بمتغيّرين متتبَّعين يرفع خطأ، لأن الحدّ ليس رقماً معروفاً وقت التجميع. الحلّ `lax.dynamic_slice` مع حجم ثابت.

إذا كان `x[mask]` يعمل خارج التجميع ويفشل داخله، فهل المشكلة في الفهرسة أم في وعد الشكل الذي قطعه المترجم على نفسه؟



الشكل 4.2 — ثلاثة أنماط للفهرسة: شريحة ثابتة، وقناع منطقي، وفهرس متقدّم. الأولان يختلفان في الشكل الناتج، والثالث يحتاج انتباهًا خاصًا.

أخطاء الأشكال الشائعة

أربعة أخطاء تغطّي معظم ما ستراه. اقرأ الرسالة إلى آخرها: JAX يذكر الشكلين معاً في الغالب، وهذا نصف التشخيص.

الرسالة	السبب	العلاج
`Incompatible shapes for broadcasting`	بُعدان غير متساويين ولا أحدهما واحد	أضف محوراً أو بدّل المحاور
`dot_general requires contracting dimensions to have the same shape`	ضرب مصفوفتين غير متوافقتين داخلياً	طابق البُعد الداخلي
`NonConcreteBooleanIndexError`	قناع منطقي داخل `jit`	استبدله بـ `jnp.where`
`ConcretizationTypeError`	حدّ شريحة متتبَّع لا رقم ثابت	استخدم `lax.dynamic_slice`

جدول 4.1 — أربع رسائل خطأ وأسبابها

لاحظ أن الخطأين الأخيرين ليسا خطأي شكل بالمعنى الحسابي، بل خطأ وعد: طلبت من المترجم شيئاً لا يمكن أن يعرفه قبل التشغيل.

متى تعيد التشكيل ومتى تعيد التصميم؟

إعادة التشكيل مجّانية تقريباً، ولهذا تُستعمل كثيراً. لكن كثرة الاستعمال إشارة: إن كنت تعيد التشكيل في كل دالّة فأنت تُصلح عقداً لم تُكتب جيداً. الاختيار الحقيقي هو ترتيب المحاور: `(B, T, D)` أم `(B, D, T)`. اختر ترتيباً واحداً، واكتبه في تعليق أعلى الوحدة، واجعل كل دالّة تُصرّح بما تأخذ وما تُعطي.

> ✦ **التبديل رخيص، والالتباس غالٍ**
>
> ثمن `transpose` زهيد لأن XLA يدمجه في العملية التالية؛ وثمن ترتيب محاور غير مكتوب أسبوع من التصحيح.

تمرين الفصل الرابع

1. لديك `logits` بشكل `(32, 10)` و `labels` بشكل `(32,)`. اكتب الطرح الصحيح الذي يُنتج `(32,)` لا `(32, 32)`.
2. احسب بمصفوفة واحدة المسافات بين كل نقطتين في مصفوفة `(N, D)` باستخدام البثّ.
3. أعد كتابة `x[x > 0]` بصيغة تعمل داخل `jit`.

الحلّ النموذجي

1. الخطأ ناتج عن محاذاة `(32,)` من اليمين، فتُقارن 10 بـ32. الحلّ إضافة محور صريح:

```python
import jax.numpy as jnp

logits = jnp.ones((32, 10))
labels = jnp.arange(32)

print(logits[jnp.arange(32), labels].shape)          # (32,)
print((logits - jnp.expand_dims(labels, 1)).shape)   # (32, 10)
```

2. الفرق بين كل زوج يُبنى بالبثّ بعد إضافة محور:

```python
X = jnp.ones((5, 3))
diff = jnp.expand_dims(X, 1) - jnp.expand_dims(X, 0)   # (5, 5, 3)
dist = jnp.sqrt(jnp.sum(diff ** 2, axis=-1))           # (5, 5)
print(dist.shape)
```

3. `where` يحفظ الشكل، و `sum` يُعيد العدد نفسه:

```python
import jax

f = jax.jit(lambda v: jnp.sum(jnp.where(v > 0, v, 0.0)))
print(f(jnp.array([-1.0, 2.0, -3.0, 4.0])))   # 6.0
```

خلاصة الفصل

- اقرأ الأشكال بصوت مسموع: فالشكل عقد بين الدوالّ لا تفصيلاً داخلياً.
- البثّ من اليمين: تساوٍ أو واحد، وإلّا خطأ – والخطر في الحالة التي تمرّ بالصدفة.
- `reshape` يعيد التجميع، و `transpose` يبدّل المحاور.
- `einsum` تكتب بها الضربات كلها، و `where` يحفظ الشكل.

مصطلحات الفصل

broadcasting (بثّ)

تمدّد الأبعاد ذات القيمة واحد تلقائياً لتتوافق الأشكال.

reshape (إعادة تشكيل)

تغيير توزيع العناصر على المحاور مع الحفاظ على ترتيبها الخطّي.

einsum

صياغة الضربات والجموع بمعادلة حروف تُسمّي المحاور.

segment reduction

جمع أو تعظيم عناصر مجموعات مرقّمة، بشكل ثابت وبلا حلقات.

dynamic shape (شكل ديناميكي)

شكل يعتمد على القيم وقت التشغيل، فلا يُقبل داخل `jit` .

الأجهزة والذاكرة: jax.Array

أين تعيش البيانات، وكيف تنتقل، ومتى تُنسخ، ولماذا يهمّ ذلك

يشتري الفريق بطاقةً بذاكرة 80 غيغابايت، ثم يرى رسالة نفاد ذاكرة عند دفعة صغيرة. يقلّل حجم الدفعة، فتنجح مرةً وتفشل أخرى. يغيّر الإصدار، فلا يتغيّر شيء. المشكلة في الحقيقة ليست في الحجم: هي في مكان البيانات ووقت وصولها. هذا الفصل عن الموتر بوصفه شيئاً له عنوان، لا قيمة فقط: أين يعيش، وكيف ينتقل، ومن يدفع الثمن.

jax.Array: ما الذي تحمله فعلاً

الموتر في JAX ليس مصفوفة أرقام في ذاكرة بايثون. هو وصف لكتلة على جهاز، مع عقد يحدّد كيف تُقرأ: النوع، والشكل، والتقطيع، والأجهزة التي تحملها. والقيمة قد لا تكون وصلت بعد.

لهذا تحمل `jax.Array` خصائص لا تجدها في `numpy.ndarray`: `sharding` يصف كيف توزّع الكتلة، و `devices()` يعدّد الأجهزة التي تحملها، و `aval` يعطي الشكل والنوع اللذين يعرفهما المترجم قبل التشغيل.

```python
import jax, jax.numpy as jnp

x = jnp.ones((4, 8), dtype=jnp.bfloat16)
print(x.shape, x.dtype, x.nbytes)   # (4, 8) bfloat16 64
print(x.device)                     # cuda:0
print(x.sharding)                   # SingleDeviceSharding
print(x.aval)                       # ShapedArray(bfloat16[4,8], weak_type=False)
print(jax.default_backend(), jax.devices())
```

★ **الشكل والنوع عقد، والذاكرة موقع**

`aval` هو ما يعرفه المترجم بلا نظر إلى القيم، و `sharding` ما يعرفه عن توزيع الكتلة. كل ما يخرج عن هذين الوصفين يُجبر على إعادة تجميع أو نسخ.

كيف يختار JAX الجهاز؟

عند الاستيراد يعدّد JAX الأجهزة المتاحة ويرتّبها بأولوية: TPU ثم GPU ثم CPU. `jax.default_backend()` يخبرك بما اختاره، و `jax.devices()` يخبرك بما رآه. وإن أردت التحكّم، فالمتغيّر `JAX_PLATFORMS` يُقرأ قبل التهيئة.

أوامر الطرفية

```
$ JAX_PLATFORMS=cpu python train.py        # للتجريب على المعالج
$ JAX_PLATFORMS=cuda,cpu python train.py   # جرّب CUDA، وارجع إلى CPU عند الفشل
```

وللنقل الصريح دالّتان: `jax.device_put(x, device)` تنقل إلى جهاز محدّد، و `jax.device_get(x)` تُعيد نسخة إلى المضيف. وكلتاهما لا تشتكي حين يكون الموتر على الجهاز المطلوب أصلاً – وهذا ما يجعل التكلفة غير مرئية في الاختبارات.

النقل بين الأجهزة: الثمن الخفي

ثمن النقل ليس في عدد البايتات فقط، بل في الانتظار: الناقل أبطأ من ذاكرة البطاقة بمرتبة كاملة، والنقل يوقف خطّ الأنابيب.

$$t_{\text{copy}} \approx \text{bytes} \,/\, \text{bandwidth}_{\text{bus}}$$

نسخ 64 ميغابايت على ناقل بطيء قد يكلّف أكثر من الحساب نفسه.

القاعدة: أنشئ البيانات على الجهاز الذي سيحسب عليها. `jnp.ones` و `jax.random.normal` تُنتج على الجهاز مباشرة، بينما `np.ones` تُنتج على المضيف ثم تُنسخ.

Python

```python
import numpy as np
import jax, jax.numpy as jnp

host = np.ones((4096, 4096), dtype=np.float32)   # 64 MiB على المضيف
x = jax.device_put(host, jax.devices()[0])       # نسخة واحدة صريحة

y = jnp.ones((4096, 4096))                       # تُنشأ على الجهاز
print(x.devices(), y.devices())
print(jax.device_get(y).shape)                   # عودة إلى المضيف
```

⚠ النسخ الخفيّ في حلقة التدريب

أخطر ما في النقل أنه يحدث ضمنياً: تحويل مصفوفة NumPy إلى `jnp` داخل الخطوة، أو `float(loss)` لطبعه. والأخير يوقف الحلقة أيضاً لأنه يجبر على الانتظار.

التنفيذ غير المتزامن

حين تستدعي دالّة مجمّعة، لا تنتظر حتى ينتهي الحساب؛ تُرسل العمل وتعود. القيمة تُملأ لاحقاً. ولهذا فإن قياس الزمن بلا انتظار يقيس سرعة بايثون في الإرسال، لا سرعة الجهاز في الحساب.

Python

```
import time
import jax, jax.numpy as jnp

x = jnp.ones((4096, 4096))
f = jax.jit(lambda a: a @ a)
f(x).block_until_ready()          # تسخين قبل أي قياس

t0 = time.perf_counter()
f(x)                              # يعود فوراً
print("بلا انتظار:", round(time.perf_counter() - t0, 4))

t0 = time.perf_counter()
f(x).block_until_ready()          # هنا يحدث الحساب فعلاً
print("مع انتظار :", round(time.perf_counter() - t0, 4))
```

‹‹ المخرجات ››

```
بلا انتظار: 0.0002
مع انتظار : 0.1180
```

✕ خطأ شائع: تحسين ما لا تقيسه

ترى رقماً صغيراً فتظنّ العملية سريعة، فتبني عليها قراراً معمارياً. أضف `block_until_ready` إلى كل قياس، وإلى آخر عملية في الحلقة كي لا تتراكم الطلبات في الطابور.

الذاكرة: الحجز والتبرّع

XLA يحجز ذاكرة البطاقة مسبقاً لأن التخصيص أثناء التشغيل يكلّف ويُجزّئ الذاكرة: ممتاز في جهاز مخصّص، وسيّئ في مشترك. والمتغيّران `XLA_PYTHON_CLIENT_PREALLOCATE` و

`XLA_PYTHON_CLIENT_MEM_FRACTION` يضبطان ذلك قبل التهيئة.

المفتاح	الأثر	متى تستخدمه
`XLA_PYTHON_CLIENT_PREALLOCATE=false`	لا يحجز كل الذاكرة مسبقاً	بطاقة مشتركة بين عمليات
`XLA_PYTHON_CLIENT_MEM_FRACTION=.7`	يحدّ الحجز بنسبة	تريد ترك هامش لعمليات أخرى
`XLA_PYTHON_CLIENT_ALLOCATOR=platform`	مخصّص النظام بدل حجز XLA	تشخيص أو ذاكرة موحّدة
`donate_argnums`	يتبرّع بمخزن الوسيط بعد الاستخدام	حلقة تدريب لا تعيد استخدام المدخل

جدول 5.1 — مفاتيح التحكّم في ذاكرة الجهاز

التبرّع هو الأكثر أثراً والأقلّ فهماً. حين تُعلن `donate_argnums=0` تعدك الدالّة ألّا تحتاج المخزن الأصلي بعد الاستدعاء، فيعيد XLA استخدامه للخرج. والشرط واحد: لا تُبقِ مرجعاً إلى القيمة القديمة.

```python
import jax, jax.numpy as jnp

@jax.jit(donate_argnums=0)
def step(w, g):
    return w - 0.01 * g          # الخرج يأخذ مخزن w

w = jnp.ones(1_000_000)
w = step(w, jnp.ones(1_000_000)) # إعادة الإسناد آمنة
print(w.shape)
```

✓ متى يكون التبرّع آمناً؟

حين يكون الوسيط مملوكاً لهذه الحلقة وحدها: وزن مُحدَّث في مكانه، أو مخزن مؤقّت لا يقرأه أحد. أمّا الأوزان التي تحتفظ بنسخة منها لنقطة حفظ، فالتبرّع بها يمحوها بلا إنذار.

أخطاء نفاد الذاكرة: تشخيص لا تخمين

رسالة نفاد الذاكرة لا تقول لك السبب، بل تقول إن المحصّلة تجاوزت السقف. وهناك خمسة أسباب تغطّي معظم الحالات، ولكلٍّ منها مقياس يخصّه.

1. **الدفعة أكبر من اللازم.** القياس: `x.nbytes` لكل مدخل، مضروباً في عدد الموترات الحيّة.

2. **التنشيطات محفوظة للاشتقاق العكسي.** القياس: راقب الاستهلاك أثناء الخطوة نفسها.

3. **نسخ خفيّ كل خطوة.** القياس: راقب `memory_stats()` قبل الخطوة وبعدها؛ إن نما المستخدم بلا توقّف فأنت تُنشئ موترات لا تموت.

4. **إعادة تجميع متكرّرة.** القياس: اطبع عدّاد التجميع؛ كل شكل جديد يضيف برنامجاً وذاكرة.

5. **تجزئة الذاكرة.** القياس: `jax.devices()[0].memory_stats()` يفرّق بين المحجوز والمستخدم.

السبب الثالث هو الأكثر إخفاءً: حلقة تكتب `losses.append(float(loss))` تحتفظ بكل قيمة على المضيف، وتوقف الجهاز كل خطوة. والسبب الرابع يظهر عند تتبّع أشكال متغيّرة: كل شكل يُنتج برنامجاً جديداً لا يُحرَّر تلقائياً.

إذا كان `memory_stats()` يقول إن المحجوز أكبر بكثير من المستخدم، فهل المشكلة في نموذجك أم في طريقة تخصيص الذاكرة؟ وكيف تفرّق بينهما قبل أن تُصغّر الدفعة؟ ؟

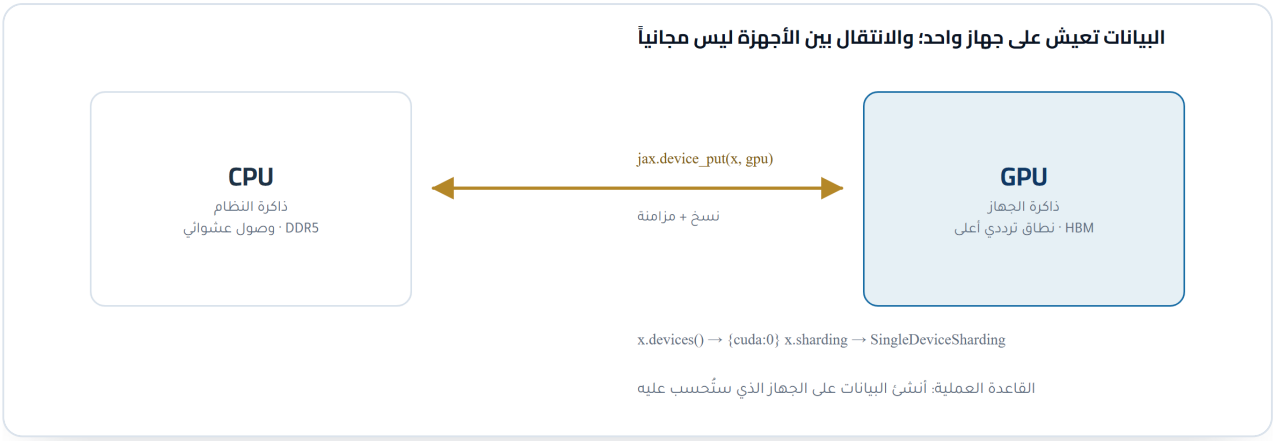
قواعد عملية للأجهزة والذاكرة

تبقى قواعد قليلة تُغنيك عن معظم التشخيص: أنشئ البيانات على الجهاز الهدف، لا تنقل داخل الحلقة، انتظر قبل أن تقيس، وتبرّع بما لا تعود إليه.

✦ الذاكرة تُدار بالتصميم لا بالحيل

تصغير الدفعة تعديل في الأعراض. التعديل الحقيقي تقليل ما تحتفظ به بين الخطوات: حالة النموذج، وحالة المحسّن، والدفعة الحالية.

لو أردت تدريب نموذجين على بطاقة واحدة بذاكرة محدودة، فما الذي تضبطه أولاً: حجم الدفعة، أم التبرّع، أم نسبة الحجز المسبق؟ ولماذا بهذا الترتيب؟ ؟



الشكل 5.1 — مسار البيانات من المضيف إلى الجهاز وما يعود. كل سهم ثمن، وكل ثمن يظهر في القياس إن انتظرت.

تمرين الفصل الخامس

1. أنشئ موتراً بشكل `(1024, 1024)` من نوع `float32`، وتحقّق من `nbytes`. هل يطابق حسابك اليدوي؟
2. قِس زمن ضربة مصفوفة كبيرة مرّتين: مرّة بلا انتظار، ومرّة مع `block_until_ready`. اشرح الفرق.
3. أضف `donate_argnums=0` إلى دالّة تحديث وزن، وقِس الفرق في الذاكرة المستخدمة.

▲ الحلّ النموذجي

1. الحساب: 1024 × 1024 × 4 بايت = 4,194,304 بايت، أي 4 ميغابايت.

```python
import jax, jax.numpy as jnp

x = jnp.ones((1024, 1024), dtype=jnp.float32)
print(x.nbytes)                       # 4194304
print(x.nbytes // (1024 * 1024), "MiB")
```

2. القياس بلا انتظار يقيس الإرسال فقط، لأن الحساب يحدث بعد عودة الدالّة:

```python
import time

f = jax.jit(lambda a: a @ a)
f(x).block_until_ready()              # تسخين

t0 = time.perf_counter(); f(x)
print("بلا انتظار:", round(time.perf_counter() - t0, 5))

t0 = time.perf_counter(); f(x).block_until_ready()
print("مع انتظار :", round(time.perf_counter() - t0, 5))
```

3. التبرّع يقلّل الذاكرة لأن الخرج يستعمل مخزن الوسيط نفسه:

```python
@jax.jit(donate_argnums=0)
def step(w, g):
    return w - 0.01 * g

w = jnp.ones(1_000_000)
g = jnp.ones(1_000_000)
w = step(w, g)                        # لا حجز إضافي بحجم w
print(w.shape)
```

خلاصة الفصل

- `jax.Array` وصف لكتلة على جهاز: نوع وشكل وتقطيع وموقع.
- اختيار الجهاز تلقائي، و `JAX_PLATFORMS` يقيّده قبل التهيئة.
- كل نقل بين المضيف والجهاز ثمن؛ أنشئ البيانات حيث ستحسب.
- لا معنى لأيّ قياس بلا `block_until_ready` .
- الحجز المسبق والتبرّع مفتاحان للذاكرة، ولكلٍّ شرط أمان.

مصطلحات الفصل

aval

القيمة المجرّدة التي يعرفها المترجم: شكل ونوع بلا بيانات.

sharding

وصف توزيع الموتر على الأجهزة.

device_put

دالّة تنقل موتراً إلى جهاز محدّد، وتنسخ إن لزم.

preallocation (حجز مسبق)

حجز XLA لنسبة من ذاكرة الجهاز قبل الحساب لتجنّب التخصيص المتكرّر.

buffer donation (تبرّع بالمخزن)

تعهّد بأن الدالّة لا تحتاج مدخلها بعد الاستدعاء، فيُعاد استخدام مخزنه.

الجزء الثاني

التبديلات الأربع

jit وgrad وvmap وshard_map: قلب JAX ونموذجها الذهني

jit: التجميع الذي يغيّر كل شيء

التتبّع، والرسم البياني، وإعادة التجميع، وحدود ما يمكن تجميعه

كتب أحدهم دالّة من عشرة أسطر، ووضع فوقها `@jax.jit` ، فرأى الزمن يهبط من أربعين ميلي ثانية إلى ثلاثة. ثم غيّر حجم الدفعة من 32 إلى 64، فعاد الزمن إلى أربعين. لم يتغيّر سطر واحد، لكن شيئاً ما أُعيد بناؤه من الصفر. هذا الفصل يفتح الصندوق: أربع مراحل بين سطرك والجهاز – بصمة تُحسب من الأشكال والأنواع.

ماذا يحدث حين تكتب @jax.jit؟

`jit` ليست «مفسّراً أسرع»، بل **حدّ**: ما داخله يُترجم قبل أن يعمل، وما خارجه يبقى بايثون عادياً.

عند الاستدعاء الأوّل تُستدعى دالّتك بمدخلات تحمل شكلاً ونوعاً فقط، فتُسجَّل عملياتها بدل أن تُحسب (**التتبّع**). ثم يُحوَّل السجلّ إلى `jaxpr` ، ويُنزل إلى `HLO` حيث تُدمج العمليات وتُخطَّط الذاكرة، ثم يُنتج **الملفّ التنفيذي** ويُخزَّن بالبصمة.

```python
import time, jax, jax.numpy as jnp

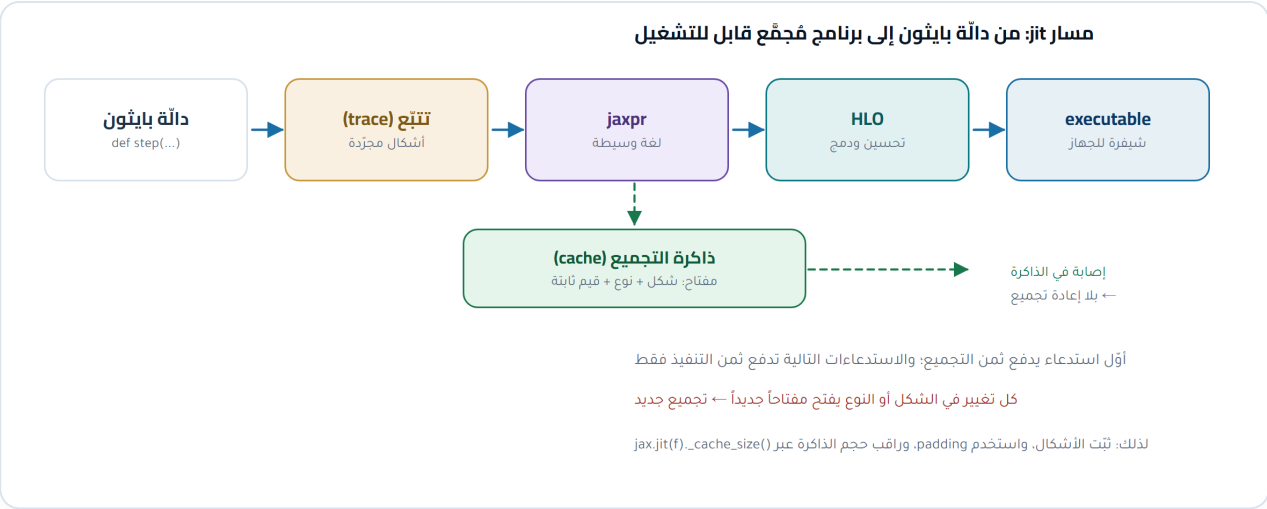
def block(x, w):
    for _ in range(20):
        x = jnp.tanh(x @ w)          # حلقة بايثون تُفكّ عند التتبّع
    return jnp.sum(x)

fast = jax.jit(block)
x = w = jnp.ones((256, 256)) * 0.01
t0 = time.perf_counter(); fast(x, w).block_until_ready()
t1 = time.perf_counter(); fast(x, w).block_until_ready()
print(f"first {(t1-t0)*1e3:.0f} ms, steady {(time.perf_counter()-t1)*1e3:.1f} ms")
```

‹‹ المخرجات ››

```
first 413 ms, steady 1.9 ms
```

في الاستدعاء الثاني نُفِّذ ملفّ واحد مُدمَج، ولم تُنفَّذ الحلقة في بايثون إطلاقاً.



الشكل 6.1 — تتبّع، ثم `jaxpr`، ثم `HLO`، ثم ملفّ تنفيذي مخزَّن.

> ★ **القاعدة التي تفسّر كلّ شيء لاحقاً**
>
> التجميع يحدث مرّة لكل **بصمة**، والبصمة من شكل كلّ وسيط ونوعه وترتيبه ومن قيم الوسائط الثابتة، لا من القيم. كل عطل في هذا الفصل نتيجة مباشرة لهذه الجملة.

التتبّع: كيف تُنفَّذ دالّتك على قيم وهمية

تخيّل مهندساً يرسم خطوات بناء جسر ولن يضع حجراً. المُتتبِّع يمرّر كائنات تحمل `shape` و `dtype` فقط، فتحدث **كل الآثار الجانبية مرّة واحدة**: اطبع داخل دالّة مجمّعة وسيظهر السطر مرّة.

```python
import jax, jax.numpy as jnp

@jax.jit
def f(x):
    print("tracing with", x.shape)  # مرّة واحدة فقط
    return jnp.sum(x * 2.0)

f(jnp.ones(3)); f(jnp.ones(3)); f(jnp.ones(3))

@jax.jit
def g(x):
    jax.debug.print("runtime: {v}", v=jnp.sum(x))   # وقت التنفيذ فعلاً
    return x

g(jnp.arange(4.0)); g(jnp.arange(4.0))
```

```
‹‹ المخرجات ››
tracing with (3,)
runtime: 6.0
runtime: 6.0
```

> ⚠ **الأثر الجانبي قد يكون خطيراً لا مزعجاً**
>
> كتابة ملفّ داخل دالّة مجمّعة تكتبه مرّة عند التتبّع ثم لا تكتبه أبداً، ولا تُصلح ذلك بعدّاد داخلي: قيمته ستُجمَّد كثابت.

> دالّة تطبع سجلّاً لكل عيّنة وتريد أن يبقى السلوك بعد التجميع. هل تحذف الطباعة، أم أن الفرق بين «البناء» و«التشغيل» يستحقّ إعادة التصميم؟ ؟

ما الذي يعيد التجميع؟

المفتاح في الذاكرة المؤقّتة ليس قيمة بل توقيع: لو كان القيمة لكان التجميع بلا معنى، إذ تُنتج كل دفعة مصفوفة جديدة.

التغيير	يعيد التجميع؟	السبب
شكل المصفوفة	نعم	الشكل جزء من البصمة
نوع العنصر	نعم	لكل نوع مسار تعليمات
قيمة وسيط ثابت	نعم	تُثبَّت في الرسم البياني
قيم المصفوفات	لا	لا تدخل البصمة إطلاقاً
متغيّر خارجي	لا – وهذا خطر	يُجمَّد عند التتبّع الأوّل

جدول 6.1 — ما يغيّر البصمة وما لا يغيّرها

السطر الأخير أخطرها لأنه لا يُنتج خطأً: اقرأ ثابتاً من الخارج ثم غيّره، فتبقى النتيجة كما كانت، لأن الرقم القديم صار جزءاً من الرسم.

✕ «غيّرت الإعداد ولم يتغيّر شيء»

يستهلك ساعات لأن الشيفرة تبدو صحيحة: أي رقم يؤثّر في الحساب ولا يأتي من مصفوفة يجب أن يكون وسيطاً صريحاً.

static_argnamesg static_argnums

بعض الوسائط ليست بيانات بل **قرارات بناء**: عدد الطبقات، اسم التنشيط، وضع التدريب. أعلنها ثابتة لتدخل قيمتها في البصمة عن قصد، بدل أن تُجمَّد بصمت.

Python

```
import jax, jax.numpy as jnp

def mlp(x, w, depth, act):
    for i in range(depth):          # depth ثابت ⇒ الحلقة تُفكّ
        x = act(x @ w[i])
    return x

fast = jax.jit(mlp, static_argnums=(2, 3))
ws = [jnp.ones((4, 4)) * 0.5 for _ in range(3)]

fast(jnp.ones((2, 4)), ws, 3, jnp.tanh)
print(fast._cache_size())           # 1
fast(jnp.ones((2, 4)), ws, 2, jnp.tanh)
print(fast._cache_size())           # 2
```

الثمن شرط صارم: الوسيط الثابت يجب أن يكون **قابلاً للتجزئة** لأنه يصير مفتاحاً؛ فالأعداد والنصوص والدوالّ صالحة، والقوائم لا.

✓ ثابتان لا أكثر

كل قيمة ثابتة جديدة تعني ملفّاً تنفيذياً جديداً؛ و `seq_len` بقيم 7 و13 و29 و64 يبني أربعة برامج حيث يكفي واحد.

الأشكال الثابتة: bucketingg padding

دفعة أطوالها 12 و37 و8 و91 لها شكل $(4, L)$ يتغيّر مع كل دفعة، لأن L هو طول أطول جملة؛ فتبني كل خطوة برنامجاً جديداً.

$$\text{cost} \approx N_{\text{shapes}} \times t_{\text{compile}} + N_{\text{steps}} \times t_{\text{run}}$$

حين يقترب عدد الأشكال من عدد الخطوات، يصبح الحدّ الأوّل هو المهيمن.

الحلّ الأوّل: **الحشو إلى طول ثابت** مع قناع في الخسارة: برنامج واحد، وثمنه حسابٌ على رموز وهمية.

الثاني: **الحزم** – أطوال قليلة (128، 256، 512) تُرفع إليها الدفعات.

Python

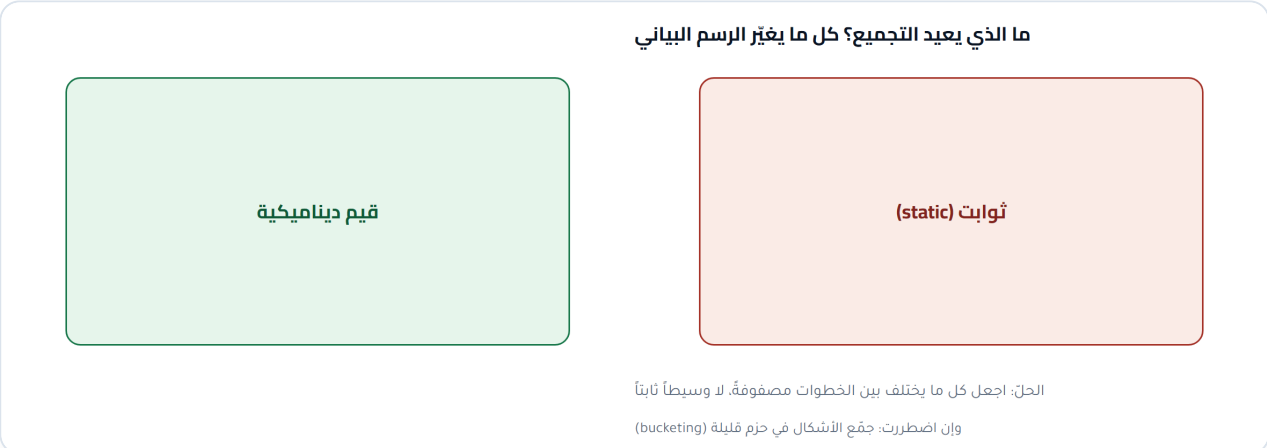
```python
import jax, jax.numpy as jnp

BUCKETS = (16, 32, 64, 128)

def pad_batch(t, buckets=BUCKETS):
    L = next(b for b in buckets if t.shape[1] <= b)
    return jnp.pad(t, ((0, 0), (0, L - t.shape[1])))

@jax.jit
def total(t):
    return jnp.sum(t.astype(jnp.float32))

for raw in (jnp.ones((2, 11)), jnp.ones((2, 30)), jnp.ones((2, 60))):
    print(raw.shape, "->", pad_batch(raw).shape)
```



الشكل 6.2 — يساراً: طول متغيّر ⇒ برنامج جديد كل مرّة. يميناً: أربع حزم ⇒ أربعة برامج فقط.

الأخطاء الشائعة داخل jit

كل خطأ هنا ينبع من مصدر واحد: أنت تكتب بايثون والمُتتبِّع يقرأ حساباً. حين تطلب ما لا يُمكن تمثيله يرفض.

تحويل إلى bool. `if x.sum() > 0:` يسأل عن قيمة لم تُحسب بعد، فتحصل على `TracerBoolConversionError`. العلاج `jax.lax.cond`.

قائمة بدل صفّ. `[1.0, 2.0]` يُنتج `TypeError: not a valid JAX type`؛ العلاج `jnp.array([...])`.

تعديل في المكان. `x[0] = 5.0` يُنتج `TypeError: arrays are immutable`؛ العلاج `x.at[0].set(5.0)`.

حلقة على قيمة. `while x.sum() > 1:` لا ينتهي عند التتبّع؛ العلاج `jax.lax.while_loop`. أمّا `for i in range(10)` فتُفكّ.

استدعاء NumPy. `np.asarray(x)` يُنتج `TracerArrayConversionError`؛ NumPy يريد بيانات والمُتتبِّع يملك شكلاً. استبدله بـ `jax.numpy`.

AOTg donate_argnums

في التدريب تصبح الأوزان القديمة قمامة بمجرّد حساب الجديدة، لكن JAX تحتفظ بالنسختين.
`donate_argnums` تصرّح: «خذه وأعد استخدام ذاكرته»؛ والمكسب ذاكرة.

```python
import jax, jax.numpy as jnp

@jax.jit(donate_argnums=(0,))
def step(w, x):
    return w - 0.01 * (w @ x)

w = x = jnp.ones((512, 512)) * 0.001
w2 = step(w, x)          # w استُهلكت: لا تقرأها بعد الآن

# AOT: ترجمة قبل أن تصل البيانات
compiled = jax.jit(lambda a, b: a - 0.01 * (a @ b)).lower(w2, x).compile()
print(compiled(w2, x).shape)
```

والترجمة المسبقة تفصل التجميع عن التشغيل: `lower` ينتج تمثيلاً منزلاً و`compile` ملفّاً قابلاً للاستدعاء. في خدمة استدلال، ثانية تجميع أمام أوّل طلب أسوأ تجربة ممكنة، والـ `AOT` يلغيها.

كيف تقيس أثر التجميع؟

أكثر قياس خاطئ شيوعاً توقيت الاستدعاء الأوّل ونسبته إلى «سرعة الدالّة». القياس الصحيح يفصل الحالتين، ويستدعي `block_until_ready` قبل إيقاف الساعة.

> ★ **ثلاثة أرقام تحكي القصّة**
>
> زمن البناء، وزمن الحالة المستقرة، وعدد البصمات: الأول جودة الإقلاع، والثاني جودة الحلقة.

دالّتك تعمل في 2 ملي ثانية وتبني في 900، وتُشغَّل مرّة لكل طلب. هل التجميع مكسب أم خسارة هنا؟ ؟

تمرين الفصل السادس

1. اكتب `f(x, k)` تُرجع `jnp.sum(x ** k)`، وجمّعها مع `k` ثابتاً. استدعها بالقيم 1 و2 و3 وعدّ البرامج المخزَّنة.
2. شغّل دالّة فيها `if x.sum() > 0` مع التجميع، وسجّل الخطأ، ثم أصلحها بـ `jax.lax.cond`.
3. قِس زمن البناء والحالة المستقرة على شكلين: `(32, 128)` و `(64, 128)`. هل يتضاعف البناء مع الدفعة؟ اشرح.

الحلّ النموذجي

1. ثلاث قيم ثابتة تعني ثلاث بصمات، أي ثلاثة برامج:

```python
import jax, jax.numpy as jnp

@jax.jit(static_argnums=(1,))
def f(x, k):
    return jnp.sum(x ** k)

x = jnp.arange(1.0, 5.0)
for k in (1, 2, 3):
    print(k, f(x, k))
print("cached:", f._cache_size())   # 3
```

2. الخطأ يذكر `Attempted boolean conversion` ويشير إلى سطر الشرط. الإصلاح يجعل التفريع جزءاً من الرسم:

```python
@jax.jit
def g(x):
    return jax.lax.cond(jnp.sum(x) > 0,
                        lambda v: v * 2.0,
                        lambda v: v * 0.0,
                        x)

print(g(jnp.array([1.0, -0.5])))   # [ 2. -1.]
```

3. لا يتضاعف: التتبّع والـ `jaxpr` يعتمدان على عدد العمليات، والتجميع على الحجم.

خلاصة الفصل

- أربع مراحل: تتبّع، ثم `jaxpr`، ثم `HLO`، ثم ملفّ تنفيذي.
- الأثر الجانبي يحدث عند التتبّع مرّة: الأرقام تُثبَّت في الرسم.
- البصمة من الأشكال والأنواع والثوابت، لا من القيم؛ وكل شكل جديد برنامج جديد.
- قِس البناء والحالة المستقرة منفصلين.

مصطلحات الفصل

trace (تتبّع)

تنفيذ الدالّة على قيم مجرّدة لتسجيل عملياتها.

jaxpr

تمثيل وسيط: متغيّرات وبدائيات بلا بايثون.

HLO

تمثيل XLA الذي يُنزل إليه الـ `jaxpr`.

cache key (بصمة)

مفتاح من الأشكال والأنواع والثوابت.

bucketing (حزم)

رفع الدفعة إلى أقرب طول محدود.

AOT

ترجمة مسبقة عبر `.lower().compile()`.

grad: التفاضل التلقائي بلا سحر

العكسي والأمامي، value_and_grad، والقيود التي يجب أن تعرفها

> سأل طالبٌ: «أين كود الاشتقاق؟». بحث في JAX عن مشتقّ تحليلي لكل دالّة فلم يجد شيئاً: لا جدول مشتقّات، ولا قاعدة سلسلة مكتوبة. والحقيقة أن `grad` لا تعرف كيف تشتقّ x^3 أكثر مما تعرف كيف تشتقّ `tanh`: هي لا تشتقّ شيئاً على الإطلاق، بل تعيد ترتيب الحساب الذي كتبته أنت.

الاشتقاق كتحويل، لا كحساب

خُذ $f(x) = x^3 + 2x$. باليد: $f'(x) = 3x^2 + 2$ ، وعند $x = 2$ تساوي 14 . الطريق اليدوي يحتاج قاعدة لكل دالّة وجهداً يتضاعف مع التعشيش. الطريق الآخر: أي حساب مؤلّف من خطوات، ولكل خطوة مشتقّتها المحلّية، والنتيجة حاصل ضربها.

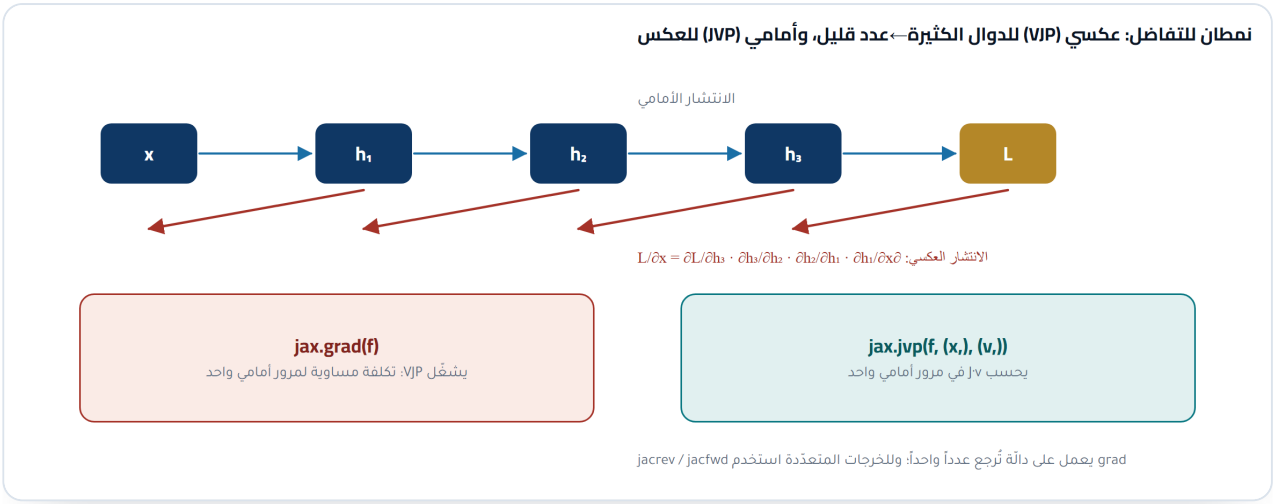
$$\partial f/\partial x = \prod_i \partial y_{i+1}/\partial y_i$$

قاعدة السلسلة ليست صيغة تُحفظ، بل مسار يُقطَع من الخرج إلى الدخل.

```python
import jax, jax.numpy as jnp

f = lambda x: x ** 3 + 2 * x
print(jax.grad(f)(2.0))            # 14.0
print(jax.grad(jax.grad(f))(2.0))  # 12.0
```

لاحظ ما أعادته `grad`: دالّة، لا رقماً. هذا هو التحويل – تأخذ دالّة وتُعيد دالّة، فتُركَّب على نفسها بلا حدّ.



الشكل 7.1 — التدرّج يعود من الخرج إلى الدخل ويجمع المساهمات عند كل عقدة؛ الشفافية هي الرسم نفسه.

> ★ **آلي لا رمزي**
>
> الاشتقاق الرمزي يبني تعبيراً قد يتضخّم أضعافاً. الآلي يطبّق قواعد محلّية على أرقام أثناء المرور، فلا ينتج تعبيراً أكبر من الأصل.

الوضع العكسي: VJP

تخيّل دالّة تأخذ مليون وزن وتُخرج رقماً واحداً – وهي حالة الخسارة في كل شبكة. الأمامي يحتاج مروراً لكل وزن، والعكسي مروراً واحداً. هذه ليست تحسيناً هندسياً بل الفرق بين ممكن ومستحيل.

الواجهة هي `jax.vjp`: تُرجع الخرج ودالّة سحب تأخذ مشتقّة الخرج وتُرجع مشتقّات المدخلات.

```python
import jax, jax.numpy as jnp

def f(x):
    return jnp.array([x[0] * x[1], jnp.sin(x[0])])

x = jnp.array([2.0, 3.0])
y, pullback = jax.vjp(f, x)
print(y)                                  # [6.  0.909]
print(pullback(jnp.array([1.0, 0.0]))[0])  # [3. 2.]
```

التكلفة لا تعتمد على عدد المدخلات بل على عدد المخارج؛ لذلك يكون العكسي رخيصاً حين يُخرج الحساب رقماً واحداً.

دالّة تُخرج 10,000 قيمة من ثلاثة مدخلات. أيّ الوضعين تختار، ولماذا ينقلب الحساب رأساً على عقب مقارنةً بدالّة الخسارة؟ ؟

الوضع الأمامي: JVP

الأمامي يسير مع الحساب: يأخذ اتجاهاً في المدخلات ويمرّره إلى الأمام. تكلفته بعدد المدخلات، وهو الأفضل حين تكون قليلة والمخارج كثيرة – كاشتقاق مسار حركة بالنسبة لثلاثة معاملات.

```python
import jax, jax.numpy as jnp

def g(x):
    return jnp.array([x[0] ** 2, x[1] ** 2, x[0] * x[1]])

y, y_dot = jax.jvp(g, (jnp.array([2.0, 3.0]),), (jnp.array([1.0, 0.0]),))
print(y, y_dot)     # [4. 9. 6.] [4. 0. 3.]
```

`jvp` تُرجع الخرج ومشتقّته في الاتجاه، ولا تحتاج تخزين الحساب الوسيط، فهي أخفّ ذاكرةً من العكسي.

value_and_grad ولماذا هو الأسرع

تحتاج في التدريب إلى الخسارة وتدرّجها معاً. الطريقة الساذجة تدفع مرورين أماميين، لأن `grad` وحدها تعيد حساب الدالّة. و`value_and_grad` تُرجع الاثنين من مرور واحد.

```python
import jax, jax.numpy as jnp

def loss(w, x):
    return jnp.mean(jnp.tanh(x @ w) ** 2)

w, x = jnp.ones((4, 4)) * 0.1, jnp.ones((8, 4))
value, grads = jax.value_and_grad(loss)(w, x)
print(value, grads.shape)

# معلومات إضافية بلا اشتقاق
def loss_aux(w, x):
    h = jnp.tanh(x @ w)
    return jnp.mean(h ** 2), h

(v, h), grads = jax.value_and_grad(loss_aux, has_aux=True)(w, x)
print(v, h.shape)
```

في شبكة كبيرة يكون المرور الأمامي جزءاً كبيراً من الزمن، فحذفه يجعل الخطوة أسرع بنسبة ملموسة بلا أي تغيير في النتيجة العددية.

> ✓ **العادة الصحيحة من البداية**
>
> اكتب `value_and_grad` دائماً، حتى لو لم تحتج القيمة اليوم؛ ستسجّلها غداً.

التدرّج على الأشجار

الأوزان في نموذج حقيقي قاموس من طبقات لا مصفوفة واحدة. و `grad` تفهم ذلك: تمرّر قاموساً فتُرجع قاموساً بالبنية نفسها، بلا تسطيح ولا فهرسة يدوية.

```python
import jax, jax.numpy as jnp

def loss(params, x):
    h = jnp.tanh(x @ params["w1"] + params["b1"])
    return jnp.mean((h @ params["w2"]) ** 2)

params = {"w1": jnp.ones((4, 8)) * 0.1, "b1": jnp.zeros(8), "w2": jnp.ones((8, 1)) * 0.1}
g = jax.grad(loss)(params, jnp.ones((16, 4)))
print(g["b1"].shape)                    # (8,)
print(len(jax.tree.leaves(g)))          # 3

gx, gp = jax.grad(loss, argnums=(1, 0))(params, jnp.ones((16, 4)))
print(gx.shape, gp["w2"].shape)
```

التدرّج يحترم البنية: `jax.tree.map` يعمل عليه كما على الأوزان. و `argnums` يحدّد موضع الاشتقاق؛ وتمرير توپل يُعيد توپلاً بالتنسيق نفسه.

> ⚠ **ترتيب الوسائط ليس تفصيلاً**
>
> `argnums=1` تعني «الوسيط الثاني». خطأ في الرقم يعطيك تدرّجاً شكله صحيح وقيمه بلا معنى، ولن يشتكي أحد.

ما لا يستطيعه grad

الاشتقاق يحتاج بنية جبرية: أعداد حقيقية أو مركّبة تُجمع وتُضرب. ما خرج عن ذلك لا مشتقّة له، لا لأن JAX قصّرت، بل لأن السؤال بلا معنى

المدخلات الصحيحة والمنطقية. `grad` على وسيط `int32` ترفض بـ `TypeError: grad requires real- or complex-valued inputs`. مرّر `float` وحوّل داخل الدالّة إن لزم.

التحكّم المعتمد على القيم. `if x > 0` يُجمَّد عند التتبّع فلا يتفرّع التدرّج. استخدم `jax.lax.cond` أو دالّة ناعمة بدل الحدّ الحادّ.

الأعداد المركّبة. الدوالّ الهولومورفية مرفوضة: `grad of a holomorphic function is not supported`، لأن تعريف التدرّج يفترض خرجاً حقيقياً. استخدم `jax.vjp` مع مشتقّة مرافقة.

NaN في التدرّج. ليس رفضاً بل نتيجة: `sqrt(0)` و`log(0)` و`abs(0)` مشتقّاتها غير معرّفة عند الصفر، فتُلوّث ما بعدها.

hessiang jacfwdg jacrev

التدرّج حالة خاصة: خرج قياسي. أمّا اليعقوبية فتحتاج قراراً – تمرّ على المدخلات (`jacfwd`) أم المخارج (`jacrev`)؟ يحدّده أيّ البعدين أصغر.

التحويل	الخرج	المرورات	متى تستخدمه
`grad`	متّجه بطول n	1 عكسي	خرج قياسي
`jacrev`	مصفوفة m×n	m عكسي	m صغير ng كبير
`jacfwd`	مصفوفة m×n	n أمامي	n صغير mg كبير
`hessian`	مصفوفة n×n	n عكسي	n صغير

جدول 7.1 — تكلفة كل تحويل لدالّة $f: \mathbb{R}^n \to \mathbb{R}^m$

```python
import jax, jax.numpy as jnp

h = lambda v: jnp.array([v[0] ** 2, v[0] * v[1]])
v = jnp.array([2.0, 3.0])

print(jax.jacrev(h)(v))                      # [[3. 2.] [1. 1.]]
print(jax.jacfwd(h)(v))                      # النتيجة نفسها
print(jax.hessian(lambda z: jnp.sum(z ** 3))(v))
```

> ★ **اختر بالبعد الأصغر**
>
> هيسيان لخسارة فيها مليون وزن تُقاس بـ 10^{12} عنصر – مستحيلة. لهذا لا تُحسب صراحةً في التدريب، بل تُستبدل بضرب مصفوفة بمتّجه عبر `jax.vjp` على التدرّج.

أخطاء التدرّج الشائعة

أكثر تدرّج خاطئ تراه ليس خطأً يرفض التشغيل، بل صفراً أو `nan` يمرّ بصمت.

stop_gradient في الموضع الخطأ. وضعها حول الجزء الذي تريد تدريبه يقطع المسار ويُعيد صفراً. استخدمها حيث تمنع التدرّج فعلاً، كأهداف ثابتة في دوالّ خسارة متقدّمة.

ثابت منفصل. بناء ثوابت بـ `numpy` ثم استخدامها داخل الحساب يقطع المسار: لا مشتقّة لشيء لم يمرّ في الرسم. ابنِ الثوابت بـ `jnp` .

float0. الاشتقاق بالنسبة لوسيط صحيح يعطي نوعاً اسمه `float0` : صفر بلا قيمة. تراه عبر `jax.vjp` لا عبر `grad` ، وهو تذكير بأن السؤال غير صالح.

> × **«التدرّج صفر ولا أعرف لماذا»**
>
> قبل أن تتّهم المعادلة، تحقّق من ثلاثة: هل كل ما تدرّبه خرج من `jnp` ؟ هل مرّ عبر `stop_gradient` ؟ هل نوعه عائم؟

خسارتك تنقص في كل خطوة والتدرّج صفر. كيف تنقص الخسارة إن كان التدرّج صفراً؟ وما الذي يخبرك به ذلك عن مصدر العطل؟ ؟

تمرين الفصل السابع

1. احسب مشتقّة $f(x) = x^3 + 2x$ عند 2 باليد وبـ `grad`، ثم تحقّق من المشتقّة الثانية.
2. عرّف دالّة من $\mathbb{R}^2$ إلى $\mathbb{R}^2$، واحسب يعقوبيّتها بالوضعين.
3. ابنِ خسارة على قاموس أوزان، واحسب التدرّج بـ `argnums` ثم بـ `has_aux`.

الحلّ النموذجي

1. $f'(2) = 14$ والثانية $6x = 12$:

```python
import jax, jax.numpy as jnp

f = lambda x: x ** 3 + 2 * x
print(jax.grad(f)(2.0))              # 14.0
print(jax.grad(jax.grad(f))(2.0))    # 12.0
```

2. الوضعان يعطيان المصفوفة نفسها لأن الأبعاد صغيرة:

```python
g = lambda v: jnp.array([v[0] * v[1], v[0] + v[1]])
v = jnp.array([2.0, 3.0])
print(jax.jacrev(g)(v))   # [[3. 2.] [1. 1.]]
print(jax.jacfwd(g)(v))   # [[3. 2.] [1. 1.]]
```

3. التدرّج يحافظ على بنية القاموس، و`has_aux` يمرّر معلومات بلا اشتقاق:

```python
params = {"w": jnp.ones((3, 3)) * 0.1}

def loss(p, x):
    h = x @ p["w"]
    return jnp.sum(h ** 2), h.mean()

(v, aux), grads = jax.value_and_grad(loss, has_aux=True)(params, jnp.ones((2, 3)))
print(v, aux, grads["w"].shape)   # (3, 3)
```

خلاصة الفصل

- `grad` تحويل يُرجع دالّة، لا حساباً لرقم.
- العكسي رخيص حين تقلّ المخارج؛ الأمامي حين تقلّ المدخلات.
- `value_and_grad` يوفّر مروراً أمامياً في كل خطوة.
- التدرّج يحترم أشجار الوسائط، و `argnums` يحدّد موضعه.
- الصفر الصامت و `nan` أخطر من الخطأ الصريح.

مصطلحات الفصل

autodiff

اشتقاق آلي يطبّق قواعد محلّية أثناء التنفيذ.

VJP

سحب المشتقّة من الخرج إلى الدخل.

JVP

دفع مشتقّة اتجاهية إلى الأمام عبر اليعقوبية.

cotangent

القيمة الواردة من الأعلى، تُضرب محلياً وتُمرَّر للأسفل.

float0

نوع صفري بلا قيمة، يظهر عند الاشتقاق بالنسبة لوسيط صحيح.

hessian

مشتقّة ثانية: مصفوفة n×n تكلفتها خطّية مع n.

vmap: الدُفعات بلا حلقات

التوجيه التلقائي، والمحاور، والتفاعل مع الحالة والتفرّع

دالّة تعمل على عيّنة واحدة، وخمسة آلاف عيّنة في الملفّ. الحلّ البديهي حلقة `for`، وهي تعمل وتُخرج أرقاماً صحيحة – ثم تنظر في الزمن فتجد أن التسعين بالمئة منه ذهب إلى بايثون لا إلى الحساب. `vmap` هي الجواب الذي لم تكتبه: تأخذ دالّة العيّنة الواحدة، وتُعيد دالّة الدفعة، بلا حلقة ولا تغيير في المنطق.

الحلقة التي لا نريد كتابتها

ابدأ بالشكل الطبيعي: دالّة تفهم عيّنة واحدة. $\text{score}(w, x) = w \cdot x$، حيث x متّجه بطول أربعة. ثم دفعة من ثمانية: إمّا حلقة، وإمّا `vmap`.

```python
import jax, jax.numpy as jnp

def score(w, x):                # عيّنة واحدة
    return jnp.dot(w, x)

w = jnp.ones(4)
batch = jnp.arange(32.0).reshape(8, 4)

loop = jnp.stack([score(w, x) for x in batch])   # حلقة بايثون
vec = jax.vmap(score, in_axes=(None, 0))(w, batch)
print(loop.shape, jnp.allclose(loop, vec))       # (8,) True
```

النتيجة واحدة، لكن ما يجري مختلف: الحلقة تُطلق ثمانية حسابات صغيرة، كل واحد يدفع ثمن عبور حدود بايثون، و `vmap` تُحوّل الدالّة إلى نسخة تعمل على الدفعة كصفّ واحد.



الشكل 8.1 — `vmap` تضيف بعداً أمام كل بعد موجود، ولا تلمس منطق الدالّة.

> ★ **ليست حلقة أسرع، بل بعداً جديداً**
>
> `vmap` لا تنفّذ دالّتك ثماني مرّات أسرع؛ إنها تعيد كتابتها لتأخذ شكلاً `(8, ...)` . هذا هو سبب اختفاء الحلقة من الشيفرة الناتجة، وسبب أن الأخطاء تظهر في أشكال لا في قيم.

out_axesg in_axes

السؤال الذي تطرحه `vmap` على كل وسيط: أيّ بعد هو الدفعة؟ الجواب في `in_axes` . القيمة `0` تعني «البعد الأوّل دفعة»، و `None` تعني «لا تبعده، مرّره كما هو لكل العيّنات».

```python
import jax, jax.numpy as jnp

def f(w, x):
    return jnp.dot(w, x)

w = jnp.ones(4)
batch = jnp.ones((8, 4))

# x، والأوزان مشتركة، الدفعة في البعد الثاني من
xt = jnp.ones((4, 8))
out = jax.vmap(f, in_axes=(None, 1))(w, xt)
print(out.shape)                                  # (8,)

# الدفعة في البعد الأول، والخرج في بعد جديد
out2 = jax.vmap(f, in_axes=(None, 0), out_axes=1)(w, batch)
print(out2.shape)                                 # (8,)
```

`out_axes` تحدّد أين يقع بعد الدفعة في الخرج؛ الافتراضي صفر.

✕ «shape mismatch» في سطر لا تعرفه

أكثر أخطاء `vmap` أن `in_axes` تصف وسيطاً غير الذي تظنّه. الوسائط تُعدّ بالترتيب، لا بالأسماء. اطبع أشكال الوسائط قبل الاستدعاء، وتذكّر أن `None` لا تعني «تجاهل» بل «مرّر بلا بعد دفعة».

دالّة تأخذ مصفوفة أوزان ومصفوفة مدخلات، وتريد أن تُدرّب أوزاناً مختلفة لكل عيّنة — أي دفعة على الأوزان أيضاً. ماذا تكتب في `in_axes`، وماذا يعني ذلك للنتيجة؟ ؟

vmap فوق pytree

الوسائط في الواقع ليست مصفوفات بل أشجار: قاموس فيه صور ونصوص وأقنعة. و`vmap` تقبل `in_axes` بالبنية نفسها، فيأخذ كل ورقة محورها الخاص.

```python
import jax, jax.numpy as jnp

def step(params, batch):
    h = jnp.tanh(batch["x"] @ params["w"] + params["b"])
    return jnp.sum(h * batch["mask"])

params = {"w": jnp.ones((4, 2)) * 0.1, "b": jnp.zeros(2)}
batch = {"x": jnp.ones((8, 4)), "mask": jnp.ones((8, 2))}

out = jax.vmap(step, in_axes=(None, 0))(params, batch)
print(out.shape)      # (8,)
```

لاحظ `(None, 0)`: الأولى للأوزان كلها مشتركة، والثانية للقاموس كله مُبعَّد من أوّله. هذا يعمل لأن `0` تُطبَّق على كل ورقة في الشجرة، لا على القاموس ككائن.

✓ اكتب الدالّة لعيّنة واحدة دائماً

أسهل طريقة لكتابة دالّة قابلة لـ `vmap` أن تتخيّلها بلا بعد دفعة إطلاقاً. إن احتجت داخل الدالّة إلى `x.shape[0]` لتعرف حجم الدفعة، فأنت تكتبها بشكل خاطئ.

randomg vmap

هنا يظهر فرق جوهري عن NumPy. لا يوجد مولّد عشوائي عالمي في JAX؛ المفتاح قيمة صريحة. فإذا أردت ضجيجاً مختلفاً لكل عيّنة، احتجت مصفوفة مفاتيح – وهذا بالضبط ما تُبعده `vmap` .

```python
import jax, jax.numpy as jnp

def noise(key, shape):
    return jax.random.normal(key, shape)

key = jax.random.key(0)
keys = jax.random.split(key, 8)          # ثمانية مفاتيح مستقلة

draws = jax.vmap(noise, in_axes=(0, None))(keys, (3,))
print(draws.shape)                       # (8, 3)
print(jnp.allclose(draws[0], draws[1]))  # False
```

لو استخدمت المفتاح نفسه لحصلت على ثماني نسخ متطابقة – خطأ يمرّ بصمت لأن الأشكال صحيحة.

> ⚠ **مفتاح واحد، ثماني نسخ**
>
> تمرير المفتاح نفسه إلى `vmap` بلا `split` يعطي ضجيجاً متطابقاً في كل العيّنات. لا رسالة خطأ، ولا شكل خاطئ – فقط تدريب يتوقّف عن التعلّم لأن كل عيّنة ترى التحويل نفسه. افحص الاستقلال باختبار صغير.

vmap gradg: أربع تركيبات

التركيب هو أهمّ ما تفعله `vmap` عملياً. الترتيب يحدّد المعنى، والنتيجتان مختلفتان عددياً لا شكلاً فقط.

التركيب	المعنى	الشكل
`vmap(grad(f), in_axes=(None, 0))`	تدرّج كل عيّنة بالنسبة للأوزان المشتركة	(B, …)
`vmap(grad(f), in_axes=(0, 0))`	تدرّج كل عيّنة بالنسبة لوسائطها	(B, …)
`grad(mean(vmap(f)))`	تدرّج متوسّط الخسارة	مثل الأوزان
`grad(sum(vmap(f)))`	تدرّج مجموع الخسائر	مثل الأوزان

جدول 8.1 — أربع تركيبات وما تعنيه

```python
import jax, jax.numpy as jnp

def loss(w, x):
    return jnp.sum((w * x) ** 2)

w = jnp.ones(4)
batch = jnp.ones((8, 4)) * 2.0

per_example = jax.vmap(jax.grad(loss), in_axes=(None, 0))(w, batch)
total = jax.grad(lambda w: jnp.mean(jax.vmap(loss, in_axes=(None, 0))(w, batch)))(w)

print(per_example.shape)   # (8, 4)
print(total.shape)         # (4,)
```

الأول يعطيك ثمانية اتّجاهات، كل واحد يصلح لتحديث منفصل. والثاني يعطيك اتجاهاً واحداً لمتوسّط الخسارة – وهو ما تحتاجه في تدريب عادي.

> ✦ **لماذا لا يتساويان؟**
>
> متوسّط التدرّجات يساوي تدرّج المتوسّط، لأن الاشتقاق خطّي. لكن تدرّج المجموع ليس مجموع التدرّجات مقسوماً على عدد العيّنات إلا إذا كانت الخسارة خطّية في الدفعة. الخلط بينهما يغيّر مقياس التحديث بلا أن يشتكي أحد.

حدود vmap

`vmap` قوية لكنها ليست سحراً. ثلاثة قيود تشرح معظم ما يخذل الناس.

لا تحكّم معتمد على القيم. `if x > 0` داخل دالّة مُبعَّدة يسأل عن شرط لكل عيّنة على حدة، وهو ما لا يمكن تمثيله كبعد. الحلّ: `jax.lax.cond` إن كان الشرط واحداً للدفعة، أو `jnp.where` إن كان لكل عيّنة، أو `jax.lax.map` إن كان التفرّع بنيوياً حقاً.

لا آثار جانبية. الطباعة والكتابة إلى ملفّ لا تتكرّر لكل عيّنة؛ هي أثر واحد عند التتبّع، كما رأينا في الفصل السادس.

الذاكرة تنمو مع الدفعة. `vmap` تُنشئ نسخة مُبعَّدة من كل قيمة وسيطة. دفعة كبيرة تعني ذاكرة كبيرة، وقد تنفد قبل أن تنفد الأفكار. البديل `jax.lax.map` ينفّذ الحلقة تسلسلياً فيوفّر الذاكرة ويخسر التوازي.

متى لا تستخدم vmap؟

السؤال الأخير هو الأهمّ: متى يكون بعد الدفعة الحقيقي أرخص من `vmap`؟ حين تكون عمليتك أصلاً مصفوفية. ضرب $(B, D) \times (D, K)$ دفعةً واحدة أسرع من `vmap` على متّجه بطول D، لأن الثانية تبني شكلاً (B, D) ثم تضربه.

استخدم `vmap` حين يكون المنطق لعيّنة واحدة بطبيعته: حلقة تدريب مصغّرة، تفرّع على حالة، تدرّج لكل مثال. واستخدم البعد الحقيقي حين تكون العملية جبراً خطّياً واحداً.

> ★ **السؤال الصحيح**
>
> لا تسأل «هل `vmap` أسرع؟» بل «هل منطق عيّنتي قابل للكتابة كمصفوفة؟». إن كان نعم، اكتبه مصفوفة. وإن كان لا، فـ `vmap` هي التي تُنجيك من الحلقة.

دالّة تعالج صورة واحدة وتحتوي على حلقة `while` تتوقّف عند عتبة. هل يمكن تبعيدها بـ `vmap` كما هي؟ وما الذي ستحتاج إلى تغييره أولاً؟ ؟

تمرين الفصل الثامن

1. اكتب `score(w, x)` لعيّنة واحدة، ثم بعّدها بدفعة من 16. قارن الناتج بحلقة بايثون.
2. ابنِ قاموس دفعة فيه `x` و `mask` ، وبعّده مع إبقاء الأوزان مشتركة.
3. احسب تدرّج كل عيّنة وتدرّج المتوسّط، وقارن الشكلين والمعنى.

الحلّ النموذجي

1. النتيجتان متطابقتان، والفرق في الشكل لا في القيمة:

```python
import jax, jax.numpy as jnp

score = lambda w, x: jnp.dot(w, x)
w, batch = jnp.ones(4), jnp.ones((16, 4)) * 3.0

vec = jax.vmap(score, in_axes=(None, 0))(w, batch)
loop = jnp.stack([score(w, x) for x in batch])
print(vec.shape, jnp.allclose(vec, loop))   # (16,) True
```

2. `in_axes=(None, 0)` تُبعد كل ورقة في القاموس من بعدها الأوّل:

```python
def step(p, b):
    return jnp.sum(jnp.tanh(b["x"] @ p["w"]) * b["mask"])

p = {"w": jnp.ones((4, 2)) * 0.1}
b = {"x": jnp.ones((16, 4)), "mask": jnp.ones((16, 2))}
print(jax.vmap(step, in_axes=(None, 0))(p, b).shape)   # (16,)
```

3. لكل عيّنة تدرّجها، وللمتوسّط تدرّج واحد: الأول شكله `(16, 4)` والثاني `(4,)` . الأول يصلح لتحديث منفصل لكل مثال، والثاني لخطوة تدريب واحدة.

خلاصة الفصل

- `vmap` تضيف بعد دفعة بدل كتابة حلقة.
- `in_axes` تحدد محور كل وسيط، و `None` تعني مشترك.
- الأشجار تُبعَّد بورقة لكل ورقة، بالبنية نفسها.
- المفاتيح تُقسَّم قبل التبعيد، وترتيب `vmap` و `grad` يغيّر المعنى.
- الذاكرة تنمو مع الدفعة؛ `lax.map` بديل تسلسلي.

مصطلحات الفصل

vmap

تحويل يضيف بعد دفعة أمام أبعاد الدالّة بلا حلقة ظاهرة.

in_axes

محور الدفعة لكل وسيط؛ `0` للأول و `None` للمشترك.

out_axes

موضع بعد الدفعة في الخرج، افتراضه الصفر.

batching

إعادة كتابة الحساب ليعمل على دفعة كصفّ واحد بدل تكراره.

lax.map

حلقة تسلسلية داخل الرسم البياني، بديل عند ضيق الذاكرة.

per-example grad

تدرّج محسوب لعيّنة واحدة، يُنتج مصفوفة بشكل الدفعة.

النقاء والحالة وأشجار pytree

لماذا لا حالة في JAX؟ وكيف تبني نماذج بحالة نظيفة

جرّب أن تكتب عدّاداً. متغيّر واحد، وزائد واحد، ودالّة تُستدعى في كل خطوة تدريب. في بايثون العادية ينتهي الأمر في سطرين؛ وفي JAX ستحصل على رقم لا معنى له: يتوقّف عند واحد، أو يقفز قفزةً واحدة عند أوّل استدعاء ثم يجمُد. المشكلة ليست في العدّاد، بل في الفكرة التي يمثّلها: أن يكون للدالّة ذاكرةٌ خارج مدخلاتها. هذا الفصل عن تلك الفكرة، وعن البنية التي استبدلتها JAX بها: شجرةٌ من المصفوفات تُمرَّر صراحةً، حتى تعرف أنت قبل المترجم ما الذي تغيّر.

لماذا لا تستطيع JAX أن تحتفظ بعدّاد؟

الدالّة النقية يعتمد خرجها على مدخلاتها وحدها. يبدو التعريف بديهياً حتى تكتب أوّل عدّاد؛ عندئذٍ يصبح الخرج دالّةً في «كم مرّة استُدعيت»، أي في الزمن. والزمن ليس مدخلاً.

ولأن `jax.jit` لا ينفّذ دالّتك بل يقرأها مرّة واحدة – وهي عملية تُسمّى التتبّع – فإن أسطر بايثون الجانبية تنفّذ أثناء القراءة، لا أثناء التشغيل.

```python
import jax
import jax.numpy as jnp

count = 0

@jax.jit
def step(x):
    global count
    count += 1          # ينفّذ عند التتبّع، لا عند التشغيل
    return x + 1.0

for _ in range(3):
    step(jnp.zeros(()))
print("count =", count)
```

‹‹ المخرجات ››

```
count = 1
```

× الخطأ الذي يبدو صحيحاً

الشيفرة أعلاه لا تُطلق استثناءً؛ تُرجع مصفوفة صحيحة ورقماً صغيراً. وهذا أخطر أنماط الخطأ: لا انهيار، بل قيمة معقولة وكاذبة. ولو مرّرت مصفوفة بشكل مختلف لقفز العدّاد إلى اثنين.

وما نكسبه ثلاثة: تخزينٌ مؤقّت، وإعادةُ ترتيب (يحذف XLA حساباً لم يُستخدم لأنه لا أحد يراقب الترتيب)، وتوزيعٌ ممكن: حين تُقطّع المصفوفة على ثمانية أجهزة، فأين تعيش الحالة العامة؟

لو سمحت JAX بمتغيّر عام واحد فقط، فما أوّل شيء سينكسر: التجميع أم التوزيع؟ وأيّهما كان سينكسر بصمت؟ ؟

pytree: البنية التي يراها JAX

حين تمرّر قاموساً إلى دالّة مجمّعة، لا ترى JAX قاموساً. ترى قشرةً (structure) وأوراقاً (leaves): القشرة تنظيم – قاموس أو قائمة أو `namedtuple` – والأوراق هي البيانات.

Python

```
import jax
import jax.numpy as jnp

params = {"w": jnp.ones((2, 3)), "b": jnp.zeros(3)}
grads  = {"w": jnp.full((2, 3), 0.5), "b": jnp.ones(3)}

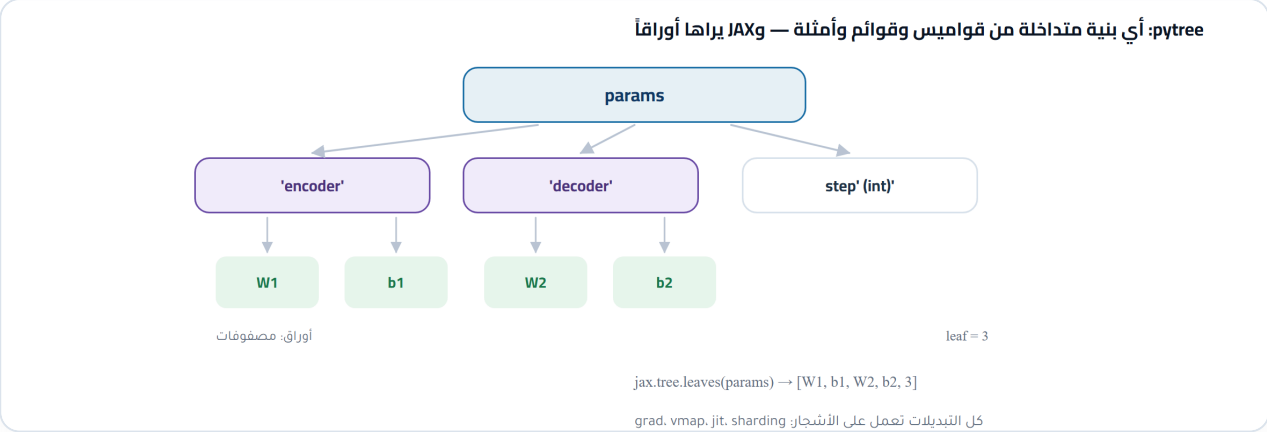
updated = jax.tree.map(lambda p, g: p - 0.1 * g, params, grads)

print(len(jax.tree.leaves(params)))                          # 2
print(jax.tree.structure(updated) == jax.tree.structure(params))  # True
```

‹‹ المخرجات ››

```
2
True
```

> ★ **القشرة جزء من التوقيع**
>
> `jax.tree.structure` يعيد وصف القشرة بلا قيم، وهو قابل للتجزئة ويُستخدم مفتاحاً في ذاكرة الترجمة. تغيير اسم مفتاح من `"w"` إلى `"weight"` يُبطل الترجمة السابقة، ولو بقي عدد المصفوفات وأحجامها كما هو.



الشكل 9.1 — الشجرة التي تراها JAX: العقد الداخلية تنظيم، والأوراق وحدها تصل إلى الحساب.

تسجيل نوعك الخاص

وماذا لو كانت حالتك كائناً من نوعك؟ `Dense` ليس قاموساً، فكيف يعرف JAX ما داخله؟ تخبره: `flatten` يفكّ الكائن إلى أبنائه، و `unflatten` يعيد بناءه.

```python
import jax
import jax.numpy as jnp
from dataclasses import dataclass

@dataclass(frozen=True)
class Dense:
    w: jax.Array
    b: jax.Array

def flatten(d):
    return (d.w, d.b), None          # الأبناء، ثم البيانات الثابتة

def unflatten(aux, children):
    return Dense(*children)

jax.tree_util.register_pytree_node(Dense, flatten, unflatten)

layer = Dense(jnp.ones((3, 4)), jnp.zeros(3))
print(jax.tree.leaves(layer))
```

✓ الثوابت في aux_data، لا بين الأوراق

الوسيط الثاني في `flatten` موضع البيانات الإضافية: ما يلزم لإعادة البناء ولا يُعدّ بيانات. لو أعدت رقم الميزات كابنٍ ثالث، عامله `jit` كورقة وحوّله إلى مُتتبَّع، فتفشل في بناء شكل.

الحالة كمدخل ومخرج

إذن ننقل الحالة إلى التوقيع: تأخذ القديمة وتعيد الجديدة، ولا تحتفظ بشيء بين الاستدعاءين.

```python
import jax
import jax.numpy as jnp

def step(state, x):
    count, total = state
    return (count + 1, total + x), x * 2.0

state = (jnp.array(0), jnp.array(0.0))
step_jit = jax.jit(step)

for i in range(3):
    state, y = step_jit(state, jnp.array(float(i)))

print(state)    # (3, 3.0)
print(y)        # 4.0
```

هذا النمط تعتمده كل مكتبة: Optax يعيد حالة جديدة، وFlax قاموس متغيّرات، و `jax.lax.scan` يحمل الحالة عبر الحلقة. الثمن توقيع أطول ونسخ يدوي.

BatchNorm كمثال حقيقي

> ؟ في BatchNorm، المتوسّط المتحرّك ليس وزناً يتعلّمه الانحدار. فلماذا يُخزَّن مع الأوزان، ولماذا لا يكفي أن يوضع في مكان آخر؟

أثناء التدريب تحسب الطبقة متوسّط الدفعة وتباينها وتطبّع بهما، وتحدّث متوسّطاً متحرّكاً للاستدلال. هذا المتوسّط ليس معاملاً: لا تدرّج له. إنه إحصاء، ولهذا يفصل Flax بين `params` و `batch_stats`:

```python
import jax, jax.numpy as jnp
from flax import linen as nn

class Block(nn.Module):
    @nn.compact
    def __call__(self, x, train: bool):
        x = nn.Dense(8)(x)
        x = nn.BatchNorm(use_running_average=not train)(x)
        return nn.relu(x)

model = Block()
x = jnp.ones((4, 8))
variables = model.init(jax.random.key(0), x, train=True)

y, updates = model.apply(
    {"params": variables["params"], "batch_stats": variables["batch_stats"]},
    x, train=True, mutable=["batch_stats"])

print(y.shape, updates["batch_stats"]["mean"].shape)
```

‹‹ المخرجات ››

```
(4, 8) (8,)
```

> ⚠ **لا تضع `batch_stats` في `params`**
>
> لم نمرّر «النموذج» ككائن حيّ، بل قاموساً أعلنّا أيّ فرع منه قد يتغيّر؛ والإعلان يسمح لـ `jit` بمعرفة شكل الخرج. ولو وُضعت الإحصاءات في `params` لطبّق عليها AdamW تناقص الأوزان فشُوّهت بصمت.

الحالة في NNX

ثم جاء NNX بجواب معاكس: لماذا نفصل؟ أعِد الحالة إلى داخل الكائن. `nnx.Module` يحمل أوزانه وإحصاءاته معاً ويعدّلها في مكانها، لكن داخل نظام يحافظ على قابليتها للتجميع.

```python
import jax
from flax import nnx

class Block(nnx.Module):
    def __init__(self, rngs: nnx.RNGs):
        self.linear = nnx.Linear(8, 8, rngs=rngs)
        self.norm = nnx.BatchNorm(8, rngs=rngs)

    def __call__(self, x):
        return nnx.relu(self.norm(self.linear(x)))

model = Block(nnx.RNGs(0))
graphdef, state = nnx.split(model)      # الشجرة عند الحدود
print(len(jax.tree.leaves(state)))
model = nnx.merge(graphdef, state)      # عودة إلى كائن حيّ
```

الانضباط لم يختفِ بل انتقل: ما زلت تحتاج `nnx.split` عند الحدود، قبل الحفظ وقبل المُحسّن؛ لكن الفصل صار عمليةً تُطلب عند الحاجة لا بنيةً تُفرض دائماً. ولا تخلط Linen وNNX في ملف واحد.

أنماط خاطئة للحالة

ثلاثة أنماط تتكرّر، وكلها تعمل في بايثون العادي وتخون في JAX: قائمة عامّة تُلحق بها السجلات (`log.append(...)` داخل دالّة مجمّعة تعطي عنصراً واحداً بعد مئة استدعاء)، وإغلاق على كائن قابل للتغيير، وذاكرة مؤقّتة على مستوى الوحدة.

وتشترك الثلاثة في عَرَض واحد: تعمل بلا أخطاء وتعطي نتائج تبدو معقولة، ولهذا هي أخطر من الخطأ الذي يوقف التشغيل.

قاعدة: الحالة تُمرَّر، لا تُشار

«تُمرَّر» تعني أنها في التوقيع؛ و«لا تُشار» أنه لا مكانين يملكان النسخة نفسها. القاعدة شرط هندسي: الحالة المُشار إليها تجعل السلوك يعتمد على ترتيب التنفيذ، وهذا الترتيب غير مضمون على GPU.

الاستراتيجية	أين تعيش	مع `jit`؟	الثمن
متغيّر عام	خارج الدالّة	لا	قيمة كاذبة بلا خطأ
إغلاق على كائن	في خلية الدالّة	لا	العيب نفسه، وأخفى
قاموس يُمرَّر ويُعاد	عند المُستدعي	نعم	توقيع أطول ونسخ يدوي

الاستراتيجية	أين تعيش	مع `jit`؟	الثمن
متغيّرات Linen	قاموس `variables`	نعم	فصل صريح بين الأوعية
كائن NNX	داخل الكائن	نعم	`split` عند الحدود

جدول 9.1 — خمس استراتيجيات للحالة، وثمن كل واحدة

تمرين الفصل التاسع

1. عرّف `Pair` كـ `dataclass` وسجّله في pytree، ثم اجمع نسختين منه بـ `jax.tree.map`. ماذا يحدث قبل التسجيل؟
2. حوّل عدّاداً عالمياً إلى حالة مُمرَّرة، وشغّل خمس خطوات بمدخل ثابت يساوي 2.0.
3. اشرح لماذا لا يصحّ وضع إحصاءات BatchNorm في `params`.

الحلّ النموذجي

1. قبل التسجيل يعامل JAX الكائن كورقة واحدة لا يفتحها، فيصل الكائن نفسه إلى الدالّة ويُرفض عند التجميع لأنه ليس مصفوفة.

```python
import jax, jax.numpy as jnp
from dataclasses import dataclass

@dataclass(frozen=True)
class Pair:
    a: jax.Array
    b: jax.Array

jax.tree_util.register_pytree_node(
    Pair,
    lambda p: ((p.a, p.b), None),
    lambda aux, ch: Pair(*ch))

p = Pair(jnp.ones(2), jnp.arange(2.0))
print(jax.tree.map(lambda t: t * 2, p))
```

2. الحالة زوج (عدّاد، مجموع)، والناتج بعد خمس خطوات `(5, 10.0)`: يزيد العدّاد خمساً، ويتراكم المجموع 5×2.0.

3. لأن المُحسّن يرى كل ما في `params` قابلاً للتعلّم: يطبّق عليه تناقص الأوزان ويحسب له عزماً. والإحصاءات نتيجة حساب لا معامل؛ تطبيق التناقص عليها يزيحها نحو الصفر بلا سبب ويشوّه التطبيع في الاستدلال.

خلاصة الفصل

- الدالّة المجمّعة تُقرأ مرّة وتُشغَّل كثيراً؛ الأثر الجانبي يحدث في القراءة.
- pytree = قشرة + أوراق؛ القشرة تدخل في مفتاح الترجمة، والأوراق هي البيانات.
- سجّل نوعك بـ `register_pytree_node` : أبناء في الأول، وثوابت في الثاني.
- الحالة تُمرَّر كمدخل وتُعاد كمخرج؛ هذا ما يجعل الخطوة قابلة للحفظ والتوزيع.
- Flax يفصل `params` عن `batch_stats` ؛ وNNX يعيد الحالة إلى الكائن مع `split` .

مصطلحات الفصل

purity (النقاء)

خاصية دالّة يعتمد خرجها على مدخلاتها وحدها، بلا أثر جانبي.

pytree

بنية متداخلة من حاويات معروفة وأوراق؛ الوحدة التي يفهمها JAX في المعاملات.

leaf (ورقة)

العنصر الذي يصل إلى العمليات الحسابية: مصفوفة أو عدد، لا حاوية.

treedef

وصف القشرة بلا قيم؛ قابل للتجزئة ويُستخدم مفتاحاً في ذاكرة الترجمة.

aux_data

بيانات ثابتة تُرفق بالشجرة لإعادة بناء الكائن، ولا تُعدّ من الأوراق.

random: العشوائية الصريحة

المفاتيح، split، fold_in، وفلسفة التكرار الحتمي

سأل مهندسان في الفريق نفسه السؤال ذاته: «هل نتائجك قابلة للتكرار؟» قال الأول نعم، وأرسل سكربتاً يعمل ويطبع الأرقام نفسها في كل مرّة. وقال الثاني نعم، وأرسل سكربتاً يطبع أرقاماً مختلفة في كل تشغيل. الاثنان يستخدمان JAX، والاثنان وضعا البذرة نفسها. الفرق أن أحدهما كان يسحب عشوائيته من مولّد بايثون العام، والآخر من مفتاح يمرّره بيده. هذا الفصل عن ذلك المفتاح: لماذا ليس رقماً، ولماذا لا يُعاد استعماله، وكيف تبني منه شجرةً تكفي تدريباً كاملاً.

لماذا لا يوجد np.random.seed في JAX؟

في NumPy، حين تنادي `np.random.seed(0)`، فإنك تضع رقماً في مكان خفيّ داخل المكتبة. كل استدعاء لاحق يقرأ ذلك المكان ويعدّله ويقدّم الحالة خطوة. النتيجة أن `np.random.normal()` بدالّة بلا وسائط تعطي أرقاماً مختلفة في كل مرّة: الخرج يعتمد على تاريخ الاستدعاءات، لا على المدخلات.

وهذا ما يرفضه النقاء (الفصل 9). والأسوأ مع ثمانية أجهزة: أيّها يملك ذلك العدّاد؟ إمّا عدّاد مشترك يتنازع عليه الجميع، أو ثمانية عدّادات تعطي ثمانية تيارات. الحلّان سيئان.

```python
import jax, numpy as np, jax.numpy as jnp

@jax.jit
def noise():
    return jnp.asarray(np.random.normal(size=3))

print(noise())
print(noise())      # نفس الأرقام تماماً
```

‹‹ المخرجات ››

```
[ 0.49671414 -0.1382643   0.64768857]
[ 0.49671414 -0.1382643   0.64768857]
```

✕ عشوائية تجمّدت في الرسم

السطر `np.random.normal` نُفّذ مرّة واحدة عند التتبّع، فصار ثابتاً مدفوناً في البرنامج المجمّع. لا خطأ، ولا تحذير: فقط ضجيجٌ لا يتغيّر، ونموذج يتعلّم على الدفعة نفسها في كل خطوة. اعتبر أي استدعاء لمولّد بايثون داخل دالّة مجمّعة خطأً حتى تثبت العكس.

لو وضعت البذرة مرّة واحدة قبل حلقة التدريب، ثم أضفت جهازاً ثانياً، فماذا يحدث للعشوائية على الجهازين؟ وأيّ جزء من البرنامج كان سيقرّر ذلك؟

المفتاح ليس رقماً

`jax.random.key(0)` لا يعيد عدداً، بل مصفوفة صغيرة من نوع خاص: `key<fry>` بشكل `()`. النوع نفسه يمنعك من جمع مفتاحين أو ضربهما في عدد: فالمفتاح ليس قيمة حسابية، بل بطاقة هوية لمولّد.

```python
import jax

k = jax.random.key(0)
print(k.dtype, k.shape)          # key ()
print(jax.random.key_data(k))    # uint32[2]

try:
    k + 1
except TypeError as exc:
    print("لا حساب على المفاتيح:", exc)
```

داخل كل مفتاح مادّة خام من 64 بت؛ والبذرة ليست «الرقم صفر» بل مادة أوّلية يُشتقّ منها تيار كامل. والمولّد – Threefry – عدّاد: يأخذ مفتاحاً وموضعاً، فيعيد أرقاماً بلا حالة محفوظة.

fold_ing split

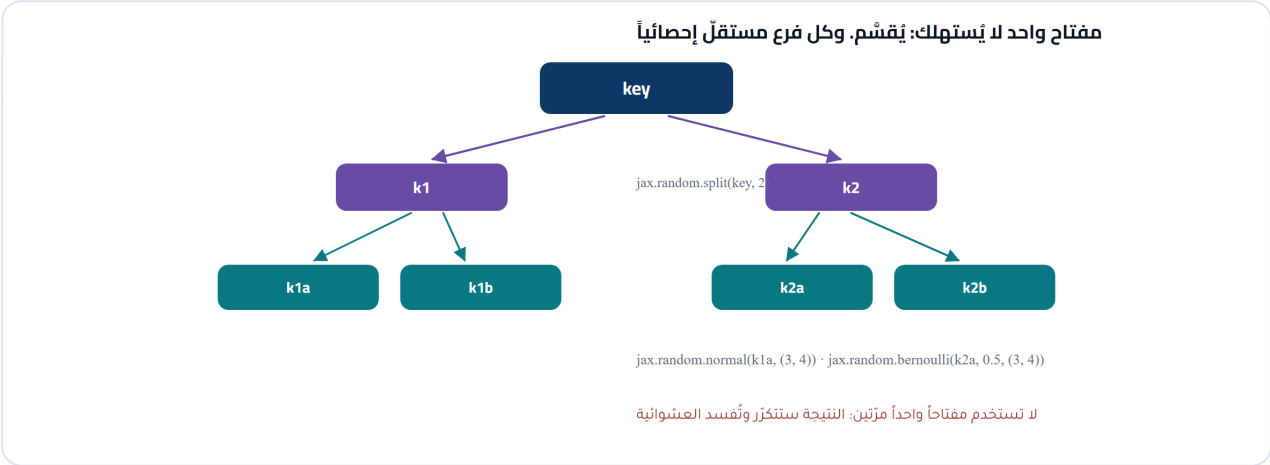
لديك مفتاح واحد، وتحتاج ألف مفتاح مستقلّ. الطريقة الأولى `jax.random.split`: تستهلك مفتاحاً وتنتج مفتاحين – أو `n` مفاتيح – لا علاقة إحصائية بينها. الطريقة الثانية `jax.random.fold_in`: تدمج عدداً صحيحاً في مفتاح فتحصل على مفتاح جديد، دون أن تستهلك الأصل.

```python
import jax

key = jax.random.key(42)
k1, k2 = jax.random.split(key)          # مفتاحان من مفتاح
ka, kb, kc = jax.random.split(k1, 3)    # ثلاثة من الأول

print(jax.random.normal(k1, (2,)))
print(jax.random.normal(k2, (2,)))
print(jax.random.normal(jax.random.fold_in(key, 7), (2,)))
```

تصوّر الشجرة: الجذر مفتاح التدريب، وتحته فروع للتهيئة والتدريب والتقييم، وتحت التدريب فرع لكل خطوة. القاعدة: كل عقدة تُشتقّ من أب أعلى، ولا تُستخدم مرّتين لغرضين.



الشكل 10.1 — شجرة المفاتيح: كل مسار من الجذر إلى الورقة يعطي تياراً مستقلاً، ولا يُسلك المسار نفسه مرّتين.

الوجه	`split`	`fold_in`
المدخل	مفتاح وعدد فروع	مفتاح وعدد صحيح
الأصل	يُستهلك، فلا تُكمل به	يبقى صالحاً للاستعمال
الاستعمال	فروع متوازية لا يعرف بعضها بعضاً	خطوة أو جهاز أو طبقة
الخطر	نسيان استعمال الأصل	تكرار العدد نفسه

جدول 10.1 — `split` مقابل `fold_in`

الأخطاء القاتلة: إعادة استخدام المفتاح

الخطأ الأكثر شيوعاً ليس خطأً في الصياغة، بل في التصوّر: يظنّ كثيرون أن كل استدعاء «يستهلك» المفتاح، كما كانت البذرة تُستهلك في NumPy. في JAX المفتاح قيمة ثابتة: استدعاؤه مرّتين يعطي النتيجة نفسها مرّتين.

في مشروع فيه مئة ملف، كيف تعرف أن مفتاحاً واحداً لم يُستخدم مرّتين لغرضين مختلفين؟ ؟

Python

```
key = jax.random.key(7)

a = jax.random.normal(key, (3,))
b = jax.random.normal(key, (3,))
print(jnp.allclose(a, b))       # True

k1, k2 = jax.random.split(key)
c = jax.random.normal(k1, (3,))
d = jax.random.normal(k2, (3,))
print(jnp.allclose(c, d))       # False
```

‹‹ المخرجات ››

```
True
False
```

⚠ التسريب الصامت

إعادة استخدام مفتاح في تهيئة طبقتين تعني أن الأوزان تبدأ متطابقة، فتكسر التماثل الذي وُجدت العشوائية لأجله. ولن ترى خطأً: الخسارة تنزل، والنموذج يتدرّب، والنتيجة أضعف قليلاً فقط. هذا أسوأ من الانهيار، لأنه لا يترك أثراً تبحث عنه.

التوزيعات الشائعة

كل دالّة توليد في `jax.random` تأخذ مفتاحاً أوّلاً، ثم الشكل. هذا الترتيب ليس ذوقاً: المفتاح هو المدخل الذي يجعل الدالّة نقية، والشكل هو ما يحدّد بنية الخرج.

```python
k1, k2, k3, k4 = jax.random.split(jax.random.key(0), 4)

print(jax.random.uniform(k1, (2, 3), minval=-1.0, maxval=1.0).shape)
print(jax.random.bernoulli(k2, 0.7, (5,)).dtype)
print(jax.random.categorical(k3, jnp.zeros((4, 5))).shape)
print(jax.random.permutation(k4, jnp.arange(6)))
```

‹‹ المخرجات ››

```
(2, 3)
bool
(4,)
[3 0 5 2 1 4]
```

لاحظ `categorical`: تأخذ لوغاريتمات بشكل `(4, 5)` وتعيد أربعة أرقام، واحداً لكل صفّ. و `permutation` تبديلٌ كامل لا اختيار؛ وللاختيار مع التكرار استخدم `jax.random.choice` مع `replace=True`.

العشوائية داخل jit وvmap

بما أن المفتاح مصفوفة، فهو مدخل كأي مدخل: يُمرَّر إلى الدالّة المجمّعة كوسيط، ويُشتقّ منه داخل الرسم. أما في `vmap` فتمرّر مصفوفة مفاتيح، فيصبح لكل عنصر في الدفعة تياره الخاص.

```python
def dropout(key, x, rate):
    keep = jax.random.bernoulli(key, 1.0 - rate, x.shape)
    return jnp.where(keep, x / (1.0 - rate), 0.0)

@jax.jit
def train_step(key, x, step):
    k = jax.random.fold_in(key, step)      # مفتاح جديد لكل خطوة
    return dropout(k, x, 0.2)

keys = jax.random.split(jax.random.key(0), 8)
masks = jax.vmap(lambda k: jax.random.bernoulli(k, 0.5, (4,)))(keys)
print(masks.shape)      # (8, 4)
```

✓ **الخطوة في المفتاح، لا في المتغيّر**

مرّر رقم الخطوة إلى `fold_in` بدل أن تحمل مفتاحاً متغيّراً في حلقة بايثون. بهذا تصبح الخطوة 900 قابلة لإعادة الإنتاج وحدها، حتى لو بدأت من نقطة حفظ في منتصف التدريب، وحتى لو أعدت ترتيب الخطوات على عدّة أجهزة.

التكرار الحتمي عبر الأجهزة

السؤال الذي يقلق الفرق: هل يعطي المفتاح نفسه الأرقام نفسها على CPU وGPU؟ الجواب: نعم، ما دامت الخوارزمية نفسها والشكل نفسه ونوع البيانات نفسه. المولّد حسابٌ صحيح على 32 بت، لا عملية عائمة تعتمد على ترتيب الجمع، ولهذا لا يتأثّر باختلاف العتاد.

لكن الحتمية أضيق مما تظنّ: تتغيّر النتائج إذا غيّرت إصدار `jax`، أو انتقلت إلى مولّد آخر مثل `rbg`، أو فعّلت `jax_enable_x64`. القاعدة: المفتاح يضمن التكرار داخل البيئة نفسها، لا عبر كل البيئات.

★ **ثلاثة أشياء تُخزَّن مع النموذج**

الأوزان، وحالة المُحسّن، ومفتاح التدريب. الثالث يُنسى غالباً، فتستعيد نموذجاً لا يمكن متابعة تدريبه بنتائج قابلة للتكرار. خزّن المفتاح كما تخزّن الأوزان، أو خزّن البذرة ورقم الخطوة واشتقّ المفتاح منهما.

تمرين الفصل العاشر

1. أنتج أربع أقنعة dropout لأربع خطوات من مفتاح واحد، وتحقّق من أنها مختلفة.
2. اشرح لماذا يعطي استدعاء `jax.random.normal` مرّتين بالمفتاح نفسه النتيجة نفسها، وكيف يحلّ `split` المشكلة.
3. ابنِ جدول مفاتيح لخطوتين وجهازين باستخدام `fold_in`، وأثبت أن التشغيل مرّتين يعطي الأرقام نفسها.

الحلّ النموذجي

1. نشتقّ مفتاحاً لكل خطوة بـ `fold_in`، ثم نقارن الأقنعة.

```python
import jax, jax.numpy as jnp

root = jax.random.key(0)
masks = [jax.random.bernoulli(jax.random.fold_in(root, s), 0.5, (6,))
         for s in range(4)]
print(jnp.stack(masks).astype(jnp.int32))
print(len({tuple(m.tolist()) for m in masks}))   # 4 أقنعة مختلفة
```

2. لأن المفتاح قيمة ثابتة لا حالة متغيّرة: الدالّة دالّة في (المفتاح، الشكل) وحدها، فالمدخل نفسه يعطي الخرج نفسه. و `split` يشتقّ مفتاحين مختلفين من الأصل، فيصبح لكل استعمال مدخل مختلف.

3. ندمج رقم الجهاز ورقم الخطوة في مفتاح واحد، فيصبح المسار فريداً:

```python
def step_key(root, device, step):
    return jax.random.fold_in(jax.random.fold_in(root, device), step)

root = jax.random.key(1234)
run1 = [float(jax.random.normal(step_key(root, d, s), ()).sum())
        for d in range(2) for s in range(2)]
run2 = [float(jax.random.normal(step_key(root, d, s), ()).sum())
        for d in range(2) for s in range(2)]
print(run1 == run2)      # True
```

خلاصة الفصل

- لا مولّد عام في JAX: الحالة الخفية تكسر النقاء وتستحيل مع التقطيع.
- المفتاح مصفوفة من نوع `key<fry>`، لا عدداً؛ ولا يُحسب عليه.
- `split` يشتقّ فروعاً ويستهلك الأصل، و `fold_in` يدمج رقماً ويُبقي الأصل.
- إعادة استخدام مفتاح تعطي أرقاماً متطابقة بصمت – وهي أشهر أخطاء الفصل.
- المفتاح يُمرَّر إلى `jit` و `vmap` كأي مدخل، ويُخزَّن مع الأوزان.

مصطلحات الفصل

PRNG key

مصفوفة صغيرة تحمل مادّة المولّد؛ تُمرَّر صراحةً بدل بذرة عامة.

split

اشتقاق مفتاحين أو أكثر من مفتاح أب، باستهلاك الأصل.

fold_in

دمج عدد صحيح في مفتاح لإنتاج مفتاح جديد مع إبقاء الأصل صالحاً.

key<fry>

نوع بيانات المفتاح في JAX 0.12، ويمنع الحساب المباشر على المفاتيح.

threefry

خوارزمية المولّد الافتراضية: عدّاد مع مفتاح، بلا حالة محفوظة.

من pmap إلى shard_map: التوازي

تاريخ التوازي في JAX، والطريق الحديث إلى عدّة أجهزة

في 2018 كان أمامك طريق واحد لتشغيل حساب على عدّة أجهزة: `pmap`. وفي 2021 جاء `pjit` ومعه GSPMD، فتغيّر السؤال من «كيف أوزّع الدفعة؟» إلى «كيف أصف التقطيع؟». وفي 2023 جاء `shard_map` ليعيد لك السيطرة على ما يحدث داخل الجهاز. ثلاث أدوات في خمس سنوات، وثلاثة نماذج ذهنية. هذا الفصل يحكي القصة، ثم يعلّمك أيّها تختار اليوم.

ثلاثة أجيال في خمس سنوات

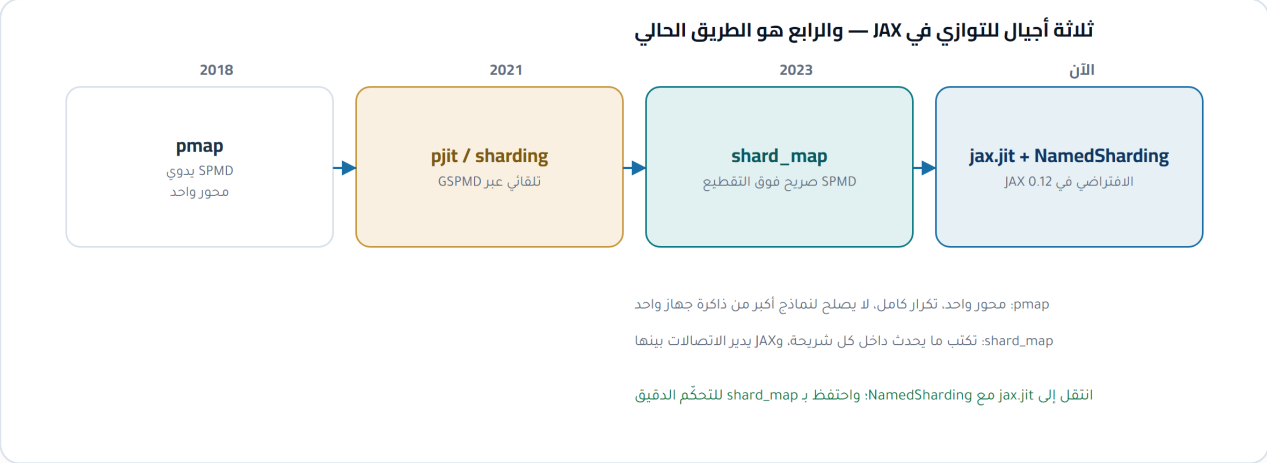
الجيل الأول، `pmap`، كان بسيطاً كفكرة: خذ دالّة وطبّقها على المحور الأول، جهازاً لكل شريحة. والثاني نقل العبء إلى المترجم: تصف التقطيع، ويتولّى GSPMD إدخال الاتصالات. والثالث أعاد ما يحدث داخل الشريحة إلى يدك.

أما اليوم، فالمسار الافتراضي هو `jax.jit` مع `NamedSharding`: تكتب الدالّة كما تكتبها على جهاز واحد، وتصف التقطيع في الحدود. و`jax.experimental.pjit` لم يبقَ واجهةً منفصلة، و`pmap` صار قديماً يُذكر في التاريخ لا في شيفرة جديدة.

SPMD بلغة بسيطة

SPMD تعني «برنامج واحد، بيانات متعدّدة»: الأجهزة الثمانية تشغّل البرنامج نفسه، وكل واحد يرى شريحة مختلفة. لا تكتب ثمانية برامج؛ تكتب واحداً وتصف التقطيع.

والمصفوفة التي تراها **شكلها عالمي**: `(8, 1024)` في نظرك، بينما يحمل كل جهاز `(1, 1024)`. والاتصال (collective) هو ما يعبر الحدود بين الأجهزة.



الشكل 11.1 — الشكل عالمي في نظرك، والشظايا محلية، والاتصال هو الجسر بين المستويين.

> ★ **الشكل عالمي، والبيانات محلية**
>
> كل ما تكتبه خارج `shard_map` يرى الشكل الكامل. وكل ما تكتبه داخله يرى شكل الشريحة. الخلط بين المستويين هو مصدر أكثر أخطاء التوزيع إزعاجاً.

pmap: لماذا لم يعد الخيار؟

`pmap` يطبّق الدالّة على المحور الأول فقط، ويعامل كل وسيط آخر بالتكرار الكامل. مقبول في دفعة بسيطة، وكارثي في نموذج كبير: تكرّر الأوزان ثماني مرّات.

ثم يفرض التفكير بمحور الدفعة دائماً، ويمنع التركيب الحرّ مع تقطيعات أخرى. ولهذا صار قديماً عملياً: اقرأه كتاريخ، ولا تبنِ عليه نظاماً جديداً.

> لماذا يحتاج `pmap` إلى محور أول دائماً، بينما `NamedSharding` لا يحتاج إلى أيّ محور؟ ؟

NamedShardingg mesh

الmesh شبكة مُسمّاة من الأجهزة: تقول إن لديك ثمانية أجهزة مرتّبة على محور اسمه `"x"`، أو شبكة `2 × 4` بمحورين. ثم تصف تقطيع كل مصفوفة بـ `PartitionSpec`.

```python
import jax, jax.numpy as jnp
from jax.sharding import NamedSharding, PartitionSpec as P

devices = jax.devices()                          # ثمانية أجهزة على مضيف واحد
assert len(devices) == 8
mesh = jax.make_mesh((8,), ("x",), devices=devices)
sharding = NamedSharding(mesh, P("x"))

x = jax.device_put(jnp.arange(64.0).reshape(8, 8), sharding)
print(x.sharding)                                # P('x',)
print(x.addressable_shards[0].data.shape)        # (1, 8)

@jax.jit
def col_sum(a):
    return a.sum(axis=0)

print(col_sum(x).shape)                          # (8,) على محور مكرّر
```

و `P("x", None)` تعني: قسّم البُعد الأول على `"x"`، وكرّر الثاني. وللتحكّم الدقيق في الحدود تمرّر `in_shardings` و `out_shardings` إلى `jax.jit`.

الوصف	المعنى	شكل الشريحة
`P("x")`	البُعد الأول على `x`	مقسوم على حجم المحور
`P("x", None)`	الأول مقطّع، والثاني مكرّر	مقسوم في الأول فقط

جدول 11.1 — قراءة `PartitionSpec`

shard_map: تكتب داخل الشريحة

`shard_map` يقلب المنظور: الدالّة التي تكتبها تُنفَّذ على جهاز واحد، وترى المصفوفة بشكلها المحلي. أنت من يكتب الاتصال صراحةً بـ `jax.lax.psum` أو `all_gather`.

```python
from functools import partial
from jax.experimental.shard_map import shard_map

@partial(shard_map, mesh=mesh, in_specs=(P("x"),), out_specs=P("x"))
def local_center(x):
    return x - jax.lax.pmean(x.mean(), "x")

y = local_center(x)
print(y.shape)      # (8, 8) عالمي
```

داخل الدالّة لا يوجد mesh، بل مصفوفة محلية وأسماء محاور للاتصال. ولهذا هو مثالي حين تريد أن تعرف بالضبط أيّ اتصال يحدث، وهو ما يخفيه المقطّع التلقائي.

> ✓ **متى تنزل إلى shard_map؟**
>
> حين تخالف خطّة التقطيع التلقائية ما تريد: توازٍ على المحاور، أو اتصال مخصّص، أو خوارزمية موزّعة تكتبها بنفسك. أما التقطيع العادي للدفعات والأوزان فدعه لـ `NamedSharding`.

الاتصالات: all-reduce، all-gather، reduce-scatter

كل اتصال يجيب عن سؤال مختلف. `psum` يجمع القيم عبر المحور ويعيد المجموع للجميع، وهو ما يحدث في مزامنة التدرّجات. و`all_gather` يعيد لكل جهاز القيم كلها، فتنمو الذاكرة بعدد الأجهزة.

وreduce-scatter يجمع ثم يوزّع: كل جهاز يحصل على جزء من الناتج. وهو ما يستخدمه XLA تحت الغطاء، ولهذا لا تكتبه عادةً بل تتركه للمترجم.

الاتصال	ما يفعله	أين يظهر
`psum`	جمع عبر المحور، والنتيجة للجميع	مزامنة التدرّجات
`pmean`	متوسّط عبر المحور	تطبيع، خسارة موزّعة
`all_gather`	تجميع الشظايا عند الجميع	طبقة تحتاج المدخل كاملاً
reduce-scatter	جمع ثم توزيع الأجزاء	داخل all-reduce الكبير
all-to-all	تبادل غير متجانٍ بين الأجهزة	خبراء، انتباه موزّع

جدول 11.2 — الاتصالات ومتى تظهر

لو قسّمت الأوزان على ثمانية أجهزة ثم احتجت في طبقة ما إلى المصفوفة كاملة، فما ثمن `all_gather`؟ وهل يبقى التوفير في الذاكرة؟ ؟

أيّ نمط أختار؟

الاختيار يبدأ من سؤال: ما الذي لا يتّسع في جهاز واحد؟ إن كانت الدفعة فالتوازي على البيانات، وإن كانت الأوزان فالتوازي على النموذج، وإن كان الاثنان فالتقطيع المُجزَّأ.

النمط	ما يُقطَّع	الاتصال الغالب	متى تختاره
بيانات	الدفعة	`psum` على التدرّجات	النموذج يتّسع في جهاز
موتر	الأوزان على محور داخلي	`all_gather`	طبقة أوسع من ذاكرة الجهاز
FSDP	الأوزان، مع جمع عند الحاجة	all-gather ثم reduce-scatter	نموذج أكبر من الذاكرة
خط أنابيب	الطبقات على مراحل	إرسال بين المراحل	عمق كبير ودفعات صغيرة

جدول 11.3 — أنماط التوزيع وحدودها

لاحظ أن الأنماط تُركَّب: يمكنك تقطيع الدفعة على محور والأوزان على آخر، في شبكة `(4, 2)`. وكلّما زادت المحاور زادت الاتصالات.

أخطاء التوزيع الشائعة

الخطأ الأول هو التكرار الصامت: تنسى التقطيع، فتُوضع المصفوفة كاملة على كل جهاز. لا خطأ، لكن الذاكرة تُستهلك ثماني مرّات. افحص `x.sharding` بعد كل `device_put`.

والثاني ترتيب المحاور: `P("x", "y")` مع mesh بشكل `(2, 4)` يقسم البُعد الأول على محور حجمه اثنان. والثالث القابلية للقسمة: شكل لا يقبل القسمة يُرفض عند الترجمة، والحلّ حشو.

× تقطيع يُلغى بصمت

حين تعيد عملية ما تشكيل المصفوفة (`reshape` أو `transpose`) قد يُعيد المترجم توزيعها على محور مختلف، فتصبح الحدود اتصالات لم تخطّط لها. اطبع `y.sharding` عند كل حدّ تشكّ فيه.

> ⚠ **لا تفترض أن التوزيع يعمل لأنه يعمل**
>
> برنامج موزّع يعطي النتيجة الصحيحة قد يكون أبطأ من نسخة الجهاز الواحد، لأن كل الحدود صارت اتصالات. قِس زمن الخطوة قبل التوزيع وبعده؛ إن لم ينقص، فالتقطيع لا يشتري شيئاً.

تمرين الفصل الحادي عشر

1. أنشئ mesh بشكل `(4, 2)` بمحورين، وقطّع مصفوفة `(8, 16)` على المحور الأول فقط. ما شكل الشريحة؟
2. اكتب دالّة `shard_map` تحسب مجموع كل الأجهزة بـ `psum`، وقارن الناتج بـ `jnp.sum` العادية.
3. اذكر خطأين يجعلان التوزيع «يعمل» بلا فائدة، وكيف تكتشف كل واحد منهما.

الحلّ النموذجي

1. الشريحة `(2, 16)`: البُعد الأول مقسوم على أربعة، والثاني مكرّر بالكامل على المحور `"y"`.

```python
import jax, jax.numpy as jnp
from jax.sharding import NamedSharding, PartitionSpec as P

mesh = jax.make_mesh((4, 2), ("data", "model"))
sh = NamedSharding(mesh, P("data", None))
w = jax.device_put(jnp.ones((8, 16)), sh)
print(w.addressable_shards[0].data.shape)   # (2, 16)
```

2. الدالّة المحلية ترى شريحة، و `psum` يجمعها عبر المحور:

```python
from functools import partial
from jax.experimental.shard_map import shard_map

m1 = jax.make_mesh((8,), ("x",))
f = partial(shard_map, mesh=m1, in_specs=(P("x"),), out_specs=P())
total = f(lambda a: jax.lax.psum(a.sum(), "x"))
print(float(total(jnp.ones((8, 4)))))   # 32.0
```

3. التكرار الصامت: يُكشف بـ `x.sharding`. وترتيب المحاور الخاطئ: يُكشف بمقارنة شكل الشريحة المتوقّع مع `addressable_shards`.

خلاصة الفصل

- ثلاثة أجيال: `pmap` ثم `pjit` ثم `shard_map`؛ والافتراضي اليوم `jax.jit` مع `NamedSharding`.
- SPMD: برنامج واحد على كل جهاز، والشكل عالمي والبيانات محلية.
- `pmap` قديم: محور أول فقط، وتكرار كامل لبقية الوسائط.
- `shard_map` يكتب داخل الشريحة، فتصبح الاتصالات صريحة.
- اختر النمط بسؤال: ما الذي لا يتّسع؟

مصطلحات الفصل

SPMD

برنامج واحد يُنفَّذ على عدّة أجهزة، كلٌّ على شريحة مختلفة من البيانات.

mesh

شبكة مُسمّاة من الأجهزة ذات محاور مثل `"data"` و `"model"`.

PartitionSpec

وصف تقطيع مصفوفة: اسم محور لكل بُعد، أو `None` للتكرار.

collective

عملية تعبر الأجهزة: جمع، أو تجميع، أو توزيع قيم بينها.

shard_map

تحويل يكتب دالّة بمستوى الشريحة، مع اتصالات صريحة وحدود معلنة.

تركيب التبديلات: القواعد الذهبية

jit(grad(vmap(f))): كيف تتركّب، وأين تنكسر، وكيف تتجنّب ذلك

سألك زميل: «هل أضع `vmap` داخل `jit` أم العكس؟». أجبت بسؤال: «ماذا تريد أن تُجمّع؟». لم يفهم. كان يفكّر في التحويلات كأزرار في لوحة تحكّم، وليست كذلك: كل تحويل في JAX دالّة تأخذ دالّة وتُرجع دالّة. وحين تدرك هذا، يتحوّل السؤال من «هل يجوز؟» إلى «ما الذي يحدث في كل ترتيب، وبأيّ ثمن؟». هذا الفصل عن ذلك الثمن.

التركيب ليس ميزة إضافية، بل الخاصية الأساسية

تخيّل مكتبة جعلت `grad` طريقةً على كائن: `model.grad()`. الآن أضف `vmap`: هل `model.vmap().grad()` أم `model.grad().vmap()`؟ كل توليفة تحتاج كائناً ورمزاً جديدين. هكذا تنهار المكتبات عند التركيب.

JAX اختارت العكس: التحويل دالّة من نوع «دالّة ← دالّة». لا شيء منها يعرف شيئاً عن الآخر، ومخرَج كلٍّ منها صالح كمدخَل لأيّ منها، فالتركيب مجّاني بالبناء لا بمجهود الفريق.

لكن «مجّاني» تعني «صحيح»، لا «رخيص». كل تحويل يرى ما فوقه صندوقاً أسود: حين يرى `jit` داخل `vmap` دالّةً تُستدعى مرّة واحدة يجمّعها مرّة واحدة، ولا يعلم أن `vmap` سينسخها على كل عنصر. من هنا يأتي فرق التكلفة.

> ★ **سؤال يُغني عن حفظ الحالات**
>
> أيّ تحويل يرى الدالّة كلها، وأيّها يراها عنصراً واحداً؟ الترتيب يحدّد ما يدخل المترجم، وحجمه يحدّد عدد مرّات التجميع.

خريطة التركيبات

ثمانية تركيبات تغطّي معظم ما ستكتبه، وثمنها:

التركيب	الحكم	ملاحظة التكلفة
`jit(grad(f))`	يعمل، وهو افتراضي التدريب	تجميع واحد لدالّة التدرّج
`grad(jit(f))`	يعمل، لكن بلا فائدة	التجميع داخل التفريق شفّاف
`vmap(jit(f))`	يعمل، والأفضل لأحجام متغيّرة	يجمّع `f` مرّة لكل حجم دفعة
`jit(vmap(f))`	يعمل، والأسرع لدفعة ثابتة	يُعاد التجميع عند كل حجم جديد
`grad(vmap(f))`	يفشل غالباً	`grad` يطلب قياسياً
`vmap(grad(f))`	يعمل، وهو تدرّج كل مثال	ذاكرة = حجم الدفعة × المعاملات
`jit(grad(vmap(f)))`	يعمل، ونمط الخسارة الكلية	الجمع داخل التجميع: الأرخص
`shard_map(vmap(f))`	يعمل، ونمط التقطيع اليدوي	`vmap` داخل الجهاز و`shard_map` عبره

جدول 12.1 — ثمانية تركيبات شائعة، وحكم كلٍّ منها وثمنه



الشكل 12.1 — كل حلقة تحويل تلفّ ما داخلها؛ الخارجي يرى شكلاً واحداً، والداخلي يرى ما فُرض عليه.

في الجدول سطران يعملان وأحدهما «بلا فائدة». هل الخطأ في الدالّة، أم في موضع الحدّ بين التفسير والتنفيذ؟ ؟

الترتيب يغيّر التكلفة لا النتيجة

عدّاد يطبع سطراً عند كل تجميع:

```python
import jax
import jax.numpy as jnp

jax.config.update("jax_log_compiles", True)  # سطر عند كل تجميع

def f(x):
    return jnp.tanh(x) @ jnp.ones((4, 1))

A = jax.jit(jax.vmap(f))   # الدفعة كلها داخل التجميع
B = jax.vmap(jax.jit(f))   # وحدها داخل التجميع f

for n in (8, 16, 32):
    x = jnp.ones((n, 4))
    print("A", n, A(x).shape, "B", n, B(x).shape)
```

‹‹ المخرجات ››

```
Compiling vmap_f ...
A 8 (8, 1) B 8 (8, 1)
Compiling vmap_f ...
A 16 (16, 1) B 16 (16, 1)
Compiling vmap_f ...
A 32 (32, 1) B 32 (32, 1)
Compiling f ...
B 8 (8, 1) B 16 (16, 1) B 32 (32, 1)
```

الأرقام متطابقة، فالنسخ عملية رياضية لا تعتمد على ترتيب التنفيذ. لكن `A` جُمّع ثلاث مرّات: الشكل الذي رآه المترجم تغيّر ثلاث مرّات. أما `B` فجُمّع مرّة لشكل `(4,)`، ثم لفّ `vmap` حول الناتج.

الفرق لا يظهر في اختبار الوحدة بل في الإنتاج: خادم بدفعات متغيّرة سيعيد التجميع عند كل حجم.

> ✓ **ثبّت الشكل بدل أن تثبّت الترتيب**
>
> احشُ الدفعة إلى مضاعفات `8` بقناع، أو استخدم `static_argnums` . حين تستقرّ الأشكال يتكافأ السؤالان.

الحالة داخل التركيب

لماذا يفشل `grad` أحياناً داخل `vmap` ؟ لأنه يحتاج قياسياً، و `vmap` يطلب شيئاً لكل مثال. الحلّ ليس حيلة: `has_aux=True` يجعل `grad` يُرجع (التدرّج، الملحق)، فيوزّع `vmap` الاثنين.

```python
import jax
import jax.numpy as jnp

def loss(params, x, y):
    logits = x @ params["w"] + params["b"]
    l = jnp.mean((logits - y) ** 2)
    return l, logits          # الخسارة + ملحق

params = {"w": jnp.zeros((4, 1)), "b": jnp.zeros(1)}
x, y = jnp.ones((2, 4)), jnp.ones((2, 1))

per_example = jax.vmap(jax.grad(loss, has_aux=True), in_axes=(None, 0, 0))
grads, logits = per_example(params, x, y)
print(jax.tree.map(lambda g: g.shape, grads))
print(logits.shape)
```

‹‹ المخرجات ››

```
{'b': (2, 1), 'w': (2, 4, 1)}
(2, 2, 1)
```

بُعد الدفعة في المقدّمة: تدرّج لكل مثال، والملحق نُسخ أيضاً.

أمّا الحالة القابلة للتعديل فقصة أخرى. `vmap` ينسخ جسم الدالّة على كل عنصر، فإن كتبت نسخةٌ في مصفوفة مشتركة كتبت كلّها فيها بلا ترتيب محدّد. في Linen يظهر هذا حين تستدعي `apply` مع `mutable=["batch_stats"]` داخل `vmap`.

> **× خطأ شائع: قراءة التحديث كأنه جاهز**
>
> تحديثات `batch_stats` تعود بشكل `(B, ...)`، وصيغة الزخم تحتاج مجاميع لا متوسطات. اقرأ صيغة التحديث في المصدر.

> لو لففتَ دالّة تكتب في عدّاد عام بـ `vmap` على 32 عنصراً، فما قيمة العدّاد؟ وهل تُثبت الجواب دون تشغيلها؟ ؟

التبديلات المخصّصة: custom_vjp و custom_jvp

أحياناً تعرف عن دالّتك أكثر ممّا يعرف التفريق التلقائي، والصيغة التحليلية أدقّ عددياً من شجرة العمليات. هنا يعطيك `jax.custom_vjp` المرورين صراحةً:

```python
import jax
import jax.numpy as jnp

@jax.custom_vjp
def softplus(x):
    return jnp.logaddexp(0.0, x)      # أمامي مستقرّ

def softplus_fwd(x):
    y = jnp.logaddexp(0.0, x)
    return y, y                        # نُخزّن y للعكسي

def softplus_bwd(y, g):
    # d/dx softplus(x) = sigmoid(x) = 1 - exp(-softplus(x))
    return (g * (1.0 - jnp.exp(-y)),)

softplus.defvjp(softplus_fwd, softplus_bwd)

print(softplus(30.0), jax.grad(softplus)(30.0))   # 30.0 1.0
print(jax.grad(softplus)(-30.0))                  # ~1.8e-13
```

عند `x = 30` يقترب أي مقام من هيئة `1 + exp(x)` من حدّ الدقّة، أما e^{-30} فصغير.

$$\sigma(x) = 1 / (1 + e^{-x}) = 1 - e^{-\text{softplus}(x)}$$

الصيغتان متكافئتان رياضياً، ومختلفتان عددياً عند الطرفين.

و`jax.custom_jvp` مكمّل: يحدّد المشتقّة الاتجاهية في المرور الأمامي، وهو أوفر حين يكون الحساب الأمامي هو الأغلى. أما `custom_vjp` فأوفر للذاكرة لأنه لا يخزّن الوسائط الوسطية.

> **⚠ لا تُصلح الخطأ العددي بتخمين**
>
> المرور العكسي المخصّص يُستخدم كما هو؛ JAX لا تتحقّق من صحّته.
> `jax.test_util.check_grads` هو الحدّ الأدنى.

التبديلات المخصّصة للتقطيع: custom_partitioning

ماذا لو عرفتَ بنية دالّتك أكثر ممّا يعرف المترجم وأردت فرض تقطيع معيّن؟ `custom_partitioning` يفعل ذلك، وهو موضوع متقدّم بعد `shard_map`.

```python
from jax.experimental.custom_partitioning import custom_partitioning
from jax.sharding import NamedSharding, PartitionSpec as P

@custom_partitioning
def row_softmax(x):
    return jax.nn.softmax(x, axis=-1)

def _partition(mesh, arg_shapes, result_shape):
    s = NamedSharding(mesh, P(None, "tp"))
    return mesh, row_softmax, s, s   # الشبكة، الدالّة، تقطيع المدخل، تقطيع الخرج

row_softmax.def_partition(partition=_partition)
```

الدالّة تقول: «وزّع المحور الأخير على `tp` ولا تقطع الأول». الثمن أنك صرت مسؤولاً عن صحّة التقطيع: الخطأ هنا لا يظهر استثناءً بل أرقاماً خاطئة.

قواعد ذهبية

1. **`jit` في الخارج دائماً.** الاستثناء: تجميع دالّة صغيرة لإعادة استخدامها بأحجام مختلفة، وعندها `vmap(jit(f))` أذكى.
2. **لا تفرّق دالّة مجمّعة.** `grad(jit(f))` يعمل ويضاعف عمل المترجم بلا مقابل.
3. **الدالّة الداخلية تُرجع قياسياً لكل مثال** قبل `vmap(grad(...))`، وإلا فـ `has_aux=True`.
4. **لا حالة قابلة للتعديل داخل `vmap`.** كل كتابة تُنسخ والترتيب غير محدّد.
5. **كل تحويل مخصّص يحتاج اختبار تدرّج**، وإلا كان خطأً صامتاً.

تمرين الفصل الثاني عشر

1. خُذ `f(x) = jnp.sin(x) * x` واكتب `jit(vmap(f))` و `vmap(jit(f))` و `vmap(f)`، وقِس زمن أول استدعاء وثانيه.
2. غيّر حجم الدفعة بين 4 و64 وسجّل عدد التجميعات بـ `jax_log_compiles`.
3. اكتب تدرّجاً مخصّصاً لـ `log1p` بمرور عكسي $g / (1 + x)$، وتحقّق منه عند `x = 1e-8`.

الحلّ النموذجي

1 و2. `vmap(jit(f))` تجمع `f` مرّة واحدة وتبقى ثابتة مهما تغيّر الحجم، لأن ما يدخل التجميع شكله `(4,)`. أما `jit(vmap(f))` فتُعيد التجميع عند كل حجم.

3.

```python
import jax
import jax.numpy as jnp
from jax.test_util import check_grads

@jax.custom_vjp
def log1p(x):
    return jnp.log1p(x)

def fwd(x):
    return jnp.log1p(x), x

def bwd(x, g):
    return (g / (1.0 + x),)

log1p.defvjp(fwd, bwd)
check_grads(log1p, (1e-8,), order=1)
print(log1p(1e-8), jax.grad(log1p)(1e-8))
```

الخرج قريب من `1e-08` و `1.0`. وإن اشتكى `check_grads` فالسبب الفروق المحدودة لا التدرّج.

خلاصة الفصل

- كل تحويل دالّة من دالّة إلى دالّة، فالتركيب صحيح بالبناء.
- الترتيب لا يغيّر الأرقام بل عدد مرّات التجميع.
- `jit` في الخارج هو الافتراضي؛ `vmap(jit(f))` استثناء مدروس.
- `vmap(grad(f))` يحتاج خسارة قياسية لكل مثال.
- الحالة القابلة للتعديل لا تعيش داخل `vmap` .

مصطلحات الفصل

composition (تركيب)

وضع تحويل داخل آخر: مخرَج كل تحويل دالّة صالحة كمدخَل.

trace (تتبّع)

تشغيل الدالّة على قيم رمزية لبناء رسم حسابي بدل أرقام.

has_aux

وسيط في `grad` يجعل الدالّة تُرجع (القيمة، ملحق).

custom_vjp

واجهة لتحديد المرور الأمامي والعكسي يدوياً.

custom_partitioning

واجهة لفرض تقطيع دالّة على شبكة.